Package 'pharos'

August 11, 2026

Title Descriptive Statistics Graphics and Utilities

Version 0.0.0.944

Description Provides plots and graphic routines used in DescToolsX.

Depends R (>= 4.2.0)

License GPL (>= 2)

Encoding UTF-8

LinkingTo Rcpp

Roxygen list(markdown = TRUE)

Imports graphics, Rcpp, bedrock, MASS, cli, withr, stringi, methods, base64enc

Suggests testthat (>= 3.0.0), R.rsp, hexbin, aplpack, prettyunits

Config/testthat/edition 3

VignetteBuilder R.rsp

URL https://andrisignorell.github.io/pharos/, https://github.com/AndriSignorell/pharos/

BugReports https://github.com/AndriSignorell/pharos/issues

Config/roxygen2/version 8.0.0

Contents

`abcCoords` *Coordinates for Named Plot Positions*

Description

Returns the xy-coordinates and text-adjustment values for named anchor positions such as `"topleft"`, `"center"`, etc., as used by `legend`. Useful for placing text or other annotations at consistent, region-aware positions.

Usage

```
abcCoords(
  x = "topleft",
  region = c("figure", "plot", "device"),
  cex = NULL,
  inset = 0
)
```

Arguments

`x`	A character string specifying the anchor position. One of `"bottomright"`, `"bottom"`, `"bottomleft"`, `"left"`, `"topleft"`, `"top"`, `"topright"`, `"right"`, or `"center"`. Partial matching is supported.
`region`	one of `"figure"` (default), `"plot"`, or `"device"`. Determines the coordinate region used.
`cex`	character expansion factor. If `NULL` (default), the current `par("cex")` is used.
`inset`	inset distance from the boundary, specified in lines of text (character heights/widths). May be a scalar (applied to both x and y) or a length-2 vector (x inset, y inset). Default `0`.

Details

The positioning logic is adapted from legend. The inset is computed in character units via strwidth and strheight, making it robust to device resizing, font changes, and plot-range scaling.

Three regions are supported:

"plot" The inner plot area (par("usr")).

"figure" The figure region within the device, accounting for par("fig").

"device" The full device area.

Value

A list with two components:

xy A list with elements x and y giving the anchor coordinates in user space.

adj A numeric vector of length 2, suitable for the adj argument of text.

See Also

text, legend

Other graphics.layout: axTicks, axisBreak(), isValidPlotRegion(), lineToUser(), mar(), plotFacet(), spreadOut()

Examples

```
plot(x = rnorm(10), type = "n", xlab = "", ylab = "")

# coordinates only
abcCoords("bottomleft")

# place text at a named position
xy <- abcCoords("bottomleft", region = "plot")
text(x = xy$xy$x, y = xy$xy$y, labels = "My Label",
     adj = xy$adj, xpd = NA)

# all nine positions with inset
sapply(c("topleft", "top", "topright",
         "left",    "center", "right",
         "bottomleft", "bottom", "bottomright"),
  function(p) {
    xy <- abcCoords(p, region = "plot", inset = 1)
    text(x = xy$xy$x, y = xy$xy$y,
         labels = p, adj = xy$adj, xpd = NA)
  })
```

addOpacity *Add an Alpha Channel to Colors*

Description

Add transparency to colors.

Usage

```
addOpacity(col, opacity = 0.5)
```

Arguments

col	vector of valid R colors
opacity	opacity value between 0 and 1

Value

Character vector of hexadecimal colors with alpha channel.

See Also

grDevices::adjustcolor

Other color.manipulation: colToOpaque(), darken(), fade(), lighten(), mixColors()

Examples

```
op <- par(no.readonly = TRUE)

addOpacity("yellow", 0.2)
addOpacity(2, 0.5)   # red

canvas(3)
polygon(circle(x=c(-1,0,1), y=c(1,-1,1), radius=2),
        col=addOpacity(2:4, 0.4))

x <- rnorm(15000)
par(mfrow=c(1,2))
plot(x, type="p", col="blue" )
plot(x, type="p", col=addOpacity("blue", .2),
     main="Better insight with alpha channel" )

par(op)
```

arc *Arc Geometry*

Description

Create one or more circular or elliptic arcs.

Usage

```
arc(
  x = 0,
  y = 0,
  radiusX = 1,
  radiusY = radiusX,
  startAngle = 0,
  endAngle = 2 * pi,
  numPoints = 100
)
```

Arguments

x, y	coordinates of the arc centre
radiusX, radiusY	horizontal and vertical radius
startAngle, endAngle	start and end angle in radians
numPoints	number of points used to approximate the arc

Value

An object inheriting from class "arcGeometry".

See Also

Other geometry.structures: band(), bezier(), circle(), ellipse(), polygon(), regPolygon(), ring()

as.CI *Confidence Interval Objects*

Description

Converts common confidence interval representations into a standardized object of class "CI". The standardized representation removes the ambiguity between ordinary numeric data and confidence interval data.

Usage

```
as.CI(x, ...)

## S3 method for class 'matrix'
as.CI(x, ...)

## S3 method for class 'data.frame'
as.CI(x, estimate = "est", lower = "lci", upper = "uci", ...)

## S3 method for class 'list'
as.CI(x, ...)

## S3 method for class 'CI'
as.CI(x, ...)

## Default S3 method:
as.CI(x, ...)

is.CI(x)
```

Arguments

x	object to convert or, for is.CI(), object to test
...	further arguments passed to methods
estimate	name of the data-frame column containing the point estimates
lower	name of the data-frame column containing the lower confidence limits
upper	name of the data-frame column containing the upper confidence limits

Details

A "CI" object is a data frame containing the columns est, lci, and uci. Additional columns are retained and can be used as grouping variables by functions such as plotDot.

The primary purpose of as.CI() is to declare explicitly that an object contains estimates and confidence limits. For example, a numeric matrix with three columns is normally ambiguous: its columns may represent three groups or the estimate, lower limit, and upper limit. Passing the matrix to as.CI() declares that its columns have the latter meaning.

Supported inputs are:

- a numeric matrix with exactly three columns, interpreted in the order est, lci, and uci
- a data frame containing columns for the estimates and confidence limits; their names can be specified with estimate, lower, and upper
- a list in which every element contains three values representing c(est, lci, uci)
- an array-like result from tapply in which every cell contains c(est, lci, uci); its dimensions are converted to grouping variables
- an existing "CI" object, which is returned unchanged

The standardized object can be passed directly to plotDot to display the estimates and their confidence intervals. This is particularly useful for matrices, because a bare matrix supplied to plotDot() is interpreted as grouped estimates rather than as confidence interval data.

Value

`as.CI()` returns a data frame of class `"CI"` containing the columns `est`, `lci`, and `uci`, followed by any grouping columns; `is.CI()` returns a single logical value

See Also

`plotDot`, `fmCI`

Examples

```
# matrix containing estimate, lower limit, and upper limit
x <- matrix(
  c(
    10, 20, 30,
     8, 18, 28,
    12, 22, 32
  ),
  ncol = 3,
  dimnames = list(
    c("A", "B", "C"),
    c("est", "lci", "uci")
  )
)

ci <- as.CI(x)
ci
is.CI(ci)

# display the estimates and confidence intervals
plotDot(ci)

# data frame using the standard column names
d <- data.frame(
  est = c(10, 20),
  lci = c(8, 18),
  uci = c(12, 22),
  sex = c("F", "M")
)

as.CI(d)

# data frame using different column names
d <- data.frame(
  item = c("A", "B"),
  estimate = c(10, 20),
  lower = c(8, 18),
  upper = c(12, 22)
)

as.CI(
  d,
  estimate = "estimate",
  lower = "lower",
  upper = "upper"
)
```

```
# confidence intervals returned by tapply()
## Not run:
xci <- with(
  Pizza,
  tapply(
    temperature,
    driver,
    lumen::meanCI,
    na.rm = TRUE
  )
)

plotDot(as.CI(xci))

## End(Not run)
```

`as.fileLink` *Link to a self-contained embedded file*

Description

Turns a file into a download link that carries the file with it: the contents travel base64-encoded inside the `href`, so the resulting HTML needs no server and no accompanying assets. The counterpart of `as.img` for non-image files – a data set next to a table, a script next to its output.

Usage

```
as.fileLink(path, label = NULL, type = NULL)
```

Arguments

`path`	path to an existing file
`label`	link text; defaults to the file name
`type`	MIME type; guessed from the extension when `NULL`

Details

Browsers limit the size of a data URI, so this suits spreadsheets and text files rather than large binaries.

Value

an object of class `c("html", "character")`

See Also

Other html: `as.html()`, `as.img()`, `embedFile()`, `escapeHtml()`, `htmlNotation`, `htmlSubscript`, `toHtmlTable()`

Examples

```
fn <- tempfile(fileext = ".csv")
write.csv(head(iris), fn, row.names = FALSE)
as.fileLink(fn, label = "iris")
unlink(fn)
```

---

as.html *Mark a character vector as HTML*

Description

Tags a character vector with the S3 class `"html"` so that it prints via `preview.html` as readable text instead of as a raw character vector.

Usage

```
as.html(x)
```

Arguments

x	a character vector, typically containing HTML markup

Value

x, with class `"html"` added

See Also

Other html: `as.fileLink()`, `as.img()`, `embedFile()`, `escapeHtml()`, `htmlNotation`, `htmlSubscript`, `toHtmlTable()`

Examples

```
as.html("<b>bold</b>")
```

---

as.img *Embed a plot as an inline HTML image*

Description

Evaluates a plotting expression in a temporary PNG device and returns the resulting image as a self-contained, base64-encoded `<img>` tag (class `"html"`, see `as.html`), suitable for embedding directly in HTML text – a report, a question, an e-mail.

Usage

```
as.img(expr, width = 520, height = 440, res = 96, ...)
```

Arguments

expr	a plotting expression, evaluated once inside the device. Character input is evaluated via eval(parse(text = expr)) for compatibility with the earlier form of this function.
width, height	size of the device in pixels
res	nominal resolution in dpi, which also scales the text: raise it for a larger picture with the same relative proportions
...	further arguments passed to png

Details

The expression is passed unevaluated and carries its own environment, so a plot built inside a function sees that function's local variables. Several statements are given in braces, as in the examples below.

Value

an object of class c("html", "character") containing an <img> tag with a ... source

See Also

Other html: as.fileLink(), as.html(), embedFile(), escapeHtml(), htmlNotation, htmlSubscript, toHtmlTable()

Examples

```
img <- as.img(plot(1:10))

# several statements, and local variables
f <- function(n) {
  x <- seq_len(n)
  as.img({
    plot(x, x^2, type = "b")
    abline(h = mean(x^2), lty = 2)
  })
}
substr(f(10), 1, 30)
```

---

axisBreak *Place a Break Mark on an Axis*

Description

Places a break mark on an axis on an existing plot.

Usage

```
axisBreak(
  axis = 1,
  breakpos = NULL,
  pos = NA,
  bgcol = "white",
  breakcol = "black",
  style = "slash",
  brw = 0.02
)
```

Arguments

axis	which axis to break
breakpos	where to place the break in user units
pos	position of the axis (see axis)
bgcol	the color of the plot background
breakcol	the color of the "break" marker
style	either 'gap', 'slash' or 'zigzag'
brw	break width relative to plot width

Details

The 'pos' argument is not needed unless the user has specified a different position from the default for the axis to be broken.

Note

There is some controversy about the propriety of using discontinuous coordinates for plotting, and thus axis breaks. Discontinuous coordinates allow widely separated groups of values or outliers to appear without devoting too much of the plot to empty space.
The major objection seems to be that the reader will be misled by assuming continuous coordinates. The 'gap' style that clearly separates the two sections of the plot is probably best for avoiding this.

Based on code by Jim Lemon and Ben Bolker, adapted to conform to package standards

See Also

Other graphics.layout: abcCoords(), axTicks, isValidPlotRegion(), lineToUser(), mar(), plotFacet(), spreadOut()

Examples

```
plot(3:10, main="Axis break test")

# put a break at the default axis and position
axisBreak()
axisBreak(2, 2.9, style="zigzag")
```

axisFmt *Draw an Axis With Formatted or Rotated Labels*

Description

A drop-in replacement for graphics::axis() that adds two conveniences: tick labels can be formatted through fm() via the fmt argument, and labels can be drawn at an arbitrary angle using srt (which graphics::axis() itself ignores). When rotated labels are requested the function also estimates the margin space they require and, optionally, widens the corresponding plot margin so the labels are not clipped.

Usage

```
axisFmt(
  side,
  at = NULL,
  fmt = NULL,
  labels = TRUE,
  srt = NULL,
  adj = NULL,
  estimateMar = TRUE,
  ...
)
```

Arguments

side	integer specifying which side of the plot the axis is drawn on, as in graphics::axis(): 1 = below, 2 = left, 3 = above and 4 = right.
at	numeric vector of tick positions. If NULL (default) the positions are determined with graphics::axTicks().
fmt	format specification passed to fm() to format the tick labels. If NULL (default) the labels are used as they are (or the bare tick values, coerced to character). See Details.
labels	either a logical or a character vector of labels. If TRUE (default) the labels are generated from at (formatted with fmt if supplied). A character vector is used verbatim.
srt	numeric, the string rotation angle in degrees. If NULL (default) labels are drawn horizontally through the ordinary graphics::axis() mechanism. Any other value triggers the rotated-label path described in Details (a common choice for long category names is srt = 45).
adj	label justification passed to graphics::text(). If NULL a sensible default is chosen from side and the rotation: right aligned with the tick for the x-axes (c(1, 0.5)) and bottom aligned for the y-axes (c(0.5, 0)).
estimateMar	logical. If TRUE (default) and srt is set, the margin on side is temporarily widened to the estimated space needed by the rotated labels (restored on exit). Set to FALSE to leave the margins untouched.
...	further arguments passed to both graphics::axis() (for the line and ticks) and graphics::text() (for the labels). Graphical parameters shared by the two (e.g. col, cex) therefore affect both.

Details

The fmt argument is passed straight to fm() and therefore accepts the full range of format specifications: a special short code (e.g. "%", "e", "eng", "p"), an ISO-8601 date pattern (e.g. "MMM yyyy"), a Style object, a bare (named) list treated as a style template (e.g. fmt = list(digits = 1,bigMark = " ")), or a function of x. See fm() for the details.

When srt is set, the axis line and ticks are drawn without labels via graphics::axis(), and the labels are added separately with graphics::text() using xpd = TRUE so they may extend into the figure margin. The required margin width is estimated by projecting the rotated label boxes onto the axis normal (see estimateMar).

Note that changing par("mar") after the plot region has been established does not resize the existing region. When estimateMar is TRUE the widened margin therefore mainly ensures enough device space outside the plot region for the (xpd = TRUE) labels. For a clean layout, set the margin *before* calling graphics::plot(); the returned mar component reports the estimated requirement so a calling routine can reserve the space in advance.

Value

invisibly, a list with components at (the tick positions used) and mar (the estimated margin requirement in lines for the rotated labels, or NA when srt is NULL).

Author(s)

Andri Signorell andri@signorell.net

See Also

graphics::axis, graphics::axTicks, graphics::text, fm

Examples

```
# formatted tick labels
plot(1:10, runif(10) * 1000, yaxt = "n", xaxt = "n")
axisFmt(2, fmt = list(digits = 0, bigMark = " "))
axisFmt(1, fmt = list(fmt = "%", digits = 1))

# rotated category labels (margin widened automatically)
plot(1:5, c(7, 6, 11, 5, 12), xaxt = "n", xlab = "")
axisFmt(1, at = 1:5, labels = paste("Category", LETTERS[1:5]), srt = 45)
```

---

axTicks *Compute Axis Tickmark Locations (For POSIXct Axis)*

Description

Compute pretty tickmark locations, the same way as R does internally. By default, gives the at values which axis.POSIXct(side, x) would use.

Usage

```
axTicks.POSIXct(side, x, at, format, labels = TRUE, ...)

axTicks.Date(side = 1, x, ...)
```

Arguments

side	see graphics::axis
x, at	date-time or date object.
format	see base::strptime
labels	either a logical value specifying whether annotations are to be made at the tick-marks, or a vector of character strings to be placed at the tickpoints
...	further arguments to be passed from or to other methods

Details

axTicks has no implementation for POSIXct axis. This function fills the gap.

Value

numeric vector of coordinate values at which axis tickmarks can be drawn.

See Also

axTicks, axis.POSIXct

Other graphics.layout: abcCoords(), axisBreak(), isValidPlotRegion(), lineToUser(), mar(), plotFacet(), spreadOut()

Examples

```
with(beaver1, {
  time <- strptime(paste(1990, day, time %/% 100, time %% 100),
                   "%Y %j %H %M")
  plot(time, temp, type = "l") # axis at 4-hour intervals.
  # now label every hour on the time axis
  plot(time, temp, type = "l", xaxt = "n")
  r <- as.POSIXct(round(range(time), "hours"))
  axis.POSIXct(1, at = seq(r[1], r[2], by = "hour"), format = "%H")
  # place the grid
  abline(v=axTicks.POSIXct(1, at = seq(r[1], r[2], by = "hour"), format = "%H"),
         col="grey", lty="dotted")
})
```

---

band *Band Geometry*

Description

Create a polygonal band from upper and lower boundaries.

Usage

```
band(x, y)
```

Arguments

x	A vector or matrix of x coordinates.
y	A vector or matrix of y coordinates. If either x or y is supplied as a two-column matrix, the second column is interpreted as the lower boundary and reversed automatically.

Details

Typically used to represent confidence or prediction bands.

Value

An object inheriting from class "bandGeometry".

See Also

graphics::polygon

Other geometry.structures: arc(), bezier(), circle(), ellipse(), polygon(), regPolygon(), ring()

Examples

```
set.seed(18)

x <- rnorm(15)
y <- x + rnorm(15)

new <- seq(-3, 3, 0.5)

pred <- predict(
  lm(y ~ x),
  newdata = data.frame(x = new),
  interval = "confidence"
)

plot(y ~ x)

polygon(
  band(
    x = new,
    y = pred[,2:3]
  ),
  col = addOpacity("grey80", 0.5),
  border = NA
)
```

barText *Place Value Labels on a Barplot*

Description

It can sometimes make sense to display data values directly on the bars in a barplot. There are a handful of obvious alternatives for placing the labels, either on top of the bars, right below the upper end, in the middle or at the bottom. Determining the required geometry - although not difficult - is cumbersome and the code is distractingly long within an analysis code. The present function offers a short way to solve the task. It can place text either in the middle of the stacked bars, on top or on the bottom of a barplot (side by side or stacked).

Usage

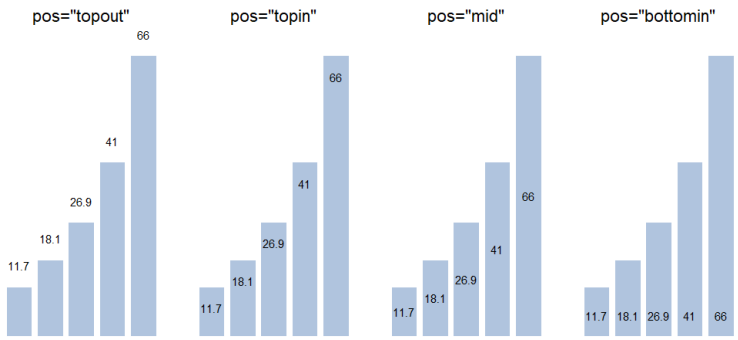
```
barText(
  height,
  b,
  labels = height,
  beside = FALSE,
  horiz = FALSE,
  cex = par("cex"),
  adj = NULL,
  pos = c("topout", "topin", "mid", "bottomin", "bottomout"),
  offset = 0,
  col = NULL,
  ...
)
```

Arguments

height	either a vector or matrix of values describing the bars which make up the plot exactly as used for creating the barplot.
b	the returned mid points as returned by b <- barplot(...).
labels	the labels to be placed on the bars.
beside	a logical value. If FALSE, the columns of height are portrayed as stacked bars, and if TRUE the columns are portrayed as juxtaposed bars.
horiz	a logical value. If FALSE, the bars are drawn vertically with the first bar to the left. If TRUE, the bars are drawn horizontally with the first at the bottom.
cex	numeric character expansion factor; multiplied by par("cex") yields the final character size. NULL and NA are equivalent to 1.0.
adj	one or two values in [0, 1] which specify the x (and optionally y) adjustment of the labels. On most devices values outside that interval will also work.
pos	one of "topout", "topin", "mid", "bottomin", "bottomout", defining if the labels should be placed on top of the bars (inside or outside) or at the bottom of the bars (inside or outside).
offset	a vector indicating how much the bars should be shifted relative to the x axis.
col	the color and to be used, possibly a vector (default in par("col")).
...	the dots are passed to the boxedText.

Details

The x coordinates of the labels can be found by using `barplot()` result, if they are to be centered at the top of each bar. `barText()` calculates the rest.



Notice that when the labels are placed on top of the bars, they may be clipped. This can be avoided by setting `xpd=TRUE`.

Value

returns the geometry of the labels invisibly

See Also

Other graphics.annotation: `boxedText()`, `colLegend()`, `errBars()`, `stamp()`, `textLegend()`, `titleRect()`

Examples

```
# simple vector
x <- c(353, 44, 56, 34)
b <- barplot(x)
barText(x, b, x)

# more complicated
b <- barplot(VADeaths, horiz = FALSE, col="steelblue", beside = TRUE)
barText(VADeaths, b=b, horiz = FALSE, beside = TRUE, cex=0.8)
barText(VADeaths, b=b, horiz = FALSE, beside = TRUE, cex=0.8, pos="bottomin",
        col="white", font=2)

b <- barplot(VADeaths, horiz = TRUE, col="steelblue", beside = TRUE)
barText(VADeaths, b=b, horiz = TRUE, beside = TRUE, cex=0.8)

b <- barplot(VADeaths)
barText(VADeaths, b=b)

b <- barplot(VADeaths, horiz = TRUE)
barText(VADeaths, b=b, horiz = TRUE, col="red", cex=1.5)
```

`bezier` *Bézier Geometry*

Description

Create a Bézier curve from a set of control points.

Usage

```
bezier(x, y = NULL, numPoints = 100)
```

Arguments

`x`, `y`	numeric vectors of control points. Alternatively, `x` may be a list with components `x` and `y`.
`numPoints`	number of points used to approximate the curve.

Value

An object inheriting from class `"bezierGeometry"`.

References

Farin, G. (1993). *Curves and Surfaces for Computer Aided Geometric Design*. Academic Press.

See Also

graphics::lines

Other geometry.structures: `arc()`, `band()`, `circle()`, `ellipse()`, `polygon()`, `regPolygon()`, `ring()`

Examples

```
canvas(xlim = c(0, 1))

lines(
  bezier(
    x = c(0, 0.5, 1),
    y = c(0, 1, 0)
  ),
  col = "red",
  lwd = 2
)
```

boxedText *Add Text in a Box to a Plot*

Description

boxedText draws the strings given in the vector labels at the coordinates given by x and y, surrounded by a rectangle.

Usage

```
boxedText(x, ...)

## Default S3 method:
boxedText(
  x,
  y = NULL,
  labels = NULL,
  adj = NULL,
  pos = NULL,
  offset = 0.5,
  vfont = NULL,
  cex = 1,
  col = NULL,
  font = NULL,
  srt = 0,
  xpad = 0.2,
  ypad = 0.2,
  density = NULL,
  angle = 45,
  bg = NA,
  border = par("fg"),
  lty = par("lty"),
  lwd = par("lwd"),
  ...
)

## S3 method for class 'formula'
boxedText(formula, data = parent.frame(), ..., subset)
```

Arguments

x, y	numeric vectors of coordinates where the text labels should be written. If the length of x and y differs, the shorter one is recycled.
...	additional arguments are passed to the text function.
labels	a character vector or expression specifying the text to be written. An attempt is made to coerce other language objects (names and calls) to expressions, and vectors and other classed objects to character vectors by as.character. If labels is longer than x and y, the coordinates are recycled to the length of labels.
adj	the value of adj determines the way in which text strings are justified. A value of 0 produces left-justified text, 0.5 (the default) centered text and 1 right-justified

	text. (Any value in `[0, 1]` is allowed, and on most devices values outside that interval will also work.) Note that the adj argument of text also allows adj = c(x, y) for different adjustment in x- and y- directions.
`pos`	a position specifier for the text. If specified this overrides any adj value given. Values of 1, 2, 3 and 4, respectively indicate positions below, to the left of, above and to the right of the specified coordinates.
`offset`	when pos is specified, this value gives the offset of the label from the specified coordinate in fractions of a character width.
`vfont`	`NULL` for the current font family, or a character vector of length 2 for Hershey vector fonts. The first element of the vector selects a typeface and the second element selects a style. Ignored if labels is an expression.
`cex`	numeric character expansion factor; multiplied by `par("cex")` yields the final character size. `NULL` and `NA` are equivalent to 1.0.
`col, font`	the color and (if vfont = NULL) font to be used, possibly vectors. These default to the values of the global graphical parameters in `par()`.
`srt`	the string rotation in degrees.
`xpad, ypad`	the proportion of the rectangles to the extent of the text within.
`density`	the density of shading lines, in lines per inch. The default value of `NULL` means that no shading lines are drawn. A zero value of density means no shading lines whereas negative values (and NA) suppress shading (and so allow color filling).
`angle`	angle (in degrees) of the shading lines.
`bg`	color(s) to fill or shade the rectangle(s) with. The default `NA` (or also NULL) means do not fill, i.e., draw transparent rectangles, unless density is specified.
`border`	color for rectangle border(s). The default is `par("fg")`. Use `border = NA` to omit borders (this is the default). If there are shading lines, `border = TRUE` means use the same colour for the border as for the shading lines.
`lty`	line type for borders and shading; defaults to `"solid"`.
`lwd`	line width for borders and shading. Note that the use of `lwd = 0` (as in the examples) is device-dependent.
`formula`	A formula of the form `lhs ~ rhs`, where `lhs` gives the response values and `rhs` the corresponding groups or explanatory variables.
`data`	an optional matrix or data frame (or similar; see `model.frame`) containing the variables in the formula. By default the variables are taken from `environment(formula)`.
`subset`	an optional vector specifying a subset of observations to be used in the analysis.

See Also

similar function in package **plotrix** plotrix::boxed.labels (lacking rotation option)

Other graphics.annotation: `barText()`, `colLegend()`, `errBars()`, `stamp()`, `textLegend()`, `titleRect()`

Examples

```
canvas(xpd=TRUE)

boxedText(0, 0, adj=0, label="This is boxed text", srt=seq(0,360,20),
          xpad=.3, ypad=.3)
points(0,0, pch=15)
```

```
plot(mpg ~ hp, data=mtcars, type="n", main="MT cars mpg/hp (log-log)",
     panel.first=quote(grid()), las=1, log="xy")
boxedText(mpg ~ hp, data=mtcars,
          labels=rownames(mtcars), cex=0.6, border=FALSE, bg="grey90")
```

---

canvas *Canvas for Geometric Plotting*

Description

This is just a wrapper for creating an empty plot with suitable defaults for plotting geometric shapes.

Usage

```
canvas(xlim = NULL, ylim = xlim, main = NULL, asp = 1, usrbg = "white", ...)
```

Arguments

xlim, ylim	the xlims and ylims for the plot. Default is c(-1, 1).
main	the main title on top of the plot.
asp	numeric, giving the aspect ratio y/x. (See plot.window for details. Default is 1.
usrbg	the color of the user space of the plot, defaults to "white".
...	additional arguments are passed to the plot() command.

Details

The plot is created with these settings:
asp = 1, xaxt = "n", yaxt = "n", xlab = "", ylab = "", frame.plot = FALSE.

Value

a list of all the previous values of the parameters changed (returned invisibly)

See Also

Other graphics.setup: polarGrid(), setBackCol()

Examples

```
canvas(7)
text(0, 0, "Hello world!", cex=5)
```

circle *Circle Geometry*

Description

Create a circular geometry.

Usage

```
circle(x = 0, y = 0, radius = 1, numPoints = 100)
```

Arguments

x, y	centre coordinates.
radius	circle radius.
numPoints	number of points used to approximate the circle.

Value

An object inheriting from class "circleGeometry".

See Also

Other geometry.structures: arc(), band(), bezier(), ellipse(), polygon(), regPolygon(), ring()

cmykToCmy *Convert CMYK to CMY*

Description

Convert CMYK colors to CMY.

Usage

```
cmykToCmy(col)
```

Arguments

col	numeric CMYK matrix.

Value

Numeric CMY matrix.

See Also

color-conversion-overview

cmykToRgb *Convert CMYK to RGB*

Description

Convert CMYK colors to RGB.

Usage

```
cmykToRgb(col, maxColorValue = 1)
```

Arguments

col	numeric CMYK matrix (columns C, M, Y, K).
maxColorValue	maximum channel value.

Value

Numeric RGB matrix.

See Also

color-conversion-overview

cmyToCmyk *Convert CMY to CMYK*

Description

Convert CMY colors to CMYK.

Usage

```
cmyToCmyk(col)
```

Arguments

col	numeric CMY matrix.

Value

Numeric CMYK matrix.

See Also

color-conversion-overview

`colLegend` *Add a Color Legend to a Plot*

Description

Draw a color legend (color strip) inside an existing plot region.

Usage

```
colLegend(
  x,
  y = NULL,
  col = rev(heat.colors(100)),
  labels = NULL,
  width = NULL,
  height = NULL,
  horiz = FALSE,
  xjust = 0,
  yjust = 1,
  inset = 0,
  region = c("plot", "figure", "device"),
  border = NA,
  box = FALSE,
  labelAdj = c("edge", "center"),
  adj = NULL,
  cex = 1,
  title = NULL,
  titleAdj = 0.5,
  ...
)
```

Arguments

`x`	left x-coordinate of the legend or a keyword specifying automatic placement: `"bottomright"`, `"bottom"`, `"bottomleft"`, `"left"`, `"topleft"`, `"top"`, `"topright"`, `"right"`, `"center"`.
`y`	top y-coordinate of the legend when `x` is numeric.
`col`	vector of colors.
`labels`	optional vector of labels.
`width`	width of the legend in user coordinates.
`height`	height of the legend in user coordinates.
`horiz`	logical; if `TRUE`, draw horizontally.
`xjust`	horizontal justification.
`yjust`	vertical justification.
`inset`	inset distance(s) when keyword positioning is used.
`region`	character string specifying the reference region used for keyword placement. One of: `"plot"`, `"figure"`, or `"device"`.
`border`	border color of individual color rectangles.

box	optional specification of an enclosing box around the whole legend. Can be: • FALSE, NULL, or NA: no box • TRUE: draw box with defaults • named list of arguments passed to rect
labelAdj	placement of labels relative to the color blocks: "edge" Labels are aligned with the strip edges. "center" Labels are centered within color blocks.
adj	text alignment passed to text.
cex	character expansion for labels.
title	optional title.
titleAdj	horizontal title adjustment.
...	additional arguments passed to text.

Details

The legend can be positioned either by explicit coordinates or by keyword placement such as "topright" or "left".

Labels may either be aligned with the edges of the color strip or centered within the color blocks.

Value

Invisibly returns a list with components:

rect List describing the color strip geometry with components width, height, left, and top.

text List containing the label coordinates x and y.

See Also

graphics::legend

Other graphics.annotation: barText(), boxedText(), errBars(), stamp(), textLegend(), titleRect()

Examples

```
plot(1:15,, xlim=c(0,10), type="n", xlab="", ylab="", main="Colorstrips")

# A
colLegend(x="right", inset=5, labels=c(1:10))

# B: Center the labels
colLegend(x=1, y=9, height=6, col=colorRampPalette(c("blue", "white", "red"),
  space = "rgb")(5), labels=1:5, labelAdj = "center")

# C: Outer frame
colLegend(x=3, y=9, height=6, col=colorRampPalette(c("blue", "white", "red"),
  space = "rgb")(5), labels=1:4, box=list(border="grey", lty="dashed"))

# D
colLegend(x=5, y=9, height=6, col=colorRampPalette(c("blue", "white", "red"),
  space = "rgb")(10), labels=sprintf("%.1f",seq(0,1,0.1)), cex=0.8)

# E: horizontal shape
```

```
colLegend(x=1, y=2, width=6, height=0.2, col=rainbow(500), labels=1:5,horiz=TRUE)

# F
colLegend(x=1, y=14, width=6, height=0.5, col=colorRampPalette(
  c("black","blue","green","yellow","red"), space = "rgb")(100),
  horiz=TRUE, box = TRUE)

# G
colLegend(x=1, y=12, width=6, height=1, col=colorRampPalette(c("black","blue",
            "green","yellow","red"), space = "rgb")(10), horiz=TRUE,
            border="black", title="From black to red", titleAdj=0)

text(x = c(8,0.5,2.5,4.5,0.5,0.5,0.5)+.2, y=c(14,9,9,9,2,14,12), LETTERS[1:7], cex=2)
```

`color-conversion-overview`

Color Conversion Functions in pharos

Description

pharos provides a family of functions for converting colors between representations. Read each matrix as **row -> column**: the cell at row X, column Y is the function that converts a color from representation X to representation Y. - marks the diagonal (no self-conversion), . marks a combination with no direct function. RGB is the hub between the two tables below.

R color, hex, HSV, and RGB

From \ To	Col	Hex	HSV	RGB
Col	-	`colToHex()`	`colToHSV()`	`colToRGB()`
Hex	`hexToCol()`	-	.	`hexToRGB()`
HSV	.	.	-	.
RGB	`rgbToCol()`	`rgbToHex()`	.	-

"Col" is any valid R color specification (name, hex string, or palette index) as accepted by `col2rgb`. No function starts from HSV: it is only ever a conversion target (via `colToHSV`), not a source – see the note below the second table for the reason this gap is left open.

RGB, CMY, CMYK, and long integer

From \ To	CMY	CMYK	Long	RGB
CMY	-	`cmyToCmyk()`	.	.
CMYK	`cmykToCmy()`	-	.	`cmykToRgb()`
Long	.	.	-	`longToRGB()`
RGB	`rgbToCmy()`	.	`rgbToLong()`	-

Not part of either conversion matrix

Two related functions transform a color without changing its representation, so they don't fit a row/column slot above:

Function	Purpose
`colToOpaque()`	Computes the equivalent opaque color for a transparent color against a background – Hex stays Hex
`grayScale()`	Converts colors to grayscale using luminance weighting – Col stays Col

Why HSV has no source functions

Base R already provides the HSV -> Col/Hex direction via `hsv()`, which builds a hex color string directly from h/s/v values – the same role `rgb()` plays for RGB triplets. pharos deliberately doesn't duplicate it; chain `hsv()` into `colToRGB()` or `colToHex()` instead (see examples).

See Also

`grDevices::col2rgb()`, `grDevices::hsv()`

Examples

```
# A "." in the matrix means chaining two functions through a hub
# (usually RGB or Col), not a missing feature.

# CMY -> RGB: no direct function: CMY -> CMYK -> RGB
c(0.2, 0.6, 0.9) |> cmyToCmyk() |> cmykToRgb()

# Long integer -> R color name: Long -> RGB -> Col
255 |> longToRGB() |> rgbToCol()

# HSV -> RGB: base R's hsv() bridges the gap noted above
hsv(h = 0.6, s = 0.8, v = 0.9) |> colToRGB()
```

---

colToHex *Convert R Colors to Hexadecimal Colors*

Description

Convert any valid R color specification to hexadecimal colors.

Usage

```
colToHex(col, opacity = 1)
```

Arguments

`col`	vector of valid R colors.
`opacity`	opacity value between 0 and 1.

Value

Character vector of hexadecimal colors.

See Also

color-conversion-overview

`colToHSV` *Convert R Colors to HSV*

Description

Convert any valid R color specification to HSV.

Usage

```
colToHSV(col, useAlphaChannel = FALSE)
```

Arguments

`col`	vector of valid R colors.
`useAlphaChannel`	logical indicating whether the alpha channel should be included.

Value

Numeric HSV matrix.

See Also

color-conversion-overview

`colToOpaque` *Equivalent Opaque Color for Transparent Color*

Description

Determine the equivalent opaque RGB color for a given partially transparent RGB color against a background of any color.

Usage

```
colToOpaque(col, opacity = NULL, bg = NULL)
```

Arguments

`col`	the color as hex value (use converters below if it's not available). `col` and `opacity` are recycled.
`opacity`	the opacity value, if left to NULL the alpha channels of the colors are used
`bg`	the background color to be used to calculate against (default is "white")

Details

Reducing the opacity against a white background is a good way to find usable lighter and less saturated tints of a base color. For doing so, we sometimes need to get the equivalent opaque color for the transparent color.

Value

An named vector with the hexcodes of the opaque colors.

See Also

Other color.manipulation: `addOpacity()`, `darken()`, `fade()`, `lighten()`, `mixColors()`

Examples

```
cols <- c(addOpacity("limegreen", 0.4),
          colToOpaque(colToHex("limegreen"), 0.4),
          "limegreen")
barplot(c(1, 1.2, 1.3), col=cols, panel.first=abline(h=0.4, lwd=10, col="grey35"))
```

---

colToRGB *Convert R Colors to RGB*

Description

Convert any valid R color specification to an RGB matrix.

Usage

```
colToRGB(col, useAlphaChannel = FALSE)
```

Arguments

`col`	vector of valid R colors.
`useAlphaChannel`	logical indicating whether the alpha channel should be included.

Value

Integer matrix with rows corresponding to RGB (and optionally alpha) channels.

See Also

grDevices::col2rgb, color-conversion-overview

Examples

```
rgbToCol(matrix(c(162,42,42), nrow=3))

rgbToLong(matrix(c(162,42,42), nrow=3))

colToRGB("peachpuff")
colToRGB(c(blu = "royalblue", reddish = "tomato")) # names kept

colToRGB(1:8)
```

`contrastColor` *Choose Optimal Text Color Based on WCAG Contrast*

Description

Computes the optimal text color (default: black or white) for a given background color based on the WCAG contrast ratio.

Usage

```
contrastColor(col, light = "white", dark = "black")
```

Arguments

`col`	vector of colors. Can be any valid R color specification: color names, hexadecimal strings (e.g. `"#RRGGBB"` or `"#RRGGBBAA"`), or palette indices.
`light`	color used for dark backgrounds (default: `"white"`).
`dark`	color used for light backgrounds (default: `"black"`).

Details

This function implements the official relative luminance definition for sRGB colors (including gamma correction) and selects the text color that maximizes the contrast ratio according to the Web Content Accessibility Guidelines (WCAG 2.1).

Compared to simple heuristics (e.g., mean RGB), this approach is perceptually more accurate and suitable for accessibility-critical applications.

The relative luminance is computed as:

$$L = 0.2126R + 0.7152G + 0.0722B$$

where R, G, B are linearized sRGB components:

$$c_{lin} = \begin{cases} \frac{c}{12.92} & c \leq 0.04045 \\ \left(\frac{c+0.055}{1.055}\right)^{2.4} & c > 0.04045 \end{cases}$$

The contrast ratio between two luminance values $L1$ and $L2$ is:

$$\frac{\max(L1, L2) + 0.05}{\min(L1, L2) + 0.05}$$

The function compares contrast ratios against `light` and `dark` and returns the better option.

Value

A character vector of the same length as `col`, containing either `light` or `dark`, depending on which yields the higher contrast ratio.

See Also

Other color.lookup: `findColor()`

Examples

```
cols <- c("black", "white", "red", "blue", "yellow", "#777777")
contrastColor(cols)

# custom text colors
contrastColor(cols, light = "#FFFFFF", dark = "#222222")
```

---

convUnit *Symbolic Unit Conversion Engine*

Description

Converts numeric values between units using a symbolic unit system, SI-derived expansions, and graph-based conversion for non-SI units.

Usage

```
convUnit(x, from, to, prefix = NULL, units = NULL)
```

Arguments

`x`	numeric value(s) to convert.
`from`	character string specifying the source unit.
`to`	character string specifying the target unit.
`prefix`	data frame of SI prefixes with columns `abbr` and `mult`.
`units`	data frame of unit conversion factors with columns `from`, `to`, and `fact`.

Details

Supports:

- SI base units (m, kg, s, A, K, mol, cd)
- Derived units (N, Pa, J, W, Hz)
- Prefixes (k, m, µ, etc.)
- Compound units (e.g. `"km/h"`, `"kg*m/s^2"`)
- Unit powers (e.g. `"m2"`, `"s^-1"`)
- Temperature conversion (C, F, K)

The function internally:

1. Parses units into symbolic components
2. Expands derived units (e.g. `N = kg*m/s^2`)
3. Computes dimensional vectors
4. Applies prefix scaling
5. Uses a graph search (Dijkstra) for non-SI conversions

The function checks dimensional consistency before conversion. If units are not compatible, an error is thrown.

For non-SI units (e.g. `"mi"`, `"bar"`), conversion paths are resolved via a graph representation of `Units`.

Temperature conversions are handled separately and do not use multiplicative scaling.

Value

Numeric value(s) converted to the target unit.

See Also

Other format: `fm()`, `fmCI()`, `print.Unit()`, `style()`, `unit()`

Examples

```
convUnit(100, "km/h", "m/s")

convUnit(1, "N", "kg*m/s^2")
convUnit(1, "Pa", "N/m^2")
convUnit(25, "C", "F")
convUnit(1, "ml", "l")
```

`coordinate-conversions`

Coordinate Transformations Cartesian/Polar/Spherical

Description

Transform cartesian into polar coordinates, resp. to spherical coordinates and vice versa.

Usage

```
polToCart(r, theta)

cartToPol(x, y)

cartToSph(x, y, z, up = TRUE)

sphToCart(r, theta, phi, up = TRUE)
```

Arguments

r	a vector with the radius of the points.
theta	a vector with the angle(s) of the points.
x, y, z	vectors with the xy-coordianates to be transformed.
up	logical. If set to TRUE (default) theta is measured from x-y plane, else theta is measured from the z-axis.
phi	a vector with the angle(s) of the points.

Details

Angles are in radians, not degrees (i.e., a right angle is pi/2). Use degToRad to convert, if you don't wanna do it by yourself.
All parameters are recycled if necessary.

Value

polToCart() returns a list of x and y coordinates of the points.
cartToPol() returns a list of r for the radius and theta for the angles of the given points.

Note

Based on code by Christian W. Hoffmann

See Also

Other geometry.conversion: degree-radians-conversion

Examples

```
cartToPol(x=1, y=1)
cartToPol(x=c(1,2,3), y=c(1,1,1))
cartToPol(x=c(1,2,3), y=1)


polToCart(r=1, theta=pi/2)
polToCart(r=c(1,2,3), theta=pi/2)

polToCart(r=c(1,2,3), theta=pi/2)

cartToSph(x=1, y=2, z=3)   # r=3.741657, theta=0.930274, phi=1.107149
```

---

darken *Darken Colors*

Description

Darken colors by mixing them with black.

Usage

```
darken(col, amount = 0.2)
```

Arguments

col	vector of valid R colors.
amount	numeric value between 0 and 1 specifying the amount of darkening. A value of 0 leaves the color unchanged, while 1 returns black.

Details

Colors are mixed linearly with black in RGB space:

$$x_{new} = xcdot(1 - amount)$$

Value

Character vector of hexadecimal colors.

See Also

Other color.manipulation: addOpacity(), colToOpaque(), fade(), lighten(), mixColors()

---

degree-radians-conversion

Convert Degrees to Radians and Vice Versa

Description

Convert degrees to radians (and back again).

Usage

```
degToRad(deg)

radToDeg(rad)
```

Arguments

deg	a vector of angles in degrees.
rad	a vector of angles in radians.

Value

degToRad returns a vector of the same length as deg with the angles in radians.
radToDeg returns a vector of the same length as rad with the angles in degrees.

See Also

Other geometry.conversion: coordinate-conversions

Examples

```
degToRad(c(90,180,270))
radToDeg( c(0.5,1,2) * pi)
```

ellipse *Ellipse Geometry*

Description

Create an elliptic geometry.

Usage

```
ellipse(x = 0, y = 0, radiusX = 1, radiusY = radiusX, numPoints = 100)
```

Arguments

x, y	centre coordinates.
radiusX, radiusY	horizontal and vertical radius.
numPoints	number of points used to approximate the ellipse.

Details

Use rotate to rotate the resulting geometry.

Value

An object inheriting from class "ellipseGeometry".

See Also

Other geometry.structures: arc(), band(), bezier(), circle(), polygon(), regPolygon(), ring()

embedFile *Base64-encode a file*

Description

The building block behind as.img and as.fileLink: reads a file and returns its contents base64-encoded, ready to be placed in a data URI or in any container format that carries binary payloads as text.

Usage

```
embedFile(path)
```

Arguments

path	path to an existing file

Value

a single character string

See Also

Other html: `as.fileLink()`, `as.html()`, `as.img()`, `escapeHtml()`, `htmlNotation`, `htmlSubscript`, `toHtmlTable()`

Examples

```
fn <- tempfile(fileext = ".txt")
writeLines("hello", fn)
substr(embedFile(fn), 1, 8)
unlink(fn)
```

---

errBars *Add Error Bars to an Existing Plot*

Description

Add vertical or horizontal error bars to an existing plot.

Usage

```
errBars(from, to = NULL, pos = NULL, horiz = TRUE, points = NULL, ...)
```

Arguments

`from`	coordinates of the lower end of the error bars. If `to = NULL` and `from` is a matrix: • a 2-column matrix is interpreted as `cbind(from, to)` • a 3-column matrix is interpreted as `cbind(point, from, to)`
`to`	coordinates of the upper end of the error bars.
`pos`	position of the error bars. For vertical bars this corresponds to x-positions; for horizontal bars to y-positions.
`horiz`	logical; if `TRUE`, horizontal error bars are drawn.
`points`	optional point specification.
`...`	additional graphical arguments passed to `arrows`.

Details

This is a lightweight wrapper around `arrows` with optional point symbols added via `points`.

Additional graphical arguments in `...` are passed to `arrows` and may be used to control the appearance of the error bars.

Common examples include:

- `col`: line color
- `lwd`: line width
- `lty`: line type
- `code`: which end caps to draw
- `length`: length of the end caps

Point symbols may optionally be added:

- `points = TRUE`: draw default points halfway between `from` and `to`
- `points = numeric`: use these values as point coordinates
- `points = list(...)`: fully specify point coordinates and graphical parameters

Example:

```
points = list(
  val = estimate,
  pch = 21,
  bg  = "gold",
  col = "black",
  cex = 1.2
)
```

The default orientation is horizontal (`horiz = TRUE`), which suits the typical use case of adding confidence intervals to a `dotchart`. Set `horiz = FALSE` for vertical bars on barplots or similar.

Value

Invisibly returns a list with components:

from Lower endpoints

to Upper endpoints

pos Positions of the error bars

points Point coordinates

See Also

`arrows`, `points`

Other graphics.annotation: `barText()`, `boxedText()`, `colLegend()`, `stamp()`, `textLegend()`, `titleRect()`

Examples

```
op <- par(no.readonly = TRUE)
par(mfrow = c(2, 2))

b <- barplot(1:5, ylim = c(0, 6))

errBars(
  from = 1:5 - 0.5,
  to = 1:5 + 0.5,
  pos = b,
  length = 0.15
)

# midpoint symbols
b <- barplot(1:5, ylim = c(0, 6))

errBars(
  from = 1:5 - 0.5,
  to = 1:5 + 0.5,
```

```
  pos = b,
  points = TRUE
)

# custom points
dotchart(1:5, xlim = c(0, 6))

errBars(
  from = 1:5 - 0.2,
  to = 1:5 + 0.2,
  horiz = TRUE,
  points = list(
    val = 1:5,
    pch = 21,
    bg = "gold",
    col = "black",
    cex = 1.2
  )
)

par(op)
```

`escapeHtml` *Escape HTML special characters*

Description

Replaces the three characters that would otherwise be read as markup. Use it on any text of unknown origin – a variable label, a file name, a user-supplied caption – before pasting it into HTML or XML.

Usage

```
escapeHtml(x, attribute = FALSE)
```

Arguments

`x`	a character vector
`attribute`	logical; also escape single and double quotes

Details

Quotes are escaped as well when `attribute = TRUE`, which is required for text placed inside an attribute value rather than between tags.

Value

a character vector of the same length

See Also

Other html: `as.fileLink()`, `as.html()`, `as.img()`, `embedFile()`, `htmlNotation`, `htmlSubscript`, `toHtmlTable()`

Examples

```
escapeHtml("Anteil < 5% & steigend")
escapeHtml('say "hi"', attribute = TRUE)
```

---

fade *Fade Colors*

Description

Apply transparency and remove the alpha channel afterwards.

Usage

```
fade(col, opacity = 0.5)
```

Arguments

col	vector of valid R colors
opacity	opacity value between 0 and 1

Value

Character vector of colors.

See Also

Other color.manipulation: addOpacity(), colToOpaque(), darken(), lighten(), mixColors()

---

findColor *Get Color on a Defined Color Range*

Description

Find a color on a defined color range depending on the value of x. This is helpful for colorcoding numeric values.

Usage

```
findColor(
  x,
  col = rev(heat.colors(100)),
  minX = NULL,
  maxX = NULL,
  allInside = FALSE
)
```

Arguments

x	numeric.
col	a vector of colors.
minX	the x-value to be used for the left edge of the first color. If left to the default NULL min(pretty(x)) will be used.
maxX	the x-value to be used for the right edge of the last color. If left to the default NULL max(pretty(x)) will be used.
allInside	logical; if true, the returned indices are coerced into 1, ..., N-1, i.e., 0 is mapped to 1 and N to N-1.

Details

For the selection of colors the option rightmost.closed in the used function findInterval is set to TRUE. This will ensure that all values on the right edge of the range are assigned a color. How values outside the boundaries of minX and maxX should be handled can be controlled by allInside. Set this value to TRUE, if those values should get the colors at the edges or set it to FALSE, if they should remain white (which is the default).

Note that findInterval closes the intervals on the left side, e.g. [0, 1). This option can't be changed. Consequently will x-values lying on the edge of two colors get the color of the bigger one.

See Also

findInterval

Other color.lookup: contrastColor()

Examples

```
canvas(7, main="Use of function findColor()")

# get some data
x <- c(23, 56, 96)
# get a color range from blue via white to red
cols <- colorRampPalette(c("blue","white","red"))(100)
colLegend(x="bottomleft", col=cols, labels=seq(0, 100, 10), cex=0.8)

# and now the color coding of x:
(xcols <- findColor(x, cols, minX=0, maxX=100))

# this should be the same as
cols[x+1]

# how does it look like?
y0 <- c(-5, -2, 1)
text(x=1, y=max(y0)+2, labels="Color coding of x:")
text(x=1.5, y=y0, labels=x)
polygon(regPolygon(x=3, y=y0, numVertices=4, startAngle=pi/4), col=xcols)
text(x=6, y=y0, labels=xcols)

# how does the function select colors?
canvas(xlim = c(0,1), ylim = c(0,1))
cols <- c(red="red", yellow="yellow", green="green", blue="blue")
```

```
colLegend(x=0, y=1, width=1, col=rev(cols), horiz = TRUE,
            labels=format(seq(0, 1, .25), digits=2, nsmall=2),
            frame="grey", cex=0.8 )
x <- c(-0.2, 0, 0.15, 0.55, .75, 1, 1.3)
arrows(x0 = x, y0 = 0.6, y1 = 0.8, angle = 15, length = .2)
text(x=x, y = 0.5, labels = x, adj = c(0.5,0.5))
text(x=x, y = 0.4, labels = names(findColor(x, col=cols,
   minX = 0, maxX = 1, allInside = TRUE)), adj = c(0.5,0.5))
text(x=x, y = 0.3, labels = names(findColor(x, col=cols,
   minX = 0, maxX = 1, allInside = FALSE)), adj = c(0.5,0.5))
```

`fm` *Format Numbers and Dates*

Description

Formatting numbers with base R tools often degenerates into a major intellectual challenge for us little minds down here in the valley of tears. There are a number of options available and quite often it's hard to work out which one to use, when a more uncommon setting is needed. The `fm()` function wraps all these functions and tries to offer a simpler, less technical, but still flexible interface.

Usage

```
fm(
  x,
  digits = NULL,
  leadDigits = NULL,
  sci = NULL,
  bigMark = NULL,
  decMark = NULL,
  naForm = NULL,
  zeroForm = NULL,
  fmt = NULL,
  pThreshold = NULL,
  width = NULL,
  align = NULL,
  lang = NULL,
  ...
)

## Default S3 method:
fm(
  x,
  digits = NULL,
  leadDigits = NULL,
  sci = NULL,
  bigMark = NULL,
  decMark = NULL,
  naForm = NULL,
  zeroForm = NULL,
  fmt = NULL,
  pThreshold = NULL,
```

```
  width = NULL,
  align = NULL,
  lang = NULL,
  ...
)

## S3 method for class 'data.frame'
fm(
  x,
  digits = NULL,
  leadDigits = NULL,
  sci = NULL,
  bigMark = NULL,
  decMark = NULL,
  naForm = NULL,
  zeroForm = NULL,
  fmt = NULL,
  pThreshold = NULL,
  width = NULL,
  align = NULL,
  lang = NULL,
  ...
)

## S3 method for class 'matrix'
fm(
  x,
  digits = NULL,
  leadDigits = NULL,
  sci = NULL,
  bigMark = NULL,
  decMark = NULL,
  naForm = NULL,
  zeroForm = NULL,
  fmt = NULL,
  pThreshold = NULL,
  width = NULL,
  align = NULL,
  lang = NULL,
  ...
)

## S3 method for class 'table'
fm(
  x,
  digits = NULL,
  leadDigits = NULL,
  sci = NULL,
  bigMark = NULL,
  decMark = NULL,
  naForm = NULL,
  zeroForm = NULL,
```

```
  fmt = NULL,
  pThreshold = NULL,
  width = NULL,
  align = NULL,
  lang = NULL,
  ...
)

## S3 method for class 'ftable'
fm(
  x,
  digits = NULL,
  leadDigits = NULL,
  sci = NULL,
  bigMark = NULL,
  decMark = NULL,
  naForm = NULL,
  zeroForm = NULL,
  fmt = NULL,
  pThreshold = NULL,
  width = NULL,
  align = NULL,
  lang = NULL,
  ...
)
```

Arguments

x	a numeric, logical, character, factor, Date, or POSIXt vector, or a matrix, table, ftable, or data frame
digits	integer, the desired (fixed) number of digits after the decimal point. Unlike formatC you will always get this number of digits even if the last digit is 0. Negative numbers of digits round to a power of ten (digits=-2 would round to the nearest hundred) for standard numeric formats; engineering formats require nonnegative values
leadDigits	number of leading zeros. leadDigits=3 would make sure that at least 3 digits on the left side will be printed, say 3.4 will be printed as 003.4. Setting leadDigits to 0 will yield results like .452 for 0.452. The default NULL will leave the numbers as they are (meaning at least one 0 digit).
sci	numeric scalar giving the absolute power-of-ten threshold for scientific notation. Its absolute value is used symmetrically: for sci = 8, nonzero values below 10^{-8} and values at or above 10^{8} are displayed scientifically. The default is based on getOption("scipen"); an option value of zero is replaced by 7
bigMark	character; if not empty used as mark between every 3 decimals before the decimal point. Default is "" (none).
decMark	character specifying the decimal mark. If NULL, the current OutDec option is used
naForm	character, string specifying how NAs should be specially formatted. If set to NULL (default) no special action will be taken.
zeroForm	character, string specifying how zeros should be specially formatted. Useful for pretty printing 'sparse' objects. If set to NULL (default) no special action will be

	taken.
fmt	a format code or date-time template, a formatting function, a named Style, or an object of class Style. See Details
pThreshold	positive numeric threshold below which p-values are shown as "< threshold"
width	nonnegative integer giving the minimum display width
align	the character on whose position the strings will be aligned. Left alignment can be requested by setting sep = "\l", right alignment by "\r" and center alignment by "\c". Mind the backslashes, as if they are omitted, strings would be aligned to the **character** l, r or c respectively. The default is NULL which would just leave the strings as they are. This argument is send directly to the function strAlign() as argument sep.
lang	optional value setting the language for the months and daynames. Can be either "local" for current locale or "en" for english. If left to NULL, the package option "lang" is used, falling back to "en"
...	additional arguments passed to methods or to a formatting function supplied through fmt

Details

There's also an easygoing interface for format templates, defined as a list consisting of any accepted format features. This enables to define templates globally and easily change or modify them later.

fm() is the workhorse for formatting numbers and dates, supporting a comprehensive range of format options that are likely to occur in everyday reporting. Among these, the argument fmt deserves a more detailed description due to its flexibility. It is used to generate a variety of different special formats.

If x is a date, it can take ISO-8601-inspired token syntax similar to .NET or Moment.js (consisting of d, M and y for day, month or year and h/H, m, s, t for hours, minutes, seconds and AM/PM designator) and defining the combination of day month and year representation.

Code	Description
d	day of the month without leading zero (1 - 31)
dd	day of the month with leading zero (01 - 31)
ddd	abbreviated name for the day of the week (e.g. Mon) in the current user's language
dddd	full name for the day of the week (e.g. Monday) in the current user's language
do	The token do (aka 'day ordinal') formats the day of month using English ordinal suffixes (e.g. 1st, 2nd, 3rd, 4th).
M	month without leading zero (1 - 12)
MM	month with leading zero (01 - 12)
MMM	abbreviated month name (e.g. Jan) in the current user's language
MMMM	full month name (e.g. January) in the current user's language
y	year without century, without leading zero (0 - 99)
yy	year without century, with leading zero (00 - 99)
yyyy	year with century. For example: 2005
H/HH	Hour in 24h format, one digit / two digits
h/hh	Hour in 12h format, one digit / two digits, note that in this case t must be set also to ensure uniqueness.
t/tt	AM/PM description (one/two characters)
m/mm	Minutes one digit / two digits
s/ss	Seconds one digit / two digits

Weekdays and month names can be expressed in the local language or in English. The language can be controlled by the argument "lang".

Even more variability is needed to display numeric values. For the most frequently used formats there are the following special codes available:

Code	Type	Description
e	scientific	forces scientific representation of x, e.g. 3.141e-05. The number of digits, alignment and zero values are further respected.
eng	engineering	forces scientific representation of x, but only with powers that are a multiple of 3.
engabb	engineering abbr.	same as eng, but replaces the exponential representation by codes, e.g. M for mega (1e6).
%	percent	multiplies the given number by 100 and appends the \ formats values as p-values. Use pThreshold to define the threshold to e.g. switch to a <0.001 representation.
frac	fractions	will (try to) convert numbers to fractions. So 0.1 will be displayed as 1/10. See fractions().
*	significance	will produce a significance representation of a p-value consisting of * and ., while the breaks are set according to the used defaults e.g. in lm as [0, 0.001] = *** (0.001, 0.01] = ** (0.01, 0.05] = * (0.05, 0.1] = . (0.1,1] =
p*	p-value stars	will produce p-value and significance stars

fmt can as well be an object of class "Style" consisting of a list out of the arguments above (as created by style()). This allows to store and manage the full format in variables or as options and use it as format template subsequently. Arguments supplied directly to fm() override the corresponding Style settings, including an explicitly supplied NULL.

For data frames, every formatting argument must have length one or the number of columns. Length-one arguments are recycled, allowing each column to use its own formatting settings without ambiguous partial recycling. Functions and Style objects count as single settings; use a list to supply different functions or Styles by column.

Finally, fmt can be a function of x. Additional arguments in ... are forwarded to that function.

Value

formatted character values with the dimensions or tabular structure of x preserved where applicable

See Also

base::format, base::formatC, base::prettyNum, base::sprintf, stats::symnum, base::Sys.setlocale, DescToolsX::weekday, DescToolsX::month, theme

Other format: convUnit(), fmCI(), print.Unit(), style(), unit()

Examples

```
fm(as.Date(c("2014-11-28", "2014-1-2")), fmt="ddd, d mmmm yyyy")
fm(as.Date(c("2014-11-28", "2014-1-2")), fmt="ddd, d mmmm yyyy", lang="en")

# using english ordinal suffixes
fm(as.Date("2026-01-21"), fmt="MMMM do yyyy", lang="en")
# e.g. in context:
gettextf("Report generated on %s",
         fm(as.Date("2026-05-04"), fmt="MMMM do, yyyy", lang="en"))

# numeric formats
x <- pi * 10^(-10:10)

fm(x, digits=3, fmt="%")
fm(x, digits=4, sci=4, leadDigits=0, width=9, align=".")


# format a matrix
m <- matrix(runif(100), nrow=10,
            dimnames=list(LETTERS[1:10], LETTERS[1:10]))

fm(m, digits=1)

# engineering format
fm(x, fmt="eng",  digits=2)
fm(x, fmt="engabb", leadDigits=2, digits=2)
# combine with grams [g]
paste(fm(x, fmt="engabb", leadDigits=2, digits=2), "g", sep="")

# example form symnum
pval <- rev(sort(c(outer(1:6, 10^-(1:3)))))
noquote(cbind(fm(pval, fmt="p"), fm(pval, fmt="*")))

# change the character to be used as the decimal point
fm(1200, digits=2, bigMark = ".", decMark=",")
```

---

fmCI *Format Confidence Intervals*

Description

Format confidence intervals using a flexible template.

Usage

```
fmCI(x, template = NULL, ...)
```

Arguments

x	the numerical values, given in the order in which they are used in the template.
template	character string as template for the desired format. %s are the placeholders for the numerical values. Default is \code{"%s [%s, %s]"} for <est> [<lci>, <uci>].
...	the dots are passed on to the fm() function.

Value

a formatted string

See Also

fm

Other format: convUnit(), fm(), print.Unit(), style(), unit()

Examples

```
x <- c(est=2.1, lci=1.5, uci=3.8)

# default template
fmCI(x)
# user defined template (note the double percent code)
fmCI(x, template="%s (95%%-CI %s-%s)", digits=1)

# in higher datastructures
tapply(warpbreaks$breaks, warpbreaks$wool,
       function(x) fmCI(c(mean(x), t.test(x)$conf.int),
                        digits=1))
```

---

ftable.list *Flat Contingency Table for tapply-Like Lists*

Description

Creates a flat contingency table from a list array, such as the result of tapply when the applied function returns a named vector.

Usage

```
## S3 method for class 'list'
ftable(x, row.vars = NULL, col.vars = NULL, ...)
```

Arguments

x	A list with a dim attribute, typically produced by tapply. Each element must be a vector of equal length and have identical names.
row.vars	row variables passed to ftable. Defaults to all dimensions except those specified in col.vars.
col.vars	column variables passed to ftable. Defaults to the dimension created from the names of the list elements.
...	further arguments passed to ftable.

Details

Each list element is expanded into an additional dimension corresponding to the names of the returned vector. The resulting array is then passed to `ftable`.

This is particularly useful for displaying multi-dimensional summaries such as confidence intervals returned by `meanCI()`, where each cell contains several statistics (e.g. estimate, lower CI, upper CI).

The names of the vectors stored in the list elements become an additional dimension of the resulting array. By default, this new dimension is shown in the columns of the flat contingency table.

Value

An object of class `"ftable"`.

See Also

`tapply`, `ftable`

Examples

```
## Not run:

x <- with(
  Pizza,
  tapply(
    temperature,
    list(area, driver),
    lumen::meanCI,
    na.rm = TRUE
  )
)

ftable(x)

ftable(
  x,
  row.vars = c(2, 3),
  col.vars = 1
)


## End(Not run)
```

`grayScale` *Convert Colors to grayScale*

Description

Convert colors to grayScale using luminance weighting.

Usage

```
grayScale(col)
```

Arguments

col	vector of valid R colors.

Details

grayScale conversion uses the standard luminance approximation:

$$0.3R + 0.59G + 0.11B$$

Value

Character vector of grayScale colors.

See Also

color-conversion-overview

Examples

```
op <- par(no.readonly = TRUE)
par(mfcol=c(2,2))

tmp <- 1:3
names(tmp) <- c('red','green','blue')

barplot(tmp, col=c('red','green','blue'))
barplot(tmp, col=grayScale(c('red','green','blue')))

barplot(tmp, col=c('red','#008100','#3636ff'))
barplot(tmp, col=grayScale(c('red','#008100','#3636ff')))

par(op)
```

---

hcol *Helsana Colors*

Description

Retrieve one or more colors from the helsana palette.

Usage

```
hcol(...)
```

Arguments

...	character strings naming the colors to retrieve. Valid names are: "blue", "red", "orange", "yellow", "ecru", "green", "pink", "moss", "slate", "sand", "brown", "plum". If none are provided, the full palette is returned.

Value

A named character vector of hex color codes.

See Also

Other color.palettes: pal(), palNames()

Examples

```
hcol("blue", "green")
hcol()
```

`hexToCol` *Convert Hex Colors to Named R Colors*

Description

Convert hexadecimal colors to the nearest named R colors.

Usage

```
hexToCol(hex, method = c("rgb", "hsv"), metric = c("euclidean", "manhattan"))
```

Arguments

`hex`	character vector of hexadecimal colors.
`method`	color space used for matching.
`metric`	distance metric.

Value

Character vector of named R colors.

See Also

color-conversion-overview

`hexToRGB` *Convert Hex Colors to RGB*

Description

Convert hexadecimal color strings to an RGB matrix.

Usage

```
hexToRGB(hex)
```

Arguments

`hex`	character vector of hexadecimal colors.

Value

Integer matrix with RGB rows.

See Also

color-conversion-overview

`htmlNotation` *HTML notation for hat and bar diacritics*

Description

Append the HTML entity for a circumflex ("hat", as in an estimator $\hat{x}$) or macron ("bar", as in a sample mean $\bar{x}$) to a label.

Usage

```
htmlHat(x)

htmlBar(x)
```

Arguments

`x`	a character vector, typically a variable name or symbol

Value

a character vector with the diacritic's HTML entity appended

See Also

Other html: `as.fileLink()`, `as.html()`, `as.img()`, `embedFile()`, `escapeHtml()`, `htmlSubscript`, `toHtmlTable()`

Examples

```
htmlHat("p")  # -> "p&#770;"   (renders as p with a circumflex)
htmlBar("x")  # -> "x&#772;"   (renders as x with a macron)
```

`htmlSubscript` *Subscript notation*

Description

Infix operator producing an HTML subscript, e.g. for indexed symbols such as x_i.

Usage

```
x %_% i
```

Arguments

`x`	a character vector (the base symbol)
`i`	a character vector (the subscript), recycled against `x`

Value

a character vector: x followed by _i

See Also

Other html: as.fileLink(), as.html(), as.img(), embedFile(), escapeHtml(), htmlNotation, toHtmlTable()

Examples

```
"x" %_% "i"  # -> "x<sub>i</sub>"
```

---

isValidPlotRegion *Check Whether the Current Plot Region Is Large Enough*

Description

Tests whether the plot region of the active graphics device meets a minimal size requirement. This is useful before drawing into small devices or heavily subdivided layouts (mfrow, layout(), Shiny render panes), where an undersized region would otherwise fail with the rather cryptic error "figure margins too large".

Usage

```
isValidPlotRegion(minPin = .getOption("plot.minPin", c(0.5, 0.5)))
```

Arguments

minPin	numeric vector of length 2, giving the minimal required width and height of the plot region in inches. Defaults to the option DescToolsX.plot.minPin and falls back to c(0.5, 0.5) if the option is not set.

Details

The plot region is the area inside the figure margins, as reported by par("pin"). Its size results from the device dimensions minus the current margins (mar, oma), so both a small device and oversized margins can render it degenerate.

If no graphics device is open (dev.cur() == 1), the function returns TRUE, since the next high-level plot call will open a fresh device with default dimensions. Note that querying par() in this state would itself open a device as a side effect, which this function deliberately avoids.

The function has no side effects and never throws; it is meant as a guard for conditional plotting, e.g. if (isValidPlotRegion()) plot(x) else message("device too small").

Value

a single logical: TRUE if the plot region is at least minPin inches in both dimensions (or if no device is open), FALSE otherwise.

See Also

par (entries pin, fin, mar), dev.cur

Other graphics.layout: abcCoords(), axTicks, axisBreak(), lineToUser(), mar(), plotFacet(), spreadOut()

Examples

```
# guard a plot against a degenerate region
if (isValidPlotRegion()) {
  plot(rnorm(20))
} else {
  message("plot region too small")
}

# require a larger region, e.g. for a wide correlation plot
isValidPlotRegion(minPin = c(4, 3))
```

---

lighten *Lighten Colors*

Description

Lighten colors by mixing them with white.

Usage

```
lighten(col, amount = 0.2)
```

Arguments

col	vector of valid R colors.
amount	numeric value between 0 and 1 specifying the amount of lightening. A value of 0 leaves the color unchanged, while 1 returns white.

Details

Colors are mixed linearly with white in RGB space:

$$x_{new} = x + amount \cdot (255 - x)$$

Value

Character vector of hexadecimal colors.

See Also

Other color.manipulation: addOpacity(), colToOpaque(), darken(), fade(), mixColors()

`lines.lm` *Add a Linear Regression Line*

Description

Add a linear regression line to an existing plot. The function first calculates predictions from an `lm` object and then adds the fitted line together with optional confidence and prediction bands.

Usage

```
## S3 method for class 'lm'
lines(
  x,
  col = pal()[1],
  lwd = 2,
  lty = "solid",
  type = "l",
  n = 100,
  cbandArgs = list(conf.level = 0.95),
  pbandArgs = NA,
  xpred = NULL,
  ...
)

## S3 method for class 'lmlog'
lines(
  x,
  col = pal()[1],
  lwd = 2,
  lty = "solid",
  type = "l",
  n = 100,
  cbandArgs = list(conf.level = 0.95),
  pbandArgs = NA,
  xpred = NULL,
  ...
)
```

Arguments

`x`	linear model object as returned by `lm`.
`col`	line color. Defaults to `pal()[1]`.
`lwd`	line width.
`lty`	line type.
`type`	plotting type passed to `lines`.
`n`	number of points used for plotting the fit.
`cbandArgs`	controls the confidence band. May be `TRUE`, `FALSE`, `NULL`, `NA`, or a named list. The confidence level is specified via `conf.level`. Default is `list(conf.level=0.95)`.

pbandArgs	controls the prediction band. May be TRUE, FALSE, NULL, NA, or a named list. The confidence level is specified via conf.level. Default is NA.
xpred	optional numeric vector defining the range over which predictions should be calculated.
...	currently ignored.

Details

In contrast to abline, polynomial models and transformed predictors are supported as long as the model contains exactly one predictor.

Confidence and prediction bands are controlled via cbandArgs and pbandArgs. These arguments can be:

- FALSE, NULL or NA: suppress the band
- TRUE: draw the band with default settings
- a named list: customize the band appearance and confidence level

Value

No return value; called for its side effect.

See Also

lines, lm

Other graphics.trendlines: lines.loess(), splineCI

---

lines.loess *Add a Loess Smoother and Its Confidence Band*

Description

Add a loess smoother to an existing plot. The function first calculates predictions from a loess object and then adds the fitted smoother together with an optional confidence band.

Usage

```
## S3 method for class 'loess'
lines(
  x,
  col = .useTheme,
  lwd = 2,
  lty = "solid",
  type = "l",
  n = 100,
  bandArgs = list(conf.level = 0.95),
  ...
)
```

Arguments

x	a fitted loess object.
col	line color of the smoother. .useTheme (default) resolves to getTheme()$twin[1] - the first of the theme's two-color pair (see theme).
lwd	line width.
lty	line type.
type	plotting type passed to lines.
n	number of points used for plotting the fit.
bandArgs	controls the confidence band. May be TRUE, FALSE, NULL, NA, or a named list. The confidence level is specified via conf.level. Default is list(conf.level = 0.95).
...	currently ignored.

Details

The confidence band is controlled via bandArgs. This argument may be:

- FALSE, NULL or NA: suppress the band
- TRUE: draw the band with default settings
- a named list: customize the band appearance and confidence level

Note

Loess can result in substantial computational load for large datasets.

See Also

loess, scatter.smooth, smooth.spline

Other graphics.trendlines: lines.lm(), splineCI

Examples

```
x <- runif(100)
y <- rnorm(100)

plot(x, y)
lines(loess(y ~ x))

plot(dist ~ speed, cars)
lines(
  loess(dist ~ speed, cars),
  bandArgs = list(
    conf.level = 0.99,
    col = addOpacity("red", 0.4),
    border = "black"
  )
)
```

lineSep *Create a Line Separator String*

Description

Generates a character string that can be used as a horizontal line separator in console output.

Usage

```
lineSep(sep = .getOption("linesep"))
```

Arguments

sep	A character string used as separator. If NULL, a default Unicode box-drawing character ("\u2500") is used. If sep consists of a single visible character, it is repeated to match the current console width (minus two characters).

Details

ANSI escape sequences (e.g., color codes) are removed using cli::ansi_strip() when determining the visible width.

The console width is determined via getOption("width", 80).

Value

A character string representing a horizontal separator line.

Examples

```
lineSep()
lineSep("-")
lineSep(cli::col_red("="))
```

lineToUser *Convert Line Coordinates To User Coordinates*

Description

Functions like mtext or axis use the line argument to set the distance from plot. Sometimes it's useful to have the distance in user coordinates. lineToUser() does this nontrivial conversion.

Usage

```
lineToUser(line, side)
```

Arguments

line	the number of lines
side	the side of the plot

Details

For the `lineToUser` function to work, there must be an open plot.

Value

the user coordinates for the given lines

See Also

`mtext`

Other graphics.layout: `abcCoords()`, `axTicks`, `axisBreak()`, `isValidPlotRegion()`, `mar()`, `plotFacet()`, `spreadOut()`

Examples

```
plot(1:10)
lineToUser(line=2, side=4)
```

---

longToRGB *Convert Long Integers to RGB*

Description

Decode long integers into RGB values.

Usage

```
longToRGB(col)
```

Arguments

`col`	integer vector.

Value

RGB matrix.

See Also

color-conversion-overview

mar *Get or set plot margins conveniently*

Description

Convenience wrapper around par for getting and setting plot margins (mar) or outer margins (oma). Individual sides can be modified without affecting the others.

Usage

```
mar(bottom = NULL, left = NULL, top = NULL, right = NULL, outer = FALSE)
```

Arguments

bottom, left, top, right	numeric scalars specifying the margin size (in lines) for each side. If NULL, the current value is retained.
outer	logical; if TRUE, outer margins (oma) are used instead of inner margins (mar).

Details

This function simplifies partial updates of par("mar") or par("oma"). It avoids the need to manually query and reconstruct the full margin vector.

For restoring graphical parameters, the recommended base R approach is:

```
op <- par(no.readonly = TRUE)
on.exit(par(op))
```

Value

If no arguments are provided, returns the current margin vector (numeric of length 4). Otherwise, sets the margins and returns the new values invisibly.

See Also

Other graphics.layout: abcCoords(), axTicks, axisBreak(), isValidPlotRegion(), lineToUser(), plotFacet(), spreadOut()

Examples

```
# Get current margins
mar()

# Set bottom margin only
mar(bottom = 2)

# Set multiple margins
mar(bottom = 2, left = 2)

# Modify outer margins
mar(top = 1, outer = TRUE)
```

mixColors *Mix Colors*

Description

Mix two sets of colors in RGB space.

Usage

```
mixColors(col1, col2, weights = 0.5)
```

Arguments

col1	first vector of colors.
col2	second vector of colors.
weights	numeric value between 0 and 1 specifying the contribution of col2. A value of 0 returns col1, while 1 returns col2.

Details

Colors are mixed linearly in RGB space:

$$x_{mix} = (1 - weights)x_1 + weightsx_2$$

All arguments are recycled as necessary.

Value

Character vector of hexadecimal colors.

See Also

Other color.manipulation: addOpacity(), colToOpaque(), darken(), fade(), lighten()

pal *Get a Color Palette*

Description

Returns n colors from a named palette. All palettes always return exactly n colors regardless of their base size:

- n < length(base): evenly spaced sample for maximum contrast
- n = length(base): returned as-is
- n > length(base): interpolated via colorRampPalette

Usage

```
pal(name, n = NA, opacity = 1)

## S3 method for class 'Palette'
plot(x, cex = 2.5, border = "grey70", ...)
```

Arguments

name	character or integer. Palette name (full match via match.arg) or index into palNames(). If missing, returns the palette named in the active theme (getTheme()$palette, see theme).
n	integer, number of colors to return. Default NA returns the colors as contained in the palette.
opacity	numeric in $[0, 1]$, opacity. Default 1 (opaque). Applied via adjustcolor.
x	palette object to be plotted.
cex	character extension.
border	color for the border.
...	further params.

Value

a character vector of n hex color codes of class c("palette", "character") with a "name" attribute.

See Also

palNames, colorRampPalette, adjustcolor

Other color.palettes: hcol(), palNames()

Examples

```
# default palette from options
pal()

# discrete palette - 3 maximally contrasting colors
pal("dark", n = 3)

# continuous gradient - 50 colors
pal("red-white-blue-2", n = 50)

# by index
pal(1, n = 10)

# with transparency
pal("helsana", n = 5, opacity = 0.5)

# show all names
palNames()
```

palNames *List Available Palette Names*

Description

Returns the names of all palettes available in `pal`, optionally filtered by type.

Usage

```
palNames(type = c("all", "continuous", "discrete"))
```

Arguments

type	character, one of `"all"` (default), `"continuous"`, or `"discrete"`.

Value

a character vector of palette names.

See Also

`pal`

Other color.palettes: `hcol()`, `pal()`

Examples

```
palNames()
palNames("continuous")
palNames("discrete")
```

plot.BlandAltman *Bland-Altman Plot*

Description

Plots a Bland-Altman agreement analysis.

Usage

```
## S3 method for class 'BlandAltman'
plot(
  x,
  main = "",
  xlab = "Mean",
  ylab = "Difference",
  xlim = NULL,
  ylim = NULL,
  pch = 16,
  col = NULL,
```

```
  grid = TRUE,
  meanLine = TRUE,
  limits = TRUE,
  conf.band = FALSE,
  trend = FALSE,
  showText = TRUE,
  stamp = NULL,
  ...
)
```

Arguments

x	an object of class "blandAltman".
main	plot title.
xlab	X-axis label.
ylab	Y-axis label.
xlim	X-axis limits.
ylim	Y-axis limits.
pch	plotting symbol.
col	point colour.
grid	logical; draw a background grid.
meanLine	logical; draw the bias line.
limits	logical; draw the limits of agreement.
conf.band	logical; draw confidence bands.
trend	logical; draw a regression line of differences versus means.
showText	logical; annotate bias and limits.
stamp	optional plot stamp.
...	further arguments passed to plot().

Details

Objects of class "blandAltman" are typically created with DescToolsX::blandAltmanData().

Value

Invisibly returns x.

See Also

Other plot.s3: plot.Desc.qn(), plot.Desc.table(), plot.Lc()

plot.Desc.qn *Plot Method for Numeric-Categorical* Desc *Objects*

Description

Visualises the relationship between a numeric variable and a categorical variable, as computed by `DescToolsX::desc(y ~ x)` (or `x ~ y`) for a numeric/categorical pair. Five panel types are available, selectable (and combinable) via `which`.

Usage

```
## S3 method for class 'Desc.qn'
plot(
  x,
  main = NULL,
  ylab = NULL,
  which = 2,
  verbose = NULL,
  col = .useTheme,
  box = .useTheme,
  legend = TRUE,
  stamp = .useTheme,
  ...
)
```

Arguments

`x`	an object of class `"Desc.qn"`, as returned by `DescToolsX::desc()` for a numeric-categorical pair.
`main`	main title. `NULL` (default) derives a title per panel from the variable names and the panel type (e.g. `"area ~ delivery_min (Spineplot)"`). `""`, `NA`, or `FALSE` suppress the title and compact the top margin. Any other string is used as-is, identically for every selected panel.
`ylab`	y-axis label. `NULL` (default) derives a label specific to each panel (e.g. `"P(y)"` for the conditional density, `"Density"` for the grouped density plot). Supplying a value overrides this for every selected panel.
`which`	integer vector selecting one or more panels to draw, in the given order. One or more of: `1` Conditional density plot (`cdplot`). `2` Spineplot (`spineplot`). Default. `3` Overlapping kernel density curves, one per group (via `plotDens`). `4` Boxplot of the numeric variable by group (via `plotBox`). `5` Prevalence (with Wilson confidence intervals) across quantile bins of the numeric variable. Only available when the categorical variable is binary (`k == 2`); a message is issued and the panel skipped otherwise. Selecting multiple panels does not change the plotting layout (no implicit `mfrow`) - arranging multiple panels on one device is left to the caller (e.g. `par(mfrow = c(2, 1))` beforehand).

verbose — integer; currently computed from x$meta$verbose/getOption("DescTools.verbose") for consistency with other plot.Desc.* methods, but not yet consulted to pick a default which.

col — color specification. .useTheme (default) resolves a panel-appropriate default rather than one shared color, since fill ramps, categorical sets, and single accents are different things:

panels 1/2 a grey ramp from "grey30" to "grey90", sized to the number of categorical levels (not theme-driven by design, to keep the unordered category fill neutral - see plotMosaic for the same rationale).

panels 3/4 pal(getTheme()$palette), the active theme's qualitative palette, sized to the number of levels.

panel 5 getTheme()$twin[1], the active theme's primary accent color.

Supplying col explicitly overrides the default uniformly for every selected panel.

box — controls the plot frame for panels 4 and 5. .useTheme (default) follows the active theme (getTheme()$box); FALSE/NA suppress it; a named list overrides box() arguments. Panels 1/2 (cdplot()/spineplot()) have no effect from this argument - they always draw their native frame unconditionally, with no toggle to override it. Panel 4 (plotBox) always draws its own frame regardless of this argument. Panel 3 (plotDens) never draws a frame, regardless of this argument.

legend — controls the legend for panel 3 (grouped density curves). TRUE (default) draws a legend with the group levels. FALSE/NA suppresses it. A named list overrides arguments forwarded to legend. Has no effect on the other panels.

stamp — controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp, drawn once after all selected panels (panels 3/4 delegate to plotDens/ plotBox, whose own stamp is suppressed internally to avoid a duplicate). TRUE/FALSE/NULL, a string, or a named list for stamp().

... — further graphical parameters, passed to par via the internal framework and to the underlying panel-drawing functions (cdplot(), spineplot(), plotDens, plotBox, or plot, depending on the selected panel).

Details

The left margin is sized automatically from the longest of: the (possibly panel-specific) y-axis labels, and - for panels 1/2 - the categorical level names drawn as axis tick labels, so neither is ever clipped regardless of which.

Value

Invisibly returns x.

See Also

DescToolsX::desc, plotDens, plotBox, cdplot, spineplot

Other plot.s3: plot.BlandAltman(), plot.Desc.table(), plot.Lc()

`plot.Desc.table` *Plot Method for Categorical-Categorical* `Desc` *Objects*

Description

Visualises a (two-dimensional) cross-tabulation, as computed by `DescToolsX::desc()` for a categorical/categorical pair. Four panel types are available, selectable (and combinable) via `which`. Higher-dimensional tables (more than two margins) are not supported; a message is issued and the call returns invisibly.

Usage

```
## S3 method for class 'Desc.table'
plot(
  x,
  main = NULL,
  ylab = NULL,
  which = 1,
  verbose = NULL,
  col = .useTheme,
  box = .useTheme,
  stamp = .useTheme,
  ...
)
```

Arguments

`x`	an object of class `"Desc.table"`, as returned by `DescToolsX::desc()` for a categorical-categorical pair.
`main`	main title. `NULL` (default) derives a title per panel from `x$meta$xname` (the deparsed expression originally passed to `desc()`, e.g. `"table(Pizza$area, Pizza$driver)"`) combined with a panel-type label for context when multiple panels are shown (e.g. `"table(Pizza$area, Pizza$driver) (Spineplot)"`). There is no `y ~ x` pair to draw on here - `x$meta` carries only a single `xname`, since a table built outside a two-sided formula has no separately named "x" and "y" variable. `""`, `NA`, or `FALSE` suppress the title and compact the top margin. Any other string is used as-is, identically for every selected panel.
`ylab`	y-axis label. `NULL` (default) leaves the panel's own default in place (typically empty/unlabeled, since the row dimension of a table built via e.g. `table(a, b)` usually has no name carried in `x$meta`). Supplying a value overrides this for every selected panel.
`which`	integer vector selecting one or more panels to draw, in the given order. One or more of: 1 Spineplot (`spineplot`). Default. 2 Mosaic plot (via `plotMosaic`). 3 Mosaic plot (swapped axis). 4 Association plot (Cohen-Friendly plot) via `plotAssoc`. 5 Heatmap of cell proportions (via `plotHeatmap`, `scale = "prop"`).

Selecting multiple panels does not change the plotting layout (no implicit `mfrow`) - as with other `plot.Desc.*` methods, arranging multiple panels on one device is left to the caller (e.g. `par(mfrow = c(2, 1))` beforehand).

`verbose` integer; currently computed from `x$meta$verbose/getOption("DescTools.verbose")` for consistency with other `plot.Desc.*` methods, but not yet consulted to pick a default `which`.

`col` color specification. `.useTheme` (default) resolves a panel-appropriate default rather than one shared color, since fill ramps, diverging palettes, and sequential heat scales are different things:

panel 1 a grey ramp from `"grey30"` to `"grey90"`, sized to the number of rows of `tab` - the panel draws `spineplot(t(tab))`, so the stacked (filled) dimension is the row dimension of `tab`, not its columns (not theme-driven by design, to keep the unordered category fill neutral).

panel 2 a grey ramp from `"grey30"` to `"grey90"`, sized to the number of columns of `tab` (the fill dimension of the untransposed mosaic), passed to `plotMosaic`.

panel 3 a grey ramp from `"grey30"` to `"grey90"`, sized to the number of rows of `tab` - with `swap = TRUE` the fill dimension is the row dimension, passed to `plotMosaic`.

panel 4 left at `plotAssoc`'s own default (`pal("red-white-blue-3", n = 100)`), a diverging palette - cell colors there encode the sign and strength of Pearson residuals, so a categorical or grey-ramp default would not be meaningful. Supplying `col` overrides this with the diverging palette of the user's choice.

panel 5 left at `plotHeatmap`'s own default (`pal("Blues", n = 100)`), a sequential ramp - cell colors there encode magnitude only. Supplying `col` overrides this.

Supplying `col` explicitly overrides the default uniformly for every selected panel.

`box` controls the plot frame. `.useTheme` (default) follows the active theme (`getTheme()$box`); `FALSE`/`NA` suppress it; a named list overrides frame-drawing arguments.

panel 1 has no effect - `spineplot()` always draws its native frame unconditionally, with no toggle to override it.

panels 2/3 `plotMosaic` always draws its own frame; this argument has no effect.

panel 4 `plotAssoc` has no frame/box concept of its own (it draws dashed reference lines instead); this argument has no effect.

panel 5 forwarded as-is to `plotHeatmap`'s own `box` argument, which draws the outer frame via `rect()` at the exact tile boundaries rather than `box()`.

`stamp` controls the corner stamp. `.useTheme` (default) resolves to `getTheme()$stamp`, drawn once after all selected panels. Panels 2-5 delegate to `plotMosaic`/`plotAssoc`/`plotHeatmap`, whose own `stamp` argument is set to `NA` internally to avoid a duplicate. `TRUE`/`FALSE`/`NULL`, a string, or a named list for `stamp()`.

`...` further graphical parameters, passed to `par` via the internal framework and to the underlying panel-drawing functions (`spineplot()`, `plotMosaic`, `plotAssoc`, or `plotHeatmap`, depending on the selected panel).

Details

The left margin is sized automatically from the longest of: the y-axis label, and - for panels 1/2 - the row names of `tab` drawn as axis tick labels, so neither is ever clipped regardless of `which`.

Only two-dimensional tables are supported. If x carries a table with more than two margins, a message is issued and the function returns invisibly without drawing anything.

Value

Invisibly returns x.

See Also

DescToolsX::desc, plotAssoc, plotHeatmap, plotMosaic, spineplot

Other plot.s3: plot.BlandAltman(), plot.Desc.qn(), plot.Lc()

---

plot.Lc *Plot Methods for Lorenz Curve Objects*

Description

Visualize objects of class "Lc" and "LcList" returned by DescToolsX::lc(). The plot() method draws a new Lorenz curve plot including the line of perfect equality; lines() and points() add to an existing plot.

Usage

```
## S3 method for class 'Lc'
plot(
  x,
  main = NULL,
  xlab = NULL,
  ylab = NULL,
  xlim = NULL,
  ylim = NULL,
  general = FALSE,
  col = NULL,
  line = TRUE,
  points = NULL,
  eqline = TRUE,
  grid = .useTheme,
  box = .useTheme,
  cbandArgs = NA,
  stamp = .useTheme,
  ...
)

## S3 method for class 'Lc'
lines(x, general = FALSE, col = NULL, lwd = 2, lty = 1, cbandArgs = NA, ...)

## S3 method for class 'Lc'
points(x, general = FALSE, pch = 16, col = NULL, ...)

## S3 method for class 'LcList'
lines(x, col = NULL, ...)
```

```
## S3 method for class 'LcList'
points(x, col = NULL, ...)

## S3 method for class 'LcList'
plot(x, col = NULL, general = FALSE, ylim = NULL, ...)
```

Arguments

x	object of class "Lc" (for plot.Lc(), lines.Lc(), points.Lc()) or "LcList" (for the *.LcList() methods).
main, xlab, ylab	main title and axis labels, used by plot.Lc() only. All default to NULL: no title, "p", and "L(p)" ("GL(p)" if general = TRUE), respectively.
xlim, ylim	numeric vectors of length 2 giving axis limits, used by plot.Lc() and plot.LcList(). Default NULL, which resolves to c(0, 1) for xlim and, for ylim, to c(0, 1) for the standard and c(0, max(L)) for the generalized curve.
general	logical. If TRUE, the generalized Lorenz curve (scaled by the mean) is displayed instead of the standard curve. Default is FALSE.
col	color of curve and symbols. For plot.Lc(), lines.Lc() and points.Lc() a single color (default NULL, i.e. "black" in plot.Lc() and the device default in the low-level methods). For the "LcList" methods a vector recycled to the number of groups (default NULL, i.e. seq_len(k)).
line	logical or list, used by plot.Lc() to control drawing of the Lorenz curve. TRUE (default) draws it with package defaults (lty = 1, lwd = 2); FALSE suppresses it (and the confidence band); a list overrides individual defaults and is forwarded to lines.Lc().
points	NULL, logical or list, used by plot.Lc() to control drawing of symbols on the curve. NULL (default) is automatic: symbols are drawn only while the curve has at most getOption("DescToolsX.plot.maxSymbols") knots (100 by default), which keeps large samples legible. TRUE always draws them with package defaults (pch = 21, bg = "white", cex = 1.4); FALSE suppresses them; a list overrides individual defaults and is forwarded to points.Lc().
eqline	logical or list, used by plot.Lc() only. Controls the line of perfect equality: TRUE (default) draws it with package defaults (col = "grey50", lty = 2), FALSE suppresses it, a list is forwarded to abline(). Its slope is 1 for the standard and max(L) for the generalized curve; overriding a/b is possible but rarely sensible.
grid, box	callIf-style specs for the grid and the box around the plot region, used by plot.Lc() only. .useTheme (default) lets getTheme() decide, TRUE/FALSE force drawing/suppression, and a named list is forwarded to grid() resp. box().
cbandArgs	used by plot.Lc() and lines.Lc(). NA to suppress the confidence band (default), or a list of arguments passed to DescToolsX::predict.Lc() to control bootstrap confidence intervals.
stamp	controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/ NULL, a string, or a named list for stamp().
...	further arguments. For plot.Lc(), graphical parameters passed to par() via .applyParFromDots() (e.g. mar, cex.axis, las). For lines.Lc() and points.Lc(), further arguments passed on to lines() and points(), respectively. For plot.LcList(), arguments are passed to plot.Lc() for the first group and, restricted to those the low-level method understands, to lines.Lc() for the remaining ones.

lwd	line width, used by plot.Lc() (via line) and lines.Lc(). Default is 2.
lty	line type, used by plot.Lc() (via line) and lines.Lc(). Default is 1.
pch	plotting symbol, used by points.Lc() only. Default is 16.

Details

For "LcList" objects (grouped Lorenz curves), plot() draws the first group and overlays the remaining groups with lines(). Colors cycle automatically when col is not supplied and are recycled to the number of groups otherwise.

The curve of plot.Lc() is drawn by lines.Lc() and the symbols by points.Lc(), so all three methods share one code path and one set of semantics - including the confidence band, which is controlled by cbandArgs in plot.Lc() exactly as it is in lines.Lc(). Pass a list of arguments to DescToolsX::predict.Lc() to control the bootstrap (e.g. cbandArgs = list(conf.level = 0.90, n = 500)). Set cbandArgs = NA (default) to suppress the band. Note that line = FALSE suppresses the band along with the curve.

With general = TRUE the generalized Lorenz curve is displayed. It ends at the mean rather than at 1, so the default ylim and the slope of the equality line follow the data; for "LcList" objects the panel is sized to accommodate *all* groups, not just the first.

Value

All methods return NULL invisibly.

See Also

DescToolsX::lc() for computing the Lorenz curve, DescToolsX::predict.Lc() for bootstrap confidence intervals, DescToolsX::gini() for the Gini coefficient.

Other plot.s3: plot.BlandAltman(), plot.Desc.qn(), plot.Desc.table()

---

plotArea *Stacked Area Plot*

Description

Draws one or several stacked area series using cumulative polygons. The function accepts either a matrix of values or separate x and y coordinates. Multiple series are displayed as stacked areas.

Usage

```
plotArea(
  x,
  y,
  prop = FALSE,
  col = NULL,
  xlab = "",
  ylab = "",
  xlim = NULL,
  ylim = NULL,
  legend = TRUE,
  main = NULL,
```

```
  grid = TRUE,
  ...
)
```

Arguments

x	numeric vector, matrix or data frame. If y is missing, x is interpreted as a matrix of series where rows correspond to x positions and columns to individual areas.
y	optional numeric vector or matrix giving the y-values. If supplied, x is interpreted as the x-coordinates.
prop	logical indicating whether rows should be converted to proportions so that stacked areas sum to one.
col	fill colours used for the areas.
xlab	label for the x-axis.
ylab	label for the y-axis.
xlim	limits for the x-axis.
ylim	limits for the y-axis.
legend	logical or list controlling the legend. If TRUE, a legend is drawn using the column names of the data. If a list is supplied, its elements are passed to the internal legend drawing routine.
main	main title of the plot.
grid	logical or list controlling the background grid. If TRUE, a default grid is drawn.
...	additional graphical parameters passed to par via .applyParFromDots() and to the plotting functions.

Details

If y is missing, x is interpreted as a matrix and each column is drawn as a separate stacked area.

The cumulative sums are calculated row-wise and displayed as polygons stacked on top of each other.

If prop = TRUE, each row is converted to proportions before plotting, so the stacked areas sum to one.

Row names are used as x-axis labels when available and y is omitted.

Value

Invisibly returns a list containing:

- x the x-values used for plotting,
- y the original y-values,
- cumulative the cumulative values used to construct the areas,
- legend the legend specification if drawn.

See Also

Other plot.univariate: plotBar(), plotBox(), plotCatDist(), plotDens(), plotDensBox(), plotDot(), plotECDF(), plotFdist(), plotLines(), plotQQ(), plotViolin()

Examples

```
plotArea(VADeaths)

plotArea(
  WorldPhones,
  col = pal("helsana")
)

plotArea(
  WorldPhones,
  prop = TRUE,
  col = rainbow(ncol(WorldPhones))
)

x <- 1:20
y <- cbind(
  A = runif(20, 1, 5),
  B = runif(20, 1, 3),
  C = runif(20, 1, 4)
)

plotArea(x, y)
```

---

plotAssoc *Association Plot for Contingency Tables*

Description

Plots an association plot (Cohen-Friendly plot) for a two-dimensional contingency table. Cells are drawn with widths proportional to the square root of expected frequencies and heights proportional to Pearson residuals. Color encodes both the direction and strength of the association using a diverging palette.

Usage

```
plotAssoc(
  x,
  main = NULL,
  xlab = TRUE,
  ylab = TRUE,
  space = 0.3,
  reorder = TRUE,
  col = pal("red-white-blue-3", n = 100L),
  border = NA,
  labels = FALSE,
  stamp = .useTheme,
  ...
)
```

Arguments

x	a two-dimensional contingency table (table or matrix).
main	main title of the plot. NULL (default) derives a title from the expression passed as x (via deparse(match.call()$x)), the same "substitute magic" convention used by plotXY/plotBox for their y ~ x default - there's no formula pair here, just the single table argument, so the default is simply that expression's text (e.g. plotAssoc(tab) titles itself "tab"). "", NA, or FALSE suppress the title entirely and compact the top margin; any other string is used as given (resolved internally via .resolveTitle()).
xlab	character, x-axis label. Defaults to the first dimension name.
ylab	character, y-axis label. Defaults to the second dimension name.
space	numeric, fraction of average cell width/height used as gap between cells. Default 0.3.
reorder	logical. If TRUE (default), rows and columns are reordered by the strength of the strongest association (max(|residual|)) in descending order.
col	character vector of colors for the diverging palette. Default uses pal("red-white-blue-3", n = 100) from the DescToolsX theme. Negative residuals map to the first color, zero to the middle, positive to the last.
border	the color of the border
labels	logical or character. If TRUE, Pearson residuals are printed inside each cell. If FALSE (default), no labels are shown. A character format string (e.g. "%.1f") can also be passed for custom formatting.
stamp	controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/ NULL, or an explicit string, as for .withGraphicsState() (internal).
...	further arguments passed to rect.

Details

The plot is based on the association plot described in Cohen (1980) and Friendly (1992). Each cell (i, j) is represented by a rectangle:

- **width** proportional to $\sqrt{e_{ij}}$ (square root of expected frequency)
- **height** proportional to the Pearson residual $d_{ij} = (f_{ij} - e_{ij})/\sqrt{e_{ij}}$

A horizontal baseline at zero represents independence. Cells above the baseline indicate positive association, cells below negative association.

Color encodes both direction and magnitude: the diverging palette runs from the negative color (strong negative residual) through white (no association) to the positive color (strong positive residual).

References

Cohen, A. (1980). On the graphical display of the significant components of a two-way contingency table. *Communications in Statistics — Theory and Methods*, 9, 1025–1041.

Friendly, M. (1992). Graphical methods for categorical data. *SAS User Group International Conference Proceedings*, 17, 190–200.

See Also

graphics::mosaicplot, DescToolsX::conf

Other plot.bivariate: plotBag(), plotCor(), plotDens2D(), plotHeatmap(), plotHexbin(), plotMosaic(), plotXY()

Examples

```
tab <- table(bedrock::Pizza$driver, bedrock::Pizza$area)

# default
plotAssoc(tab)

# custom palette
plotAssoc(tab, col = pal("red-white-green", n = 100))

# with residual labels
plotAssoc(tab, labels = TRUE)

# no reordering
plotAssoc(tab, reorder = FALSE)


plotAssoc(tab,
          main = "Association Hair ~ Eye",
          cutoff = 1,
          xlab="Hair Color", ylab="Eye Color")

cols <- pal()[c(12, 8)]
plotAssoc(tab,
          main = "Association Hair ~ Eye",
          cutoff = 1,
          col = fade(cols, 0.7), border = cols,
          reorder = TRUE, cex.axis = 0.9,
          xlab = list(labels = "Hair Color ",
                      col = "#5B2A45", cex = 1.1),
          ylab = NA, labels = TRUE)
```

---

plotBag *Create a Bagplot (Bivariate Boxplot)*

Description

Draws a bagplot (bivariate boxplot) based on halfspace (Tukey) depth. The bag contains the innermost 50\ hull of all non-outlying points, and points outside the fence (the bag inflated by factor, not drawn) are flagged as outliers.

Usage

```
plotBag(x, ...)

## S3 method for class 'formula'
plotBag(
```

```
  x,
  data = NULL,
  subset,
  na.action = na.omit,
  main = "",
  xlab = NULL,
  ylab = NULL,
  ...
)

## Default S3 method:
plotBag(
  x,
  main = "",
  xlab = "",
  ylab = "",
  xlim = NULL,
  ylim = NULL,
  factor = 3,
  eps = 0.00000001,
  dither = TRUE,
  points = TRUE,
  bag = TRUE,
  loop = TRUE,
  fence = FALSE,
  out = TRUE,
  median = TRUE,
  grid = FALSE,
  box = TRUE,
  stamp = NULL,
  ...
)
```

Arguments

x	a numeric matrix or data frame with exactly two columns, or a formula of the form y ~ x (see the formula method).
...	additional graphical parameters passed to par().
data	an optional data frame containing the variables in formula.
subset	an optional expression indicating which observations to use.
na.action	a function specifying how missing values are handled. Defaults to na.omit, as the depth computation requires complete pairs.
main, xlab, ylab	character strings for plot annotations. The formula method derives xlab and ylab from the variable names if they are not supplied.
xlim, ylim	numeric vectors of length 2 specifying axis limits.
factor	inflation factor for the fence (default: 3).
eps	numeric tolerance used in depth computation and geometry.
dither	logical, whether to add small noise to break ties.
points, bag, loop, fence, out, median, grid, box	object-oriented control of plot elements (see Details).

stamp	optional stamp passed to .withGraphicsState().
formula	a formula of the form y ~ x, where both variables are numeric. x is drawn on the horizontal, y on the vertical axis.

Details

All graphical elements are controlled via an object-oriented interface: each element can be specified as TRUE, FALSE, or a list(...) of graphical parameters. Internally, this is handled via bedrock::callIf().

The construction follows Rousseeuw, Ruts and Tukey (1999):

1. The halfspace (Tukey) depth of every observation is computed using a direct port of the original Fortran routine TUKDEPTH (Rousseeuw & Ruts, 1996).
2. The Tukey median is approximated by the mean of all observations with maximal depth.
3. The *bag* is obtained by radial interpolation between the convex hulls of two adjacent depth regions, calibrated such that it contains $\lfloor n/2 \rfloor$ observations (up to ties on the polygon boundary).
4. The *fence* is the bag inflated by factor relative to the Tukey median. Following the original proposal it is used for classification only and not drawn by default.
5. Observations outside the fence are flagged as *outliers*.
6. The *loop* is the convex hull of all non-outlying observations, so it always lies within the data range.

Two approximations remain relative to the strict theory: the Tukey median is not computed via HALFMED, and the bag interpolates hulls of sample depth regions rather than exact isodepth contours. Borderline outlier classifications may therefore differ slightly from other implementations (e.g. **aplpack**).

Exact ties and collinear configurations violate the general-position assumption of the depth algorithm; dither (default TRUE) adds negligible noise (order eps) to break them.

Value

Invisibly returns a list of class "bagplot" with components:

- center - Tukey median.
- depth - depth of the innermost region bounding the bag.
- bag - bag polygon (matrix).
- fence - fence polygon (matrix, not drawn by default).
- loop - loop polygon (matrix).
- outliers - outlier points (matrix).
- depths - halfspace depth of all observations.

Element Control

Each of the following arguments accepts:

- TRUE - draw element with defaults
- FALSE - suppress element
- list(...) - customize graphical parameters

Supported elements (in drawing order):

- `grid` - background grid
- `fence` - classification boundary (default `FALSE`)
- `loop` - convex hull of the non-outlying points
- `bag` - central 50\
- `points` - raw data points
- `out` - outliers
- `median` - Tukey median
- `box` - plot frame

References

P. J. Rousseeuw, I. Ruts, J. W. Tukey (1999): The bagplot: a bivariate boxplot, *The American Statistician*, vol. 53, no. 4, 382–387.

P. J. Rousseeuw, I. Ruts (1996): Algorithm AS 307: Bivariate location depth, *Applied Statistics*, vol. 45, no. 4, 516–526.

See Also

Other plot.bivariate: `plotAssoc()`, `plotCor()`, `plotDens2D()`, `plotHeatmap()`, `plotHexbin()`, `plotMosaic()`, `plotXY()`

Examples

```
set.seed(1)
x <- cbind(rnorm(200), rnorm(200))

# data of Rousseeuw et al. (1999): car weight vs engine displacement
cardata <- data.frame(
  Weight = c(2560, 2345, 1845, 2260, 2440,
             2285, 2275, 2350, 2295, 1900, 2390, 2075, 2330, 3320, 2885,
             3310, 2695, 2170, 2710, 2775, 2840, 2485, 2670, 2640, 2655,
             3065, 2750, 2920, 2780, 2745, 3110, 2920, 2645, 2575, 2935,
             2920, 2985, 3265, 2880, 2975, 3450, 3145, 3190, 3610, 2885,
             3480, 3200, 2765, 3220, 3480, 3325, 3855, 3850, 3195, 3735,
             3665, 3735, 3415, 3185, 3690),
  Disp = c(97, 114, 81, 91, 113, 97, 97,
           98, 109, 73, 97, 89, 109, 305, 153, 302, 133, 97, 125, 146,
           107, 109, 121, 151, 133, 181, 141, 132, 133, 122, 181, 146,
           151, 116, 135, 122, 141, 163, 151, 153, 202, 180, 182, 232,
           143, 180, 180, 151, 189, 180, 231, 305, 302, 151, 202, 182,
           181, 143, 146, 146)
)

plotBag(x)

# Custom styling
plotBag(x,
  bag = list(col = adjustcolor("green", 0.3), border = "darkgreen"),
  loop = list(border = "black", lty = 3),
  out = list(col = "red", pch = 17),
  grid = TRUE
```

```
)

# Minimal plot
plotBag(x, points = FALSE, median = FALSE)

# formula interface
plotBag(Disp ~ Weight, data = cardata)

# example of Rousseeuw et al. (1999): car weight vs engine displacement
plotBag(cardata, xlab = "Weight", ylab = "Displacement")
```

`plotBar` *Themed Barplot with Grid, Labels and Optional Connecting Lines*

Description

Creates a themed wrapper around `barplot` with support for consistent styling, optional grid lines, value labels, and connecting lines for stacked barplots.

Usage

```
plotBar(
  height,
  main = NULL,
  xlab = NULL,
  ylab = NULL,
  yax = NULL,
  beside = FALSE,
  horiz = FALSE,
  col = .useTheme,
  border = .useTheme,
  grid = .useTheme,
  box = FALSE,
  text = NULL,
  connlines = NULL,
  stamp = .useTheme,
  ...
)
```

Arguments

`height`	A vector or matrix of bar heights passed directly to `barplot`.
`main`, `xlab`, `ylab`	optional plot labels. Defaults follow base graphics behaviour.
`yax`	controls drawing of the numeric axis. Supported values are `TRUE` draw axis using package defaults `FALSE` suppress axis `NULL` do not draw a numeric axis at all `list(...)` custom axis parameters passed to the axis drawing routine

beside	logical. If TRUE, bars are drawn side-by-side. If FALSE (default), bars are stacked.
horiz	logical. If TRUE, bars are drawn horizontally. Defaults to FALSE.
col	bar fill colours. .useTheme (default) resolves to getTheme()barcol.
border	bar border colour. .useTheme (default) resolves to getTheme()barborder.
grid	controls drawing of grid lines. Can be: • .useTheme (default): follow the active theme (getTheme()$grid), restricted to the axis perpendicular to the value axis (e.g. horizontal lines only for vertical bars) • TRUE: draw grid with theme settings • FALSE, NULL, or NA: suppress grid • a named list: arguments passed to grid, overriding the theme/function defaults for this call only
box	controls drawing of the plot box. .useTheme (default) resolves to getTheme()$box. TRUE/FALSE/NA, or a named list, as for grid.
text	optional list of arguments passed to barText to draw value labels on bars.
connlines	optional list of arguments controlling connecting lines between stacked bars. Only supported when beside = FALSE.
stamp	controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/NULL, or an explicit string, as for .withGraphicsState() (internal).
...	additional arguments passed to barplot and graphical parameters (via par).

Details

The function first initializes the plotting region invisibly using barplot, optionally adds grid lines, and then draws the actual bars and additional layers (axis, connecting lines, text labels).

The function internally performs the following steps:

1. Draws an invisible barplot to establish the coordinate system.
2. Optionally adds grid lines.
3. Draws the bars.
4. Draws the numeric axis if enabled.
5. Optionally adds connecting lines for stacked bars.
6. Optionally adds value labels via barText.
7. Optionally draws a box around the plot region.

Graphical parameters such as bg, cex, las, mar, etc. can be supplied via ....

The precedence of theme-aware settings (col, border, grid, box, stamp) is

```
explicit argument  >  function-specific default  >  active theme (getTheme())
```

Value

Invisibly returns the midpoints of the bars as returned by barplot.

See Also

graphics::barplot, barText, theme

Other plot.univariate: plotArea(), plotBox(), plotCatDist(), plotDens(), plotDensBox(), plotDot(), plotECDF(), plotFdist(), plotLines(), plotQQ(), plotViolin()

Examples

```
# Simple barplot
plotBar(1:5)

# With grid lines
plotBar(1:5, grid = TRUE)

# Stacked bars with labels and connecting lines
m <- matrix(c(3,2,4,1,5,2), nrow = 2)
plotBar(m,
        text = list(pos = "mid"),
        connlines = list(col = "black"))

# Grouped bars
plotBar(VADeaths,
        beside = TRUE,
        col = gray.colors(nrow(VADeaths)))

# Horizontal bars
plotBar(VADeaths,
        horiz = TRUE,
        las = 1)


plotBar(VADeaths, ylim=c(0,250),
        grid=list(col = "grey", lty="dotted"),
        las=1, main="MyTitle",
        text = list(labels=VADeaths,
        border = NA, srt=45, bg="navajowhite"))

plotBar(VADeaths, ylim=c(0,80),
        las=1, main="MyTitle",
        box=FALSE,
        col=gray.colors(nrow(VADeaths)),
        beside=TRUE,
        text = list(col="red", bg=addOpacity("white", 0.7), border=NA))

plotBar(VADeaths, connlines = list(lwd=1, col="blue"),
        box=FALSE, las=1, main="Connecting Lines")

ptab <- proportions(VADeaths, margin=2)
plotBar(ptab,
        las=1, main="VADeaths in %",
        box=FALSE, horiz=TRUE,
        col=(cols <- gray.colors(nrow(VADeaths))),
        beside=FALSE, mar=c(right=5),
        text = list(labels=fm(ptab, fmt="%"), border=NA,
                    col=contrastColor(cols)))
legend(x="right", fill=cols, legend=rownames(VADeaths))
```

```
plotBar(VADeaths/1e3,  box=FALSE, bg="lightyellow", main="VADeaths",
        horiz=TRUE,
        text=list(border=FALSE, cex=0.8, col=c("blue", "green","orange")),
        mar=c(right=5),
        yax = list(fmt="%", d=0, big=",",
                   col="red", col.axis="blue", lwd=2))
```

`plotBinaryTree` *Binary Tree*

Description

Create a binary tree of a given number of nodes `n`. Can be used to organize a sorted numeric vector as a binary tree.

Usage

```
plotBinaryTree(
  x,
  main = "Binary tree",
  horiz = FALSE,
  text = TRUE,
  line = TRUE,
  ...
)
```

Arguments

`x`	numeric vector to be organized as binary tree.
`main`	main title of the plot.
`horiz`	logical, should the plot be oriented horizontally or vertically (default).
`text`	properties of the text, can be any of the arguments of `boxedText` (besides geometry and label).
`line`	properties of the line segments (`col`, `lwd`, `lty`).
`...`	the dots are sent to `canvas`.

Details

If we index the nodes of the tree as 1 for the top, 2–3 for the next horizontal row, 4–7 for the next, ... then the parent-child traversal becomes particularly easy. The basic idea is that the rows of the tree start at indices 1, 2, 4, ...

binaryTree(13) yields the vector `c(8, 4, 9, 2, 10, 5, 11, 1, 12, 6, 13,3, 7)` meaning that the smallest element will be in position 8 of the tree, the next smallest in position 4, etc.

Value

an integer vector of length n

Note

Substantially based on code by Terry Therneau, with major extensions and improvements by the package author.

See Also

Other plot.special: plotCirc(), plotLift(), plotMiss(), plotPolar(), plotPropCI(), plotTernary(), plotTimeSeries(), plotTreemap(), plotWeb()

Examples

```
bedrock::binaryTree(12)

x <- sort(sample(100, 24))
z <- plotBinaryTree(x, cex=0.8)

plotBinaryTree(LETTERS[1:15],
               text=list(col="deeppink4", cex=2),
               line=list(col="navajowhite3", lwd=2))


# Plot example - Titanic data, for once from a somwhat different perspective
tab <- apply(Titanic, c(2,3,4), sum)
cprob <- c(1, prop.table(apply(tab, 1, sum))
           , as.vector(aperm(prop.table(apply(tab, c(1,2), sum), 1), c(2, 1)))
           , as.vector(aperm(prop.table(tab, c(1,2)), c(3,2,1)))
)

plotBinaryTree(round(cprob[bedrock::binaryTree(length(cprob))],2), horiz=TRUE, cex=0.8,
            main="Titanic")
text(c("sex","age","survived"), y=0, x=c(1,2,3)+1)
```

---

plotBox *Grouped Boxplot*

Description

Draws boxplots for a numeric variable, optionally grouped by a categorical variable. Group means and a reference line for the overall mean can optionally be overlaid.

Usage

```
plotBox(x, ...)

## Default S3 method:
plotBox(
  x,
  g = NULL,
  main = NULL,
  xlab = "",
  ylab = "",
  ylim = NULL,
```

```
  col = NULL,
  grid = TRUE,
  means = TRUE,
  stamp = TRUE,
  ...
)

## S3 method for class 'formula'
plotBox(
  formula,
  data,
  subset,
  na.action = na.omit,
  main = NULL,
  xlab = "",
  ylab = "",
  ylim = NULL,
  col = NULL,
  grid = TRUE,
  stamp = TRUE,
  ...
)
```

Arguments

x	numeric vector, or a formula of the form x ~ g.
...	graphical parameters. Parameters recognized by the internal graphics framework are applied via par(); remaining arguments are forwarded to boxplot.
g	optional grouping variable (ignored if a formula is used).
main	main title of the plot.
xlab	label for the x-axis.
ylab	label for the y-axis.
ylim	numeric vector of length 2 specifying the y-axis limits. If NULL (default), the range of x is used.
col	vector of colors. If NULL, a palette is generated automatically.
grid	controls drawing of the background grid. Can be: • TRUE: draw grid with default settings • FALSE, NULL, or NA: suppress grid • a named list: arguments passed to grid, e.g. list(col = "red", nx = NA, ny = NULL) for vertical lines only
means	controls drawing of group means and an overall mean reference line. Can be: • TRUE: draw with default settings • FALSE, NULL, or NA: suppress • a named list: arguments passed to the internal means function. Supported arguments: col, pch, cex, lcol, lty, lwd.
stamp	controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/NULL, or an explicit string, as for .withGraphicsState() (internal).

formula	A formula of the form y ~ group.
data	an optional data frame containing variables in the formula.
subset	optional expression indicating which observations to use.
na.action	a function specifying how missing values are handled.

Details

Optional plot components are controlled using callIf semantics:

- TRUE: draw with defaults
- FALSE, NULL, or NA: suppress component
- named list: customize component arguments

Value

Invisibly returns NULL.

See Also

boxplot, callIf

Other plot.univariate: plotArea(), plotBar(), plotCatDist(), plotDens(), plotDensBox(), plotDot(), plotECDF(), plotFdist(), plotLines(), plotQQ(), plotViolin()

Examples

```
## Not run:
set.seed(1)
x <- rnorm(100)
g <- sample(c("A", "B"), 100, TRUE)

plotBox(x)
plotBox(x, g)

plotBox(x ~ g)

## End(Not run)
```

---

plotBubble *Bubble Plot*

Description

Draws a bubble plot where the position is given by x and y, and the size of each bubble is proportional to area.

Usage

```
## Default S3 method:
plotBubble(
  x,
  y,
  area,
  ...,
  add = FALSE,
  col = NA,
  border = NULL,
  cex = 1,
  grid = NA,
  xlim = NULL,
  ylim = NULL,
  na.rm = FALSE,
  main = "",
  xlab = "",
  ylab = ""
)

## S3 method for class 'formula'
plotBubble(
  formula,
  data = NULL,
  subset,
  na.action = na.omit,
  ...,
  add = FALSE,
  col = NA,
  border = NULL,
  cex = 1,
  grid = NA,
  xlim = NULL,
  ylim = NULL,
  main = "",
  xlab = "",
  ylab = ""
)
```

Arguments

x	numeric vector of x positions.
y	numeric vector of y positions.
area	numeric vector controlling bubble sizes (interpreted as area).
...	additional graphical parameters passed to par().
add	logical; if TRUE, adds to an existing plot.
col	fill color(s) of the bubbles.
border	border color(s) of the bubbles.
cex	scaling factor applied to bubble areas.
grid	logical, NA, or list controlling background grid.

xlim, ylim	axis limits.
na.rm	logical; remove missing values.
main, xlab, ylab	plot labels.
formula	A formula of the form y ~ x | area.
data	optional data frame.
subset	optional subset expression.
na.action	function to handle missing values.

Details

The function supports both a default interface and a formula interface of the form y ~ x | area.

Bubble sizes are interpreted as areas and internally converted to radii via $r = \sqrt{area/\pi}$. Aspect ratio is corrected to ensure visually accurate circles.

Graphical elements such as grids are controlled via the unified plot design system using bedrock::callIf() and .theme().

Value

Invisibly returns NULL.

See Also

symbols

Examples

```
set.seed(1)
x <- rnorm(100)
y <- rnorm(100)
a <- runif(100, 1, 10)

plotBubble(x, y, a)

df <- data.frame(x = x, y = y, a = a)
plotBubble(y ~ x | a, data = df)

US <- data.frame(state.x77, State=state.name,
                  Region=state.region, Abb=state.abb)
plotBubble(Income ~ Population | Area ,
           data=US,
           grid=TRUE, col=addOpacity(pal("helsana")[US$Region]), cex=1.2 )

text(Income ~ Population, US, labels=US$Abb, cex=0.8)
```

plotCatDist *Categorical Distribution Plot*

Description

Visualizes the distribution of a categorical variable using horizontal bar plots of absolute and relative frequencies. Optionally, cumulative proportions (ECDF-style) can be displayed.

Usage

```
plotCatDist(
  x,
  type = c("both", "freq", "perc"),
  ecdf = FALSE,
  col = "grey80",
  border = FALSE,
  maxlablen = 25,
  maxcats = NULL,
  main = NULL,
  ...
)
```

Arguments

x	a factor or character vector, a one-dimensional table of counts, or a numeric vector of precomputed frequencies. Unnamed numeric frequency vectors are labelled by their positions.
type	character; one of "both", "freq", "perc". Controls whether absolute frequencies, relative frequencies, or both are displayed.
ecdf	logical; if TRUE, cumulative proportions are shown instead of simple relative frequencies.
col	fill color for bars.
border	logical; draw borders around bars.
maxlablen	integer; maximum length of category labels before truncation.
maxcats	optional maximum number of categories to display (truncates if exceeded).
main	plot title.
...	further graphical parameters passed to par().

Details

The function produces horizontal bar plots:

- Absolute frequencies (counts)
- Relative frequencies (percentages) or cumulative proportions

If type = "both", both views are shown side by side.

Long labels are truncated, and large category sets can be limited via maxcats.

Raw categorical data and their pre-tabulated form are treated identically. When categories are truncated via maxcats, proportions remain based on the total frequency before truncation.

Value

Invisibly returns a list with frequencies and proportions.

See Also

Other plot.univariate: plotArea(), plotBar(), plotBox(), plotDens(), plotDensBox(), plotDot(), plotECDF(), plotFdist(), plotLines(), plotQQ(), plotViolin()

Examples

```
# Basic usage
x <- factor(sample(letters[1:5], 100, TRUE))
plotCatDist(x)
plotCatDist(table(x))

# Only proportions
plotCatDist(x, type = "perc")

# With cumulative distribution
plotCatDist(x, ecdf = TRUE)

# Many categories (truncation)
x2 <- factor(sample(letters, 200, TRUE))
plotCatDist(x2, maxcats = 10)
```

---

plotCirc *Circular Chord Diagram*

Description

Draws a circular chord diagram from a matrix, showing flows between rows and columns using sectors and ribbons. The plot is rendered using base graphics and supports flexible styling via object-based arguments.

Usage

```
plotCirc(
  x,
  sector = TRUE,
  ribbon = TRUE,
  labels = TRUE,
  gap = 5,
  main = NULL,
  ...
)
```

Arguments

x	A numeric matrix. Rows and columns define the connections between sectors.
sector	sector styling. Can be: • a logical (TRUE/FALSE) to enable/disable sectors,

- a vector of colors,
- or a list with elements `col` and `border`.

Colors are recycled to match the number of sectors (`nrow(x) + ncol(x)`).

`ribbon` ribbon styling. Can be:

- a logical (`TRUE`/`FALSE`) to enable/disable ribbons,
- a vector of colors,
- or a list with elements `col` and `border`.

Colors are recycled to match the number of row categories (`nrow(x)`).

`labels` label styling. Can be:

- a logical (`TRUE`/`FALSE`) to enable/disable labels,
- a character vector of labels,
- or a list with parameters passed to internal label drawing.

`gap` numeric. Gap between sectors in degrees.

`main` character. Main title of the plot.

`...` additional graphical parameters passed to internal plotting functions.

Details

The function constructs a circular layout where:

- Columns of `x` are placed on one half of the circle
- Rows of `x` are placed on the opposite half
- Ribbon widths are proportional to matrix entries

Sector sizes correspond to marginal sums of the matrix.

Internally, angles are computed in radians and mapped to Cartesian coordinates.

Value

Invisibly returns a list with label positions:

x x-coordinates of labels

y y-coordinates of labels

See Also

Other plot.special: `plotBinaryTree()`, `plotLift()`, `plotMiss()`, `plotPolar()`, `plotPropCI()`, `plotTernary()`, `plotTimeSeries()`, `plotTreemap()`, `plotWeb()`

Examples

```
set.seed(1)
x <- matrix(sample(1:10, 36, replace = TRUE), nrow = 6)
rownames(x) <- LETTERS[1:6]
colnames(x) <- LETTERS[1:6]

plotCirc(x)

# Custom colors
plotCirc(
```

```
  x,
  sector = list(col = rainbow(12), border = "grey50"),
  ribbon = list(col = rainbow(6), border = NA)
)

# Custom labels
plotCirc(
  x,
  labels = list(cex = 0.8, col = "blue", las = 2)
)
```

---

plotCor *Correlation Matrix Plot with Theming and Optional Labels*

Description

Draws a correlation matrix using `image` with optional clustering, triangular display, grid lines, color legend, and numeric labels inside the cells.

Usage

```
plotCor(
  x,
  main = NULL,
  xlab = NULL,
  ylab = NULL,
  xax = TRUE,
  yax = TRUE,
  cluster = FALSE,
  mincor = 0,
  triangle = c("full", "upper", "lower"),
  diag = TRUE,
  col = .useTheme,
  grid = .useTheme,
  box = .useTheme,
  legend = TRUE,
  text = FALSE,
  ...
)
```

Arguments

`x`	A numeric correlation matrix.
`main`, `xlab`, `ylab`	optional plot labels.
`xax`, `yax`	controls drawing of the axes. Supported values are `TRUE` draw axis using default settings `FALSE` suppress axis `list(...)` custom axis parameters passed to `axis`

cluster	logical; if TRUE, variables are reordered by hierarchical clustering to place similar correlations together.
mincor	numeric threshold; correlations with absolute value smaller than this are suppressed (set to NA).
triangle	which part of the matrix to display. One of "full", "upper", or "lower".
diag	logical; should the diagonal be displayed.
col	color palette used for the correlation values. .useTheme (default) builds a diverging ramp from getTheme()$twin - the active theme's two-color pair - through white: twin[1] at the negative end (-1), white at zero, twin[2] at the positive end ($+1$).
grid	controls drawing of cell-separator grid lines at the half-integer matrix boundaries (clipped to the matrix extent, so they never bleed into the margins). Can be: • .useTheme (default): follow the active theme (getTheme()$grid$col/$lwd) • TRUE: draw with theme settings • FALSE, NULL, or NA: suppress • a named list: override col/lwd for this call only
box	controls drawing of the plot box. .useTheme (default) resolves to getTheme()$box. TRUE/FALSE/NA, or a named list, as for grid.
legend	logical; draw a color legend for the correlation scale.
text	controls numeric labels drawn inside the matrix cells. Supported values are FALSE no labels TRUE default labels based on the correlation values list(...) custom parameters passed to the internal text drawing routine
...	additional graphical parameters passed to par and image.

Details

The function follows the DescToolsX plotting conventions:

- User arguments override theme settings.
- Theme settings override base graphics defaults.
- Graphical parameters (e.g. cex, las, mar) can be supplied via ....

The function internally:

1. Optionally reorders the matrix using hierarchical clustering.
2. Masks parts of the matrix according to triangle and diag.
3. Adjusts plot margins based on label sizes.
4. Draws the matrix using image.
5. Optionally adds grid lines, numeric labels, axes, and a color legend.

Grid lines are drawn via clipped abline() calls at the matrix's half-integer cell boundaries rather than via grid(): grid()'s nx/ny divide the full plot region (par("usr")), which may carry axis padding unrelated to image()'s integer cell geometry, whereas the clipped approach stays exact regardless of that padding.

Value

Invisibly returns the (possibly reordered) matrix used for plotting.

See Also

image, cor, theme

Other plot.bivariate: plotAssoc(), plotBag(), plotDens2D(), plotHeatmap(), plotHexbin(), plotMosaic(), plotXY()

Examples

```
m <- cor(swiss)

# full correlation matrix
plotCor(m, legend=FALSE)

# upper triangle only
plotCor(m, triangle = "upper")

# clustered variables
plotCor(m, cluster = TRUE)

# with correlation values
plotCor(m, text = TRUE)

# customized labels
plotCor(m,
         text = list(col = "black", cex = 0.9))

# hide grid
plotCor(m, grid = FALSE)

plotCor(m, cols=colorRampPalette(c("red", "black", "green"), space = "rgb")(20))
plotCor(m, cols=colorRampPalette(c("red", "black", "green"), space = "rgb")(20),
         args.colLegend=NA)

m <- cor(mtcars)
plotCor(m, col=pal("red-white-blue-1", 100), border="grey",
         args.colLegend=list(labels=format(seq(-1,1,.25), digits=2), frame="grey"))

# display only correlation with a value > 0.7
plotCor(m, mincor = 0.7)
x <- matrix(rep(1:ncol(m),each=ncol(m)), ncol=ncol(m))
y <- matrix(rep(ncol(m):1,ncol(m)), ncol=ncol(m))
txt <- format(m, digits=3)
idx <- upper.tri(matrix(x, ncol=ncol(m)), diag=FALSE)

# place the text on the upper triagonal matrix
text(x=x[idx], y=y[idx], label=txt[idx], cex=0.8, xpd=TRUE)

# put similiar correlations together
plotCor(m, clust=TRUE)

# same as
idx <- order.dendrogram(as.dendrogram(
          hclust(dist(m), method = "mcquitty")
```

```
       ))
plotCor(m[idx, idx])

# plot only upper triangular matrix and move legend to bottom
m <- cor(mtcars)
m[lower.tri(m, diag=TRUE)] <- NA

# get the p-values
p <- outer(
  (vars <- colnames(mtcars)), vars,
  Vectorize(function(v1, v2)
    cor.test(mtcars[[v1]], mtcars[[v2]], method = "pearson")$p.value
  )
)
dimnames(p) <- list(vars, vars)
m[p > 0.05] <- NA

plotCor(m, mar=c(8,8,8,8), yaxt="n",
         args.colLegend = list(x="bottom", inset=-.15, horiz=TRUE,
                               height=abs(lineToUser(line = 2.5, side = 1)),
                               width=ncol(m)))
mtext(text = rev(rownames(m)), side = 4, at=1:ncol(m), las=1, line = -5, cex=0.8)
```

---

plotDens *Grouped Density Plot*

Description

Draws kernel density estimates for one or more groups. Supports both classical density plots and conditional density plots.

Usage

```
plotDens(x, ...)

## S3 method for class 'formula'
plotDens(
  formula,
  data,
  subset,
  na.action = na.omit,
  ...,
  main = NULL,
  xlab = "",
  ylab = NULL,
  xlim = NULL,
  ylim = NULL,
  add = FALSE,
  bw = "nrd0",
  type = NULL,
  col = NULL,
  lwd = 2,
```

```
  lty = 1,
  fill = FALSE,
  grid = NA,
  stamp = TRUE
)
```

Arguments

x	A numeric vector or list of numeric vectors.
...	additional data vectors (unnamed, default method) or graphical parameters passed to par().
formula	A formula of the form y ~ group, y ~ x (x numeric, conditional density), or y ~ x | group.
data	optional data frame.
subset	optional subset expression.
na.action	function to handle missing values.
main, xlab, ylab	plot labels.
xlim, ylim	axis limits.
add	logical; if TRUE, adds to an existing plot.
bw	bandwidth passed to density or cdplot.
type	character string specifying the plot type. One of "density", "conditional", or NULL (default, determined by resolveFormula()'s design classification).
col	line color(s).
lwd	line width(s).
lty	line type(s).
fill	for type = "density": FALSE (default, no fill), TRUE (translucent fill derived from each group's col via adjustcolor(col, alpha.f = 0.3)), or one or more explicit fill colors recycled over groups. For type = "conditional" on a single, unstratified, binary curve: TRUE for cdplot-style grey shading, or a vector of 2 colors for the regions below/above the boundary curve.
grid	logical, NA, or list controlling background grid.
stamp	controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/NULL, or an explicit string, as for .withGraphicsState() (internal).

Details

The function defers entirely to resolveFormula()'s design classification to pick a mode when type = NULL:

- y ~ g (g categorical) → density, one curve per group.
- y ~ x (x numeric) → conditional density $P(Y|X)$, a single curve - equivalent to cdplot(x, factor(y)).
- y ~ x | g → conditional density, one curve per level of g.

type can be set explicitly to override the default for a given design (e.g. to force an error rather than silently doing the wrong thing if a formula's shape is ambiguous).

Graphical elements such as grids are controlled via the unified plot design system using bedrock::callIf() and .theme().

Value

Invisibly returns NULL.

See Also

density, cdplot, resolveFormula

Other plot.univariate: plotArea(), plotBar(), plotBox(), plotCatDist(), plotDensBox(), plotDot(), plotECDF(), plotFdist(), plotLines(), plotQQ(), plotViolin()

Examples

```
set.seed(1)
x <- rnorm(100)
g <- rep(c("A", "B"), each = 50)

# standard density (k = 2 groups)
plotDens(x ~ g)

# conditional density, single curve - auto-detected, no type= needed
y <- rbinom(100, 1, plogis(x))
plotDens(y ~ x)

# same, with cdplot-style fill
plotDens(y ~ x, fill = c("red", "blue"))

# conditional density, stratified by group
plotDens(y ~ x | g)
```

---

plotDens2D *Two-Dimensional Kernel Density Plot*

Description

Computes a two-dimensional kernel density estimate and visualises it using contour, image, or perspective plots.

Usage

```
plotDens2D(
  x,
  y,
  main = NULL,
  xlab = NULL,
  ylab = NULL,
  xlim = NULL,
  ylim = NULL,
  type = c("contour", "image", "persp"),
  col = rev(pal("red-black", n = 100)),
  grid = .useTheme,
  box = .useTheme,
  ...
)
```

Arguments

x	numeric vector of x-coordinates.
y	numeric vector of y-coordinates. Must have the same length as x.
main	optional main title of the plot.
xlab, ylab	axis labels.
xlim, ylim	numeric vectors of length two specifying axis limits.
type	character string specifying the plot type. One of "contour", "image", or "persp".
col	color specification used for type = "image". Defaults to a reversed "red-black" sequential ramp (pal()), running from black (low density) to red (high density) - hardcoded rather than theme-driven, since this is a continuous, unidirectional gradient, unlike the active theme's categorical palette or diverging twin pair, neither of which fits a density surface.
grid	controls drawing of the background grid. .useTheme (default) follows the active theme (getTheme()$grid). TRUE/FALSE/NA, or a named list, as for grid.
box	controls drawing of the plot box. .useTheme (default) resolves to getTheme()$box. TRUE/FALSE/NA, or a named list, as for box.
...	additional graphical parameters passed to underlying plotting functions.

Details

The function estimates a bivariate density surface using a Gaussian kernel with bandwidths determined via a normal reference rule. The density is evaluated on a regular grid and visualised using one of three base graphics representations:

- "contour": contour lines of equal density
- "image": raster representation of the density surface
- "persp": three-dimensional perspective plot

The choice of representation affects interpretability: contour and image plots emphasise structure in the data distribution, while perspective plots highlight global shape but may distort local density.

Bandwidth selection follows a rule-of-thumb approach based on spread (interquartile range and variance), which provides a reasonable default for unimodal distributions but may oversmooth multimodal structures.

Missing or non-finite values are not allowed and will result in an error.

Value

Invisibly returns the result of the selected plotting call. Typically a list containing grid coordinates and estimated density values.

See Also

Other plot.bivariate: plotAssoc(), plotBag(), plotCor(), plotHeatmap(), plotHexbin(), plotMosaic(), plotXY()

Examples

```
set.seed(1)
x <- rnorm(200)
y <- x + rnorm(200)

plotDens2D(x, y)

plotDens2D(x, y, type = "image")
```

---

plotDensBox *Density and Boxplot Combination (Grouped)*

Description

Combines density plots and horizontal boxplots for a numeric variable, optionally grouped by a categorical variable. The density plot shows the distribution shape, while the boxplot summarizes key statistics such as median, spread, and outliers.

Usage

```
plotDensBox(x, ...)

## Default S3 method:
plotDensBox(
  x,
  g = NULL,
  main = "",
  xlab = "",
  ylab = "",
  xlim = NULL,
  layout_heights = c(2, 1.4),
  col = NULL,
  grid = TRUE,
  densArgs = TRUE,
  boxArgs = TRUE,
  stamp = NULL,
  ...
)

## S3 method for class 'formula'
plotDensBox(
  formula,
  data,
  subset,
  na.action = na.omit,
  main = "",
  xlab = "",
  ylab = "",
  xlim = NULL,
  layout_heights = c(2, 1.4),
```

```
  col = NULL,
  grid = TRUE,
  densArgs = TRUE,
  boxArgs = TRUE,
  stamp = NULL,
  ...
)
```

Arguments

x	numeric vector, or a formula of the form x ~ g.
...	further graphical parameters passed to par via the internal framework.
g	optional grouping variable (ignored if a formula is used).
main	main title of the plot.
xlab	label for the x-axis.
ylab	label for the y-axis.
xlim	numeric vector of length 2 specifying the x-axis limits.
layout_heights	numeric vector of length 2 specifying the relative heights of the density plot (top) and boxplot (bottom).
col	vector of colors. If NULL, a palette is generated.
grid	controls drawing of the background grid. Can be: • TRUE: draw grid with default settings • FALSE, NULL, NA: suppress grid • a named list: arguments passed to grid
densArgs	controls density estimation via density. Can be: • TRUE: use default density settings • FALSE, NULL, NA: suppress densities • a named list: additional arguments passed to density
boxArgs	controls drawing of boxplots via boxplot. Can be: • TRUE: use default boxplot settings • FALSE, NULL, NA: suppress boxplots • a named list: additional arguments passed to boxplot
stamp	optional annotation passed to the plotting framework.
formula	A formula of the form y ~ group.
data	an optional data frame containing variables in the formula.
subset	optional expression indicating which observations to use.
na.action	a function specifying how missing values are handled.

Details

The function arranges two plots vertically using layout: a density plot on top and a horizontal boxplot below. When a grouping variable is provided, densities and boxplots are drawn for each group.

Optional plot components are controlled using callIf semantics:

- TRUE: draw with defaults
- FALSE: suppress component
- named list: customize component arguments

Value

Invisibly returns NULL.

See Also

density, boxplot, callIf

Other plot.univariate: plotArea(), plotBar(), plotBox(), plotCatDist(), plotDens(), plotDot(), plotECDF(), plotFdist(), plotLines(), plotQQ(), plotViolin()

Examples

```
## Not run:
set.seed(1)
x <- rnorm(100)
g <- sample(c("A", "B"), 100, TRUE)

plotDensBox(x)
plotDensBox(x, g)

plotDensBox(
  x,
  densArgs = list(adjust = 2),
  boxArgs  = list(notch = TRUE)
)

plotDensBox(
  x,
  boxArgs = FALSE
)

plotDensBox(x ~ g)

## End(Not run)
```

---

plotDot *Dot Plot for Estimates and Confidence Intervals*

Description

Displays numeric estimates as points on a horizontal scale. Optional confidence limits are shown as horizontal lines with capped endpoints. Several series of estimates can be arranged in labelled groups.

Usage

```
plotDot(
  x,
  items = NULL,
  groups = NULL,
  main = NULL,
  xlim = NULL,
```

```
  gap = 1,
  axes = TRUE,
  xax = NULL,
  box = .useTheme,
  grid = .useTheme,
  pch = .useTheme,
  ...
)
```

Arguments

x	numeric estimates or confidence interval data. Supported formats are a numeric vector, a numeric matrix, a three-dimensional numeric array, or a "CI" object created with as.CI
items	optional character vector containing the item labels; defaults to the row names or first dimension names of x
groups	optional character vector containing the group labels; defaults to the column names or third dimension names of x
main	optional main title
xlim	numeric vector containing the limits of the horizontal axis; by default, the range of all estimates and confidence limits
gap	non-negative numeric value controlling the vertical space between groups
axes	logical; whether the horizontal and item axes are drawn
xax	optional specification for the horizontal axis, interpreted by the internal axis renderer
box	specification controlling the plot box. The default .useTheme uses the active theme. A logical value, NA, or a named list of graphical parameters can also be supplied
grid	specification controlling the horizontal item and group grid lines. The default .useTheme follows the active theme. A logical value, NA, or a named list of graphical parameters can also be supplied
pch	specification for the estimate points. The default .useTheme uses the point settings of the active theme. A plotting symbol or a named list containing parameters such as pch, col, bg, and cex can also be supplied
...	additional graphical parameters passed to par

Details

A numeric vector represents one estimate for each item.

A numeric matrix represents estimates only: rows define the items and columns define the groups. Consequently, a matrix with three columns is interpreted as three groups and not automatically as estimates with lower and upper confidence limits.

Use as.CI to declare explicitly that a matrix, data frame, list, or result from tapply contains confidence interval data:

```
plotDot(as.CI(x))
```

A "CI" object contains the columns est, lci, and uci. Additional columns can define the item and group structure. If two additional columns are present, the first defines the items and the second defines the groups.

Confidence interval data can alternatively be supplied as a three-dimensional numeric array with dimensions items × 3 × groups. The second dimension must contain, in this order, the estimate, lower confidence limit, and upper confidence limit.

Values supplied directly as arguments take precedence over the corresponding settings of the active theme.

Value

invisibly, a list containing:

ypos vertical positions of the items within each group

group_y vertical positions of the group labels

sep_y vertical positions of the group separators

See Also

as.CI, is.CI, dotchart

Other plot.univariate: plotArea(), plotBar(), plotBox(), plotCatDist(), plotDens(), plotDensBox(), plotECDF(), plotFdist(), plotLines(), plotQQ(), plotViolin()

Examples

```
# estimates for a single series
est <- c(A = 12, B = 18, C = 28, D = 40, E = 65)

plotDot(
  est,
  main = "Estimates"
)

# matrix columns represent groups of estimates
groupedEst <- cbind(
  Control = c(A = 12, B = 18, C = 28),
  Treatment = c(A = 16, B = 24, C = 35)
)

plotDot(
  groupedEst,
  main = "Grouped estimates"
)

# confidence intervals stored in a matrix
ci <- cbind(
  est = est,
  lci = est - c(2, 3, 4, 5, 6),
  uci = est + c(2, 3, 4, 5, 6)
)

plotDot(
  as.CI(ci),
  main = "Estimates with confidence intervals"
```

```
)

# grouped confidence intervals stored in a data frame
groupedCI <- data.frame(
  item = rep(c("A", "B", "C"), 2),
  group = rep(c("Control", "Treatment"), each = 3),
  estimate = c(12, 18, 28, 16, 24, 35),
  lower = c(10, 15, 24, 13, 20, 30),
  upper = c(14, 21, 32, 19, 28, 40)
)

plotDot(
  as.CI(
    groupedCI,
    estimate = "estimate",
    lower = "lower",
    upper = "upper"
  ),
  main = "Grouped confidence intervals"
)

# returned positions can be used to add graphical elements
pos <- plotDot(est)

points(
  est + 3,
  y = unlist(pos$ypos),
  pch = 4
)
```

---

plotECDF *Empirical Cumulative Distribution Function*

Description

Fast plotting of the empirical cumulative distribution function (ECDF), designed to stay performant even for very large vectors (n ~ 1e6-1e7).

Usage

```
plotECDF(x, ...)

## Default S3 method:
plotECDF(
  x,
  main = NULL,
  xlab = NULL,
  ylab = "",
  xlim = NULL,
  breaks = 1000,
  add = FALSE,
  col = .useTheme,
```

```
  lwd = 2,
  grid = .useTheme,
  box = .useTheme,
  stamp = .useTheme,
  ...
)

## S3 method for class 'formula'
plotECDF(
  formula,
  data,
  subset,
  na.action = na.omit,
  main = NULL,
  xlab = NULL,
  ylab = "",
  xlim = NULL,
  breaks = 1000,
  col = .useTheme,
  lwd = 2,
  grid = .useTheme,
  box = .useTheme,
  legend = TRUE,
  stamp = .useTheme,
  ...
)
```

Arguments

x	numeric vector of the observations for the ECDF.
...	further graphical parameters passed to par via the internal framework.
main	main title of the plot. NULL (default) derives a title from deparse(substitute(x)). "", NA, or FALSE suppress the title.
xlab	label for the x-axis. NULL (default) derives a label the same way as main.
ylab	label for the y-axis. The y-axis itself always shows fixed probability labels (.00 to 1.00); ylab adds an axis title above/beside those, empty by default.
xlim	numeric vector of length 2; x-axis limits. NULL (default) uses range(x).
breaks	controls the rendered resolution. A single integer (default 1000) subsamples the ECDF at that many evenly-spaced quantiles whenever length(x) exceeds it; for length(x) <= breaks, full resolution is used regardless (no data is ever thinned below its actual size). NULL, FALSE, or Inf force full resolution unconditionally, regardless of length(x).
add	logical; if TRUE, adds to an existing plot instead of starting a new one.
col	color of the step line and the min/max marker points. .useTheme (default) resolves to getTheme()$twin[1] - a single accent color, consistent with lines.loess and plotQQ()'s confidence band.
lwd	line width.
grid	controls drawing of the background grid (vertical lines at default tick positions, plus a fixed set of horizontal reference lines at the probability ticks 0/.25/.5/.75/1

- the latter always grey regardless of the active theme, a deliberately distinct look, not theme-driven). `.useTheme` (default) follows the active theme's grid on/off state (`getTheme()$grid`). `TRUE`/`FALSE`/`NA`, or a named list, as for `grid`.

`box`	controls drawing of the plot box. `.useTheme` (default) resolves to `getTheme()$box`. `TRUE`/`FALSE`/`NA`, or a named list, as for `box`.
`stamp`	controls the corner stamp. `.useTheme` (default) resolves to `getTheme()$stamp`. `TRUE`/`FALSE`/`NULL`, a string, or a named list of arguments for `stamp()`.
`formula`	a formula of the form `y ~ x`.
`data`	an optional data frame containing variables in the formula.
`subset`	optional expression indicating which observations to use.
`na.action`	a function specifying how missing values are handled. Defaults to `na.omit`.
`legend`	logical or list controlling the legend. If `TRUE`, a legend is drawn using the column names of the data. If a list is supplied, its elements are passed to the internal legend drawing routine.

Details

The base `plot.ecdf`/`ecdf` machinery becomes impractically slow well below `n = 1e7`, since it tracks every single jump. Beyond a few thousand points, individual jumps are visually indistinguishable anyway, so `plotECDF()` caps the rendered resolution at `breaks` points by default.

Resolution is achieved via quantile subsampling, not histogram binning: `breaks` evenly-spaced probability points are mapped back to their corresponding quantiles (`quantile`, `type = 7`). Each rendered point therefore lies exactly on the true ECDF - unlike an equal-width-histogram approximation, this adapts automatically to the shape of the distribution (no resolution is wasted on near-empty bins in a skewed or heavy-tailed distribution, nor lost in the tails).

Value

Invisibly returns `NULL`.

See Also

`plot.ecdf`, `plotFdist`, `theme`

Other plot.univariate: `plotArea()`, `plotBar()`, `plotBox()`, `plotCatDist()`, `plotDens()`, `plotDensBox()`, `plotDot()`, `plotFdist()`, `plotLines()`, `plotQQ()`, `plotViolin()`

Examples

```
plotECDF(faithful$eruptions)

# large vector - automatically thinned to 1000 points, no breaks= needed
x <- rnorm(1e6)
plotECDF(x)

# force full resolution regardless of size
plotECDF(x, breaks = NULL)

# grouped ECDFs via the formula interface
plotECDF(Sepal.Length ~ Species, data = iris)
```

`plotFacet` *Facet Panel Matrix in Base Graphics*

Description

Draws a lattice-like matrix of panels in base graphics with identical plot region sizes, panel strips, axes on the outer panels only and a user-supplied panel function for the content. Panel gaps are defined in margin lines and remain exact, independent of device size and strip height, as the strip space is reserved separately in the layout.

Usage

```
plotFacet(
  samples,
  dim,
  panelFun,
  cols = NULL,
  stripLabels = NULL,
  main = "",
  xlab = "",
  ylab = "",
  xlim = NULL,
  ylim = NULL,
  mar = c(2.5, 2.5, 0.5, 0.5),
  oma = c(3, 3, 4, 1.2),
  horiz = 1,
  vert = NULL,
  strip = TRUE,
  bg = "grey95",
  grid = TRUE,
  cex = 0.66,
  pch = 16,
  ...
)
```

Arguments

`samples`	a list of samples, each a list (or data.frame) with components `x` and `y`.
`dim`	integer vector of length 2, the number of rows and columns of the panel matrix, `c(nrow, ncol)`.
`panelFun`	the panel function, called per panel as `panelFun(x, y, col, pch, ...)` with a fully set up coordinate system.
`cols`	the colors for the panels, recycled to the number of samples. Default is `hcl.colors(n, "Dark 3")`.
`stripLabels`	the labels for the panel strips. Default is the sequence along `samples`.
`main`	the main title, placed in the outer margin.
`xlab`, `ylab`	the axis labels, placed in the outer margins.
`xlim`, `ylim`	the common axis limits for all panels. Default is the range over all samples.

mar	the margins around the whole panel matrix in lines, c(bottom, left, top, right). The bottom and left margins hold the axis annotation of the outer panels.
oma	the outer margins in lines, holding xlab, ylab and main.
horiz	the horizontal gap between adjacent columns in margin lines.
vert	the vertical gap between adjacent rows in margin lines. Default is horiz, yielding physically equal gaps.
strip	controls the panel strips, evaluated by bedrock::callIf: TRUE (default) draws strips with default settings, FALSE/NULL/NA suppresses them (no space is reserved), a named list is passed as arguments to titleRect, e.g. list(bg = "steelblue", col = "white", line = 1.5). The label argument is set per panel from stripLabels and cannot be overridden.
bg	the background color of the plot regions.
grid	controls the grid lines, evaluated by bedrock::callIf: TRUE (default) draws grid lines at the positions of axTicks with default settings (col = "grey85", lwd = 0.8), FALSE/NULL/NA suppresses them, a named list is passed as arguments to abline, e.g. list(col = "white", lty = "dotted"). The default positions v and h can be overridden, e.g. list(v = seq(0, 20, 5)).
cex	the character expansion used inside the panels (axis annotation, strip labels, panel content) and as unit for the panel margin lines. Default is 0.66, matching R's own reduction in multi-figure layouts. Set deterministically after each plot.new(), see Details.
pch	the plotting character, passed to panelFun.
...	the dots are passed to panelFun.

Details

The available device area inside the outer margins is partitioned with layout such that all plot regions have exactly the same size in inches. The horizontal gap between two adjacent columns is horiz margin lines, the vertical gap between two adjacent rows is vert lines. Since margin lines have the same physical size in both directions, horiz == vert yields visually equal gaps.

The strip is drawn with titleRect above each panel. Its height (line argument of titleRect) is reserved in the top margin of every panel, so the strip never eats into the gap between the rows.

Note that plot.new silently reduces cex (and with it csi, the physical size of a margin line) in layouts with more than two regions, which would make the realized panel margins deviate from the computed layout. The function therefore controls the character size deterministically via its cex argument and sets the panel margins in inches (mai/omi), so that all plot regions are exactly equal in size.

Value

Invisibly returns a list with the realized geometry: horiz, vert, strip_line (reserved strip height in lines) and the common plot region size plot_width_in, plot_height_in in inches.

See Also

graphics::layout, titleRect, bedrock::callIf

Other graphics.layout: abcCoords(), axTicks, axisBreak(), isValidPlotRegion(), lineToUser(), mar(), spreadOut()

Examples

```
samples <- lapply(split(ChickWeight, ChickWeight$Chick)[1:25],
                  function(z) list(x = z$Time, y = z$weight))

my_panel <- function(x, y, col, pch = 16, ...) {
  points(x, y, pch = pch, col = col)
  abline(lm(y ~ x), lwd = 1)
}

plotFacet(samples, dim = c(5, 5), panelFun = my_panel,
           xlab = "Time", ylab = "Weight", main = "ChickWeight",
           strip = list(bg = "grey80", cex = 0.8))
```

`plotFdist` *Frequency Distribution Plot*

Description

Creates a univariate graphical summary combining a histogram (or probability mass plot for discrete data), a kernel density curve, a boxplot, and an empirical cumulative distribution function in a single multi-panel figure. Optional components include a rug, and fitted theoretical distribution curves for both the histogram and the ECDF panel.

Usage

```
plotFdist(
  x,
  main = NULL,
  xlab = "",
  xlim = NULL,
  heights = NULL,
  hist = TRUE,
  dens = TRUE,
  rug = FALSE,
  curve = FALSE,
  boxplot = TRUE,
  ecdf = TRUE,
  curveEcdf = FALSE,
  stamp = .useTheme,
  ...
)
```

Arguments

`x`	numeric vector whose distribution is to be plotted.
`main`	main title. `NULL` (default) derives the title from `deparse(substitute(x))`. `""`, `NA`, or `FALSE` suppress the title and compact the top margin.
`xlab`	label for the x-axis. The variable name is typically placed in `main`, so this defaults to `""`.

xlim	range of the x-axis. NULL (default) uses a pretty range of the non-missing values in x.
heights	numeric vector of relative panel heights for layout: three values for histogram/boxplot/ecdf, two for histogram/boxplot or histogram/ecdf only. NULL (default) chooses automatically.
hist	controls the histogram panel. TRUE (default) uses package defaults; FALSE/NA suppresses the panel (xlim then falls back to the pretty range of the data); a list overrides specific arguments forwarded to hist. The element type selects the plot style: "hist" (standard histogram, chosen automatically for continuous or high-cardinality data) or "mass" (vertical bars per unique value, for discrete/low-cardinality data).
dens	controls the kernel density curve. TRUE (default) draws a curve via density; a list overrides specific arguments (e.g. list(bw = 0.1, col = "red")).
rug	controls a rug plot. FALSE (default) suppresses it; TRUE or a list draw a rug via rug.
curve	controls a fitted theoretical distribution curve on the histogram. FALSE (default), NULL, or NA suppress it; TRUE draws a normal curve with mean(x)/sd(x); a list may include expr as a character string, expression, or function of x for any distribution (e.g. list(expr = "dt(x, df=2)", col = "darkgreen")). A character expr is evaluated in the caller's environment, so local variables may be referenced (see the gamma example below).
boxplot	controls the boxplot panel. TRUE (default) draws a horizontal boxplot with a mean marker and CI band; FALSE/ NA suppresses the panel. A list overrides arguments forwarded to boxplot; two extra elements control the mean display: pch.mean (default 3) and col.meanci (default getTheme()$grid$col). Set either to NA to suppress that element.
ecdf	controls the ECDF panel. TRUE (default) calls plotECDF; a list overrides specific arguments.
curveEcdf	controls a fitted theoretical CDF curve on the ECDF panel, analogous to curve. FALSE (default) suppresses it.
stamp	controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/ NULL, a string, or a named list for stamp().
...	further graphical parameters passed to par via the internal framework. Note that mar given here sets the *outer* margins (oma) of the multi-panel figure; the inner panel margins are managed internally and cannot be overridden.

Details

Each plot component is controlled via a single argument accepting callIf semantics:

- TRUE: draw with package defaults
- FALSE, NULL, or NA: suppress entirely
- a named list: draw with the given overrides merged into the defaults

Performance: for very large vectors (n > 1e7) the density curve, ECDF, and semi-transparent boxplot outliers will still take noticeable time. For exploratory work on very large data, consider sampling first: plotFdist(x[sample(length(x), 5000)]).

See Also

hist, boxplot, plotECDF, density, rug, layout, theme

Other plot.univariate: plotArea(), plotBar(), plotBox(), plotCatDist(), plotDens(), plotDensBox(), plotDot(), plotECDF(), plotLines(), plotQQ(), plotViolin()

Examples

```
plotFdist(faithful$eruptions)

# custom histogram breaks, density color, boxplot styling
plotFdist(faithful$eruptions,
  hist    = list(breaks = 50),
  dens    = list(col = "olivedrab4"),
  boxplot = list(col = "olivedrab2", pch.mean = NA, col.meanci = NA))

# no density, no ecdf, add rug instead
plotFdist(faithful$eruptions,
  dens = FALSE, ecdf = FALSE,
  hist = list(xaxt = "s"),
  rug  = TRUE,
  heights = c(3, 2.5),
  main = "Eruption time")

# overlay a normal density curve
x <- rnorm(1000)
plotFdist(x, curve = TRUE, boxplot = FALSE, ecdf = FALSE)

# compare with a t-distribution curve
plotFdist(x,
  curve   = list(expr = "dt(x, df=2)", col = "darkgreen"),
  boxplot = FALSE, ecdf = FALSE)

# overlay gamma distribution on both histogram and ECDF
ozone <- airquality$Ozone
m <- mean(ozone, na.rm = TRUE)
v <- var(ozone, na.rm = TRUE)
plotFdist(ozone,
  hist       = list(breaks = 15),
  curve      = list(expr = "dgamma(x, shape = m^2/v, scale = v/m)",
                    col = "navajowhite3"),
  curveEcdf  = list(expr = "pgamma(x, shape = m^2/v, scale = v/m)",
                    col = "navajowhite3"),
  main = "Airquality - Ozone")
```

---

plotFun *Plot Mathematical Functions*

Description

Flexible plotting of mathematical functions in Cartesian, polar, and parametric form.

Usage

```
plotFun(
  expr,
  main = NULL,
  xlab = "",
  ylab = "",
  xlim = NULL,
  ylim = NULL,
  from = 0,
  to = 1,
  n = 500,
  args = NULL,
  add = FALSE,
  col = .useTheme,
  lwd = 1,
  lty = 1,
  grid = .useTheme,
  stamp = .useTheme,
  ...
)
```

Arguments

expr	expression defining the function. Either a formula (y ~ f(x)), a polar formula (r ~ f(phi)), or a list of two formulas for parametric plots.
main	main title of the plot. NULL (default) derives a title from expr. "", NA, or FALSE suppress the title and compact the top margin.
xlab, ylab	labels for the axes.
xlim, ylim	numeric vectors of length 2 defining axis limits. For Cartesian functions, xlim defaults to c(from, to). For polar and parametric plots, limits are derived from the data. If only xlim is supplied for polar/parametric plots, ylim mirrors it (ensures asp=1 renders correctly).
from, to	numeric; lower and upper bound of the parameter domain. Default from=0, to=1.
n	integer; number of evaluation points. Default 500.
args	named list of additional parameters fixed for the function expression (e.g. list(a = 2) for y ~ sin(a*x)).
add	logical; if TRUE, adds to an existing plot without redrawing axes or grid. Default FALSE.
col	color of the line. .useTheme (default) resolves to getTheme()$twin[1] - the primary accent color, consistent with lines.loess and plotQQ().
lwd	line width. Default 1.
lty	line type. Default 1.
grid	controls drawing of the background grid. .useTheme (default) follows the active theme (getTheme()$grid). TRUE/FALSE/NA, or a named list, as for grid.
stamp	controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/ NULL, a string, or a named list for stamp().
...	further graphical parameters passed to par via the internal framework.

Details

The function automatically detects the type of input:

- **Cartesian:** `y ~ f(x)`
- **Polar:** `r ~ f(phi)` or `r ~ f(theta)`
- **Parametric:** `list(x ~ f(t), y ~ g(t))`

Additional parameters fixed for the expression can be passed via `args`.

Value

Invisibly returns a list with components `x` and `y` (the plotted coordinates after finite filtering).

See Also

Other plot.distribution: `plotProbDist()`, `shade()`

Examples

```
# Cartesian
plotFun(y ~ x^2)

# Damped oscillation - add second curve to existing plot
plotFun(y ~ 3*exp(-x/5)*sin(4*x), from = 0, to = 10)
plotFun(y ~ 3*exp(-x/5)*sin(6*x), from = 0, to = 10, add = TRUE)

# Cardioid (polar)
plotFun(r ~ 2*(1 + cos(phi)), from = 0, to = 2*pi)

# Heart curve (parametric)
plotFun(
  list(
    x ~ 13*cos(t) - 5*cos(2*t) - 2*cos(3*t) - cos(4*t),
    y ~ 16*sin(t)^3
  ),
  from = 0, to = 2*pi, lwd = 2
)

# Parameter sweep
for (a in 1:3)
  plotFun(y ~ sin(a*x), args = list(a = a),
          from = 0, to = 2*pi,
          add = (a != 1), col = a)
```

---

plotHeatmap *Heatmap for Categorical Data*

Description

Visualizes a contingency table using a heatmap representation. Cell values are mapped to colors based on counts or proportions, optionally with text labels overlaid.

Usage

```
plotHeatmap(
  x,
  main = NULL,
  xlab = "",
  ylab = "",
  xlim = NULL,
  ylim = NULL,
  scale = c("count", "prop", "row", "col"),
  col = .useTheme,
  border = NA,
  naCol = "gray90",
  text = FALSE,
  zlim = NULL,
  box = .useTheme,
  stamp = .useTheme,
  ...
)
```

Arguments

x	a contingency table, matrix, or a pair of categorical vectors coercible via table.
main	main title of the plot. NULL (default) derives a title from the expression passed as x (via deparse(match.call()$x)), the same "substitute magic" convention used by plotXY/plotBox/ plotAssoc for their default titles - there's no formula pair here, just the single table argument, so the default is simply that expression's text (e.g. plotHeatmap(tab) titles itself "tab"). "", NA, or FALSE suppress the title entirely and compact the top margin; any other string is used as given (resolved internally via .resolveTitle()).
xlab	label for the x-axis.
ylab	label for the y-axis.
xlim, ylim	numeric vectors of length 2 specifying axis limits.
scale	character specifying how values are computed: "count" absolute frequencies "prop" joint proportions $P(X, Y)$ "row" row-wise proportions $P(Y \mid X)$ "col" column-wise proportions $P(X \mid Y)$
col	optional vector of colors. Default is a hardcoded sequential white-to-navy ramp (pal("Blues", n = 100)) - deliberately not theme-driven: cell values here are sequential (one direction, no sign change), unlike the active theme's categorical palette or diverging twin pair, neither of which fits a heat scale.
border	color of tile borders. Defaults to NA.
naCol	color used for missing values.
text	logical; if TRUE, cell values are printed on top of the tiles using fm() formatting.
zlim	numeric vector of length 2 specifying the range used for color scaling. If NULL, the range of the data is used.
box	controls drawing of the outer frame around the tile grid, drawn via rect() at the exact cell boundaries rather than box() (the initial plot suppresses the standard

box via `frame.plot = FALSE`, since cell bounds differ from the default plot region). `.useTheme` (default) resolves border color/width from `getTheme()$box`. `TRUE/FALSE`, or a named list overriding `rect()` arguments for this call only.

`stamp` controls the corner stamp. `.useTheme` (default) resolves to `getTheme()$stamp`. `TRUE/FALSE/ NULL`, or an explicit string, as for `.withGraphicsState()` (internal).

`...` further graphical parameters passed to `par` via the internal framework.

Details

The heatmap represents values in a contingency table using color intensity. Depending on `scale`, the plot shows either absolute counts or different types of proportions. Rows are drawn in reading order: the first table row appears at the top, matching the printed table and the other bivariate plots. This plot complements association and spine plots by focusing on overall structure rather than conditional distributions or statistical inference.

Value

Invisibly returns the matrix used for plotting.

See Also

`plotAssoc`, `image`, `theme`

Other plot.bivariate: `plotAssoc()`, `plotBag()`, `plotCor()`, `plotDens2D()`, `plotHexbin()`, `plotMosaic()`, `plotXY()`

Examples

```
## Not run:
tab <- table(UCBAdmissions)

plotHeatmap(tab,
            scale = "prop",
            text = TRUE)

## End(Not run)
```

---

plotHexbin *Hexagonal Binning Plot*

Description

Displays a two-dimensional density estimate using hexagonal bins. Observations are aggregated into hexagons and coloured according to the number of observations falling into each cell.

Usage

```
plotHexbin(
  x,
  y,
  bins = 30,
  col = NULL,
  border = NA,
  grid = FALSE,
  xlim = NULL,
  ylim = NULL,
  main = NULL,
  xlab = "",
  ylab = "",
  ...
)
```

Arguments

x	numeric vector of x-values.
y	numeric vector of y-values.
bins	number of hexagons across the x-axis.
col	colours used for the count scale. If NULL, a default sequential palette is used.
border	border colour of the hexagons.
grid	logical or list controlling the background grid.
xlim	limits for the x-axis.
ylim	limits for the y-axis.
main	main title.
xlab	label for the x-axis.
ylab	label for the y-axis.
...	additional graphical parameters passed to .applyParFromDots().

Value

Invisibly returns a list containing the computed hexbin object and the original x and y.

See Also

Other plot.bivariate: plotAssoc(), plotBag(), plotCor(), plotDens2D(), plotHeatmap(), plotMosaic(), plotXY()

plotLift *Lift Chart*

Description

Draws the cumulative lift curve, the cumulative gain curve, or the per-group lift bars of a binary classifier, together with the baseline of a random model. The chart answers the operational question directly: how much better is acting on the top-scored share of the cases than acting on a random share of the same size.

Usage

```
plotLift(
  x,
  type = c("cumulative", "gain", "decile"),
  main = NULL,
  xlab = NULL,
  ylab = NULL,
  ylim = NULL,
  col = .useTheme,
  lwd = 2,
  grid = .useTheme,
  box = .useTheme,
  baseline = TRUE,
  perfect = FALSE,
  legend = TRUE,
  stamp = .useTheme,
  ...
)
```

Arguments

x	an object of class "Lift", as returned by alloy::lift().
type	the curve to draw. One of "cumulative" (cumulative lift over depth, the default), "gain" (share of all positives captured, over depth), or "decile" (per-group lift as bars).
main	main title of the plot. NULL (default) derives a title from deparse(substitute(x)). "", NA, or FALSE suppress the title entirely (and compact the top margin accordingly); any other string is used as given.
xlab	label for the x-axis. NULL (default) derives a label from type.
ylab	label for the y-axis. NULL (default) derives a label from type.
ylim	numeric vector of length 2; y-axis limits. NULL (default) spans the curve together with the baseline.
col	color of the curve or the bars. .useTheme (default) resolves to getTheme()$twin[1] - a single accent color, consistent with plotECDF.
lwd	line width of the curve. Has no effect for type = "decile".
grid	controls drawing of the background grid. Can be: • .useTheme (default): follow the active theme (getTheme()$grid)

- TRUE: draw grid with theme settings
- FALSE, NULL, or NA: suppress grid
- a named list: arguments passed to grid, overriding the theme defaults for this call only

box — controls drawing of the plot box. .useTheme (default) resolves to getTheme()$box. TRUE/FALSE/NA, or a named list, as for grid.

baseline — controls the reference line of a random model - a horizontal line at 1 for type = "cumulative" and "decile", the diagonal for type = "gain". Can be:

- TRUE (default): draw with default settings
- FALSE, NULL, or NA: suppress
- a named list: arguments passed to lines (or abline for type = "decile"), e.g. list(col = "black", lty = "dotted")

perfect — controls the curve of a perfect ranking - the theoretical maximum attainable at each depth, given the base rate. FALSE by default, since it compresses the interesting part of the y-axis; TRUE or a named list to draw it, as for baseline. Has no effect for type = "decile".

legend — controls drawing of the legend. Can be:

- TRUE (default): draw with default settings
- FALSE, NULL, or NA: suppress
- a named list: arguments passed to legend, e.g. list(x = "bottomleft")

stamp — controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/ NULL, a string, or a named list of arguments for stamp().

... — further graphical parameters passed to par() via the internal framework.

Details

Reading the cumulative curve at depth 0.2 gives the factor by which contacting the top-scored fifth of the cases beats contacting a random fifth. The gain variant answers the complementary question - what share of all positives that fifth captures.

The curve necessarily converges to 1 (cumulative lift) or to the diagonal endpoint (gain) at depth 1, where all cases are selected and the model no longer discriminates. Only the left part of the curve carries decision value: a model that separates well over the first two deciles and poorly afterwards is preferable for a small campaign to one with the reverse profile and identical AUC.

Optional plot components (grid, box, baseline, perfect, legend) follow callIf semantics:

- TRUE: draw with defaults
- FALSE, NULL, or NA: suppress component
- named list: customize component arguments

col, grid, box, and stamp default to .useTheme, deferring to the package's active theme (see theme) rather than a hardcoded value.

The number of groups is a property of the lift table, not of the plot - set it via the nBins argument of alloy::lift().

Value

Invisibly returns x.

See Also

alloy::lift(), alloy::roc(), plotECDF, callIf, theme

Other plot.special: plotBinaryTree(), plotCirc(), plotMiss(), plotPolar(), plotPropCI(), plotTernary(), plotTimeSeries(), plotTreemap(), plotWeb()

Examples

```
## Not run:
fitLogit <- alloy::fitMod(admit ~ gre + gpa + rank, Admit, fitfn = "logit")
lft <- alloy::lift(fitLogit)

plotLift(lft)
plotLift(lft, type = "gain")

# Per-group bars, coarser grouping set at computation time
plotLift(alloy::lift(fitLogit, nBins = 5), type = "decile")

# Add the perfect-ranking reference, suppress the legend
plotLift(lft, perfect = TRUE, legend = FALSE)

# No title, compact top margin
plotLift(lft, main = "")

## End(Not run)
```

---

plotLines *Line Plot for Multiple Series*

Description

Draws one or several line series using matplot. The function accepts either a matrix of values or separate x and y coordinates and supports optional point symbols, grid lines, and an automatically positioned legend.

Usage

```
plotLines(
  x,
  y,
  main = NULL,
  xlab = "",
  ylab = "",
  xlim = NULL,
  ylim = NULL,
  xaxt = NULL,
  yaxt = NULL,
  lty = 1,
  lwd = 2,
  col = .useTheme,
  points = FALSE,
  grid = .useTheme,
```

```
  legend = TRUE,
  stamp = .useTheme,
  ...
)
```

Arguments

x	numeric vector, matrix or data frame. If y is missing, x is interpreted as a matrix of series where rows correspond to x positions and columns to individual lines.
y	optional numeric vector or matrix giving the y-values. If supplied, x is interpreted as the x-coordinates.
main	main title of the plot. NULL (default) derives a title from the input. "", NA, or FALSE suppress the title and compact the top margin.
xlab, ylab	labels for the axes.
xlim, ylim	limits for the axes.
xaxt, yaxt	axis specification passed to axis.
lty	line type(s).
lwd	line width(s).
col	colours for the lines. .useTheme (default) resolves to pal(getTheme()$palette), the active theme's qualitative palette.
points	controls drawing of points on the lines. FALSE (default) suppresses points; TRUE draws with theme defaults (getTheme()$points); a named list overrides individual elements (pch, col, bg, cex).
grid	controls drawing of the background grid. .useTheme (default) follows the active theme (getTheme()$grid). TRUE/FALSE/NA, or a named list, as for grid.
legend	controls the legend. TRUE (default) draws an inline legend via textLegend at the last value of each series. FALSE/NA suppresses it. A list overrides legend arguments.
stamp	controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/NULL, a string, or a named list for stamp().
...	additional graphical parameters passed to par via .applyParFromDots() and to the plotting functions.

Details

If y is missing, x is interpreted as a matrix and each column is drawn as a separate line. Row names are used for the x-axis labels if available. The legend labels default to the column names of the data.

Value

Invisibly returns a list containing:

- x the x-values used for plotting,
- y the y-values (if supplied),
- legend the legend specification if drawn.

See Also

Other plot.univariate: plotArea(), plotBar(), plotBox(), plotCatDist(), plotDens(), plotDensBox(), plotDot(), plotECDF(), plotFdist(), plotQQ(), plotViolin()

Examples

```
m <- matrix(c(3,4,5,1,5,4,2,6,2), nrow = 3,
            dimnames = list(
              dose = c("A","B","C"),
              age  = c("2000","2001","2002")
            ))

plotLines(m, lwd = 2, main = "Dose ~ Age")

# with points
plotLines(m, points = TRUE)

# custom legend
plotLines(m, legend = list(cex = 0.8))
```

---

plotMiss *Plot Missing Data*

Description

Takes a data frame and displays the location of missing data. The missings can be clustered and be displayed together.

Usage

```
plotMiss(
  x,
  col = "deeppink4",
  bg = fade("navajowhite3", 0.3),
  clust = FALSE,
  main = NULL,
  ...
)
```

Arguments

x	a data.frame to be analysed.
col	the colour of the missings.
bg	the background colour of the plot.
clust	logical, defining if the missings should be clustered. Default is FALSE.
main	the main title.
...	the dots are passed to plot.

Details

A graphical display of the position of the missings can be help to detect dependencies or patterns within the missings.

Value

if clust is set to TRUE, the new order will be returned invisibly.

Note

Following an idea of Henk Harmsen henk@carbonmetrics.com

See Also

hclust, countCompCases

Other plot.special: plotBinaryTree(), plotCirc(), plotLift(), plotPolar(), plotPropCI(), plotTernary(), plotTimeSeries(), plotTreemap(), plotWeb()

Examples

```
plotMiss(airquality, main="Missing data (in orignal order)")
plotMiss(airquality, main="Missing data (clustered)", clust=TRUE)
```

---

plotMosaic *Mosaic Plot for 2-Way Contingency Tables*

Description

Draws a mosaic plot (spine plot) for a 2-way contingency table, with proportional tile areas, optional cell labels (counts or percentages), and a "Few"-style minimal appearance (white tile separators, muted palette, optional legend).

Usage

```
plotMosaic(
  x,
  main = "",
  xlab = NULL,
  ylab = NULL,
  swap = FALSE,
  horiz = FALSE,
  gap = 0.01,
  col = .useTheme,
  border = "white",
  legend = TRUE,
  labels = c("p", "n", "none"),
  labCex = 0.8,
  labDigits = 1,
  stamp = .useTheme,
  ...
)
```

Arguments

x	a 2-way contingency table, matrix, or array coercible via as.table(). Higher-dimensional arrays must be collapsed first, e.g. via apply(x, c(1,2), sum).
main	character. Plot title. Default "" (no title).
xlab, ylab	character or NULL. Axis labels. Default NULL (no labels), since the category levels together with main and the legend title are usually self-explanatory.
swap	logical. If TRUE, transpose the two dimensions of x before plotting, so the second table dimension determines the x-axis split and the first becomes the fill/legend variable. Default FALSE.
horiz	logical. If TRUE, draw the mosaic horizontally: the first table dimension is shown top-to-bottom on the y-axis instead of left-to-right on the x-axis. Default FALSE.
gap	numeric. Width of the white separator drawn between adjacent tiles, in plot-region units ([0, 1]). Default 0.01.
col	vector of colors for the fill/legend variable (the second table dimension, or first if swap = TRUE). .useTheme (default) resolves to pal(getTheme()$palette, n = <number of levels - the active theme's qualitative palette (see theme), sampled or interpolated to match the number of category levels. A diverging or sequential color ramp is deliberately not used here: the fill variable is an unordered categorical variable, and a ramp would visually suggest an ordering between levels that doesn't exist.
border	color of the tile borders. Default "white".
legend	logical. Draw a legend for the fill variable. Default TRUE.
labels	character, one of "p", "n", "none". Cell labels showing the proportion of the table total ("p"), the absolute frequency ("n"), or no labels ("none"). Labels are only drawn for tiles large enough to hold them. Default "p".
labCex	numeric. Character expansion factor for cell labels. Default 0.8.
labDigits	integer. Number of decimal digits for percentage cell labels when labels = "p". Default 1.
stamp	controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/ NULL, or an explicit string, as for .withGraphicsState() (internal).
...	further graphical parameters passed to par() via .applyParFromDots(), e.g. mar, cex.axis, las.

Details

Cells belonging to a row with a row total of zero are drawn as zero-height tiles and receive no label, but do not cause an error or NaN in the geometry.

In both orientations only the first table dimension receives an axis (its split points are constant across the whole plot); the second dimension's split points vary per column/row and are represented via the legend only. No numeric probability axis is drawn. The second dimension is stacked top-down (vertical) resp. left-to-right (horiz = TRUE): its first level sits at the top / left, matching graphics::spineplot() and the legend order.

Value

Invisibly, the data.frame of tile geometry as produced by .computeMosaicTiles(), with columns x0, x1, y0, y1, n, p, pCond and one column per table dimension. This allows users to add further annotations (e.g. via rect() or text()) on top of the plot.

See Also

plotCatDist(), plotBar(), getTheme()

Other plot.bivariate: plotAssoc(), plotBag(), plotCor(), plotDens2D(), plotHeatmap(), plotHexbin(), plotXY()

Examples

```
tab <- apply(HairEyeColor, c(1, 2), sum)

plotMosaic(tab)
plotMosaic(tab, swap = TRUE)
plotMosaic(tab, horiz = TRUE, main = "Hair ~ Eyecolor")
```

---

plotPolar *Polar Plot for Radial Data*

Description

Creates a polar coordinate plot for one or multiple radial series.

Usage

```
plotPolar(
  r,
  theta = NULL,
  main = NULL,
  type = "p",
  rlim = NULL,
  col = NULL,
  border = NULL,
  add = FALSE,
  ...
)
```

Arguments

r	numeric vector or matrix of radial values. Each row represents one series.
theta	optional numeric vector or matrix of angles (in radians). Must match the dimensions of r. If NULL, equally spaced angles are used.
main	optional plot title.
type	character vector specifying plot type for each series: **"p"** points **"l"** polygon (connected and optionally filled) **"h"** radial segments ("histogram"-style)
rlim	numeric limit for radial axis. If NULL, determined automatically.
col	color for points/lines.
border	color for border in case of type polygon.
add	logical; if TRUE, adds to an existing plot.
...	additional graphical parameters passed to base plotting functions.

Details

The function supports plotting multiple radial series simultaneously. Each row of `r` is treated as a separate series.

Angles are interpreted in radians. If `theta` is not provided, points are distributed evenly over $[0, 2\pi)$.

Graphical parameters such as `lwd`, `lty`, `pch`, `cex`, `fill`, and `mar` can be passed via `...`.

Value

Invisibly returns `NULL`.

See Also

Other plot.special: `plotBinaryTree()`, `plotCirc()`, `plotLift()`, `plotMiss()`, `plotPropCI()`, `plotTernary()`, `plotTimeSeries()`, `plotTreemap()`, `plotWeb()`

Examples

```
r <- matrix(runif(20), nrow = 2)
plotPolar(r, type = c("l", "p"), col = c("blue", "red"))

# with custom angles
theta <- seq(0, 2*pi, length.out = 10)
plotPolar(r[1,], theta = theta, type = "h")
```

---

plotProbDist *Plot Probability Distribution*

Description

Produces a plot of a probability distribution function with shaded areas. Useful for illustrating critical regions, p-values, or probability masses in statistics teaching materials.

Usage

```
plotProbDist(
  breaks,
  FUN,
  main = "",
  xlim = NULL,
  col = NULL,
  density = 7,
  ylab = "density",
  areaLabels = NULL,
  breakLabels = NULL,
  grid = FALSE,
  box = .useTheme,
  ...
)
```

Arguments

breaks	numeric vector of break points defining the boundaries between shaded areas. The first and last value define the plot range passed to shade(); interior values are the actual boundaries.
FUN	the distribution density function to plot, typically of the form function(x) dnorm(x, mean=0, sd=1).
main	main title for the plot. NULL or "" suppresses the title and reduces the top margin automatically.
xlim	numeric vector of length 2. x-axis limits passed to curve().
col	color(s) for the shaded areas. Recycled if shorter than the number of areas. Defaults to the "helsana" palette.
density	density of shading lines passed to shade(). Default is 7.
ylab	label for the y-axis. Default is "density".
areaLabels	controls labels placed in the centre of each shaded area. NULL (default) suppresses labels. TRUE uses LETTERS as default labels. A character vector sets explicit labels. A named list overrides individual arguments passed to boxedText() (e.g. list(cex = 3) or list(x = c(-2, 3), y = 0.1) for manual positioning).
breakLabels	controls labels placed on the x-axis at interior break points via mtext(). NULL (default) suppresses labels. TRUE uses LETTERS as default labels. A character vector sets explicit labels. A named list overrides individual arguments (e.g. list(cex = 1.8, font = 1)).
grid	controls background grid. FALSE (default) suppresses the grid. TRUE or .useTheme draws a grid according to the current theme. A named list overrides individual arguments passed to grid().
box	controls the plot box. .useTheme (default) uses the current theme setting. TRUE/FALSE forces the box on or off. A named list overrides individual arguments passed to box().
...	further graphical parameters passed to curve() and .applyParFromDots(), e.g. las, col.axis.

Value

NULL, invisibly.

See Also

curve

Other plot.distribution: plotFun(), shade()

Examples

```
# Normal distribution with two areas
plotProbDist(breaks = c(-10, -1, 12),
             FUN    = function(x) dnorm(x, mean = 2, sd = 2),
             main   = "Normal-Distribution N(2,2)",
             xlim   = c(-7, 10),
             col    = c("deeppink4", "skyblue3"),
             density = c(20, 7),
             breakLabels = TRUE)
```

```
# t-distribution with three areas
plotProbDist(breaks = c(-6, -2.3, 1.5, 6),
             FUN    = function(x) dt(x, df = 8),
             main   = "t-Distribution (df=8)",
             xlim   = c(-4, 4),
             col    = c("deeppink4", "skyblue3", "darkorange2"),
             density = c(20, 7),
             areaLabels = FALSE,
             breakLabels = list(text=c("A", "B")))

# Chi-square distribution
plotProbDist(breaks  = c(0, 15, 35),
             FUN     = function(x) dchisq(x, df = 8),
             main    = expression(chi^2~"-Distribution (df=8)"),
             xlim    = c(0, 30),
             col     = c("deeppink4", "skyblue3"),
             density = c(0, 20),
             breakLabels = list(text="B"))
```

`plotPropCI` *Plot Proportions with Confidence Intervals*

Description

Displays a horizontal bar with nested confidence interval bands from `min(ciLevels)` to `max(ciLevels)` to visualise the uncertainty of a proportion. Bands are drawn in a semi-transparent grey so that the accumulated overlap creates a natural density gradient - darker in the centre, lighter at the edges.

Usage

```
plotPropCI(
  x,
  main = NULL,
  labels = c("", ""),
  xlab = "",
  xlim = c(0, 1),
  col = .useTheme,
  ci.col = addOpacity("grey80", 0.12),
  border = NA,
  ciLevels = seq(0.99, 0.8, by = -0.01),
  grid = .useTheme,
  box = FALSE,
  legend = TRUE,
  stamp = .useTheme,
  ...
)
```

Arguments

`x`	A two-column integer matrix where each row represents a group and the two columns contain counts for the two categories. A numeric vector of length 2 is also accepted and will be coerced to a one-row matrix.

main	main title of the plot. NULL (default) derives a title from deparse(substitute(x)). "", NA, or FALSE suppress the title and compact the top margin. Any other string is used as given.
labels	character vector of length 2 with labels for the two categories, displayed at the top of the plot. Default c("", "").
xlab	label for the x-axis. Default "".
xlim	numeric vector of length 2 for the x-axis limits. Default c(0, 1).
col	character vector of length 2 specifying fill colours for the stacked bar. .useTheme (default) resolves to getTheme()$twin - the active theme's two-color pair. Note this is purely "first label gets the first color"; unlike plotCor/plotWeb, there is no positive/ negative sign convention here, since proportions of two arbitrary categories (e.g. "yes"/"no") have no inherent sign.
ci.col	colour for the confidence interval bands. Default is a semi-transparent grey (addOpacity("grey80", 0.12)). Deliberately not theme-driven (like the sequential scales in plotDens2D/plotHeatmap): this is a structural mechanism (many overlapping translucent bands building a gradient via overdraw), not a categorical or diverging color choice.
border	border colour for the confidence interval bands and the stacked bar. Default NA (no border).
ciLevels	numeric vector of confidence levels for the nested bands. Default seq(0.99, 0.80, by = -0.01) (20 bands, 99\ 80\ bands share the same translucent color and no border.
grid	controls drawing of the background grid (vertical lines at the proportion ticks only - there is no meaningful horizontal grid for the categorical group axis). .useTheme (default) follows the active theme (getTheme()$grid). TRUE/FALSE/ NA, or a named list, as for grid.
box	controls drawing of the plot box. Default FALSE (no frame, consistent with this chart's minimal "Few"-style appearance). TRUE/NA, or a named list, as for box.
legend	controls the legend explaining the CI band range. TRUE (default) draws it. FALSE/NA suppresses it. A named list overrides arguments forwarded to legend.
stamp	controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/ NULL, a string, or a named list for stamp().
...	further arguments passed to par via the internal framework, and to barplot.

Details

Each row of x is displayed as a horizontal stacked bar showing the proportion of the first category. Confidence intervals are calculated using prop.test() and drawn as nested bands per ciLevels, all in the same semi-transparent colour. The repeated overdraw naturally darkens the centre of the interval where all bands overlap. A vertical segment marks the observed proportion.

Value

Invisibly returns NULL. Called for its side effect of producing a plot.

See Also

prop.test, theme

Other plot.special: plotBinaryTree(), plotCirc(), plotLift(), plotMiss(), plotPolar(), plotTernary(), plotTimeSeries(), plotTreemap(), plotWeb()

Examples

```
m <- matrix(c(22, 111, 33, 120, 80, 100), nrow = 3, byrow = TRUE)
plotPropCI(m, labels = c("yes", "no"), main = "Response by Group")

# single row - vector input is accepted
plotPropCI(m[1, ], labels = c("yes", "no"))
```

---

plotQQ *QQ-Plot for Any Distribution*

Description

Create a QQ-plot for a variable of any distribution. The assumed underlying distribution can be defined as a function of f(p), including all required parameters. Confidence bands are provided by default.

Usage

```
plotQQ(
  x,
  qdist = stats::qnorm,
  main = NULL,
  xlab = NULL,
  ylab = NULL,
  datax = FALSE,
  add = FALSE,
  grid = .useTheme,
  box = .useTheme,
  cband = list(conf.level = 0.95),
  qqline = TRUE,
  stamp = .useTheme,
  ...
)
```

Arguments

x	the data sample
qdist	the quantile function of the assumed distribution. Can either be given as simple function name or defined as own function using the required arguments. Default is qnorm(). See examples.
main	the main title for the plot. This will be "Q-Q-Plot" by default
xlab	the xlab for the plot
ylab	the ylab for the plot
datax	logical. Should data values be on the x-axis? Default is FALSE.
add	logical specifying if the points should be added to an already existing plot; defaults to FALSE.
grid	controls drawing of the background grid. .useTheme (default) follows the active theme (getTheme()$grid). TRUE/FALSE/NA, or a named list, as for grid.

box	controls drawing of the plot box. .useTheme (default) resolves to getTheme()$box. TRUE/FALSE/NA, or a named list, as for box.
cband	controls the confidence band. FALSE, NULL, or NA suppress it. A named list configures it; its conf.level element (default 0.95) controls the confidence level of the pointwise Kolmogorov-Smirnov-based band (conf.level is consumed before the band is drawn and never forwarded as a graphical parameter). Other list elements (e.g. col, border) configure the band's appearance.
qqline	arguments for the qqline. This will be estimated as a line through the 25\ procedure as qqline() does for normal distribution (instead of set it to abline(a = 0, b = 1)). The quantiles can however be overwritten by setting the argument probs to some user defined values. Also the method for calculating the quantiles can be defined (default is 7, see quantile). The line defaults are set to col = par("fg"), lwd = par("lwd") and lty = par("lty"). No line will be plotted if args.qqline is set to NA.
stamp	controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/NULL, a string, or a named list of arguments for stamp() (e.g. list(text = "...", las = 2)).
...	the dots are passed to the plot function.

Details

The function generates a sequence of points between 0 and 1 and transforms those into quantiles by means of the defined assumed distribution.

Note

The code is inspired by the tip 10.22 "Creating other Quantile-Quantile plots" from R Cookbook and based on R-Core code from the function qqline. The calculation of confidence bands are rewritten based on an algorithm published in the package BoutrosLab.plotting.general.

Based on code by Ying Wu

References

Teetor, P. (2011) *R Cookbook*. O'Reilly, pp. 254-255.

See Also

stats::qqnorm, stats::qqline, stats::qqplot, theme, lines.loess

Other plot.univariate: plotArea(), plotBar(), plotBox(), plotCatDist(), plotDens(), plotDensBox(), plotDot(), plotECDF(), plotFdist(), plotLines(), plotViolin()

Examples

```
y <- rexp(100, 1/10)
plotQQ(y, function(p) qexp(p, rate=1/10))

w <- rweibull(100, shape=2)
plotQQ(w, qdist = function(p) qweibull(p, shape=4))

z <- rchisq(100, df=5)
plotQQ(z, function(p) qchisq(p, df=5),
       args.qqline=list(col=2, probs=c(0.1, 0.6)),
```

```
       main=expression("Q-Q plot for" ~~ {chi^2}[nu == 3]))
abline(0,1)

# 90% confidence band instead of the 95% default
plotQQ(y, function(p) qexp(p, rate=1/10), cband = list(conf.level = 0.90))

# add 5 random sets
for(i in 1:5){
  z <- rchisq(100, df=5)
  plotQQ(z, function(p) qchisq(p, df=5), add=TRUE, args.qqline = NA,
         col="grey", lty="dotted")
}
```

---

plotRidge *Ridge Plot (Stacked Density Plot)*

Description

Draws stacked kernel density estimates (ridge plot) for grouped data. Each group is displayed as a density curve shifted along the y-axis.

Usage

```
## Default S3 method:
plotRidge(
  x,
  ...,
  add = FALSE,
  bw = "nrd0",
  scale = 1,
  spacing = 1,
  col = NULL,
  border = NULL,
  lwd = 1,
  lty = 1,
  fill = TRUE,
  grid = NA,
  main = "",
  xlab = "",
  ylab = "",
  xlim = NULL,
  ylim = NULL
)

## S3 method for class 'formula'
plotRidge(
  formula,
  data = NULL,
  subset,
  na.action = na.omit,
  ...,
```

```
  add = FALSE,
  bw = "nrd0",
  scale = 1,
  spacing = 1,
  col = NULL,
  border = NULL,
  lwd = 1,
  lty = 1,
  fill = TRUE,
  grid = NA,
  main = "",
  xlab = "",
  ylab = "",
  xlim = NULL,
  ylim = NULL
)
```

Arguments

x	A numeric vector, or a list of numeric vectors representing groups.
...	additional graphical parameters passed to par().
add	logical; if TRUE, adds to an existing plot.
bw	bandwidth for density.
scale	scaling factor for density height.
spacing	vertical spacing between ridges.
col	fill color(s).
border	border color(s).
lwd	line width(s).
lty	line type(s).
fill	logical; fill area under densities.
grid	logical, NA, or list controlling grid.
main, xlab, ylab	plot labels.
xlim, ylim	axis limits.
formula	A formula of the form y ~ group.
data	optional data frame.
subset	optional subset expression.
na.action	function to handle missing values.

Details

Ridge plots are useful for comparing distributions across multiple groups. Each density is normalized and vertically offset, improving readability compared to overlaid density plots.

Value

Invisibly returns NULL.

See Also

plotDens

Examples

```
set.seed(1)
df <- data.frame(
  value = c(rnorm(100), rnorm(100, 2), rnorm(100, 4)),
  group = rep(c("A", "B", "C"), each = 100)
)

plotRidge(value ~ group, data = df)
```

---

plotTernary *Ternary Plot*

Description

Draws a ternary plot for compositional data consisting of three non-negative components per observation. Each row is interpreted as a composition and internally rescaled to sum to 1 if necessary.

Usage

```
plotTernary(
  x,
  y = NULL,
  z = NULL,
  ...,
  add = FALSE,
  col = NULL,
  pch = 16,
  cex = 1,
  grid = NA,
  lbl = NULL,
  main = "",
  xlim = c(-1, 1),
  ylim = c(-0.5, 1)
)
```

Arguments

x	A numeric vector, matrix, or data frame. If y and z are provided, x, y, and z are combined into a matrix. Otherwise, x must contain exactly three columns.
y	optional numeric vector for the second component.
z	optional numeric vector for the third component.
...	additional graphical parameters passed to par().
add	logical; if TRUE, adds to an existing plot.
col	point color(s).
pch	plotting symbol.

cex	point size.
grid	logical, NA, or list controlling the ternary grid.
lbl	character vector of length 3 specifying axis labels.
main	plot title.
xlim, ylim	plot limits (usually left at defaults).

Details

A ternary plot represents three-part compositions on an equilateral triangle. Each observation (x_1, x_2, x_3) is mapped to 2D coordinates using barycentric transformation. If rows do not sum to 1, they are automatically normalized with a warning.

Graphical elements such as grids are controlled via the unified plot design system using `bedrock::callIf()` and `.theme()`.

Value

Invisibly returns NULL.

See Also

plotDens, plotRidge

Other plot.special: plotBinaryTree(), plotCirc(), plotLift(), plotMiss(), plotPolar(), plotPropCI(), plotTimeSeries(), plotTreemap(), plotWeb()

Examples

```
set.seed(1)
x <- runif(100)
y <- runif(100)
z <- runif(100)

plotTernary(x, y, z)

# matrix input
M <- cbind(x, y, z)
plotTernary(M, lbl = c("A", "B", "C"))
```

---

plotTimeSeries *Combined Plot of a Time Series and Its ACF and PACF*

Description

Combined plot of a time Series and its autocorrelation and partial autocorrelation

Usage

```
plotTimeSeries(
  x,
  maxLag = 10 * log10(length(x)),
  ylab = NULL,
  main = NULL,
  ...
)
```

Arguments

x	univariate time series.
maxLag	integer. Defines the number of lags to be displayed. The default is 10 * log10(length(series)).
ylab	a title for the y axis: see title.
main	an overall title for the plot
...	the dots are passed to the plot command.

Details

plotTimeSeries plots a combination of the time series and its autocorrelation and partial autocorrelation.

Note

Rewritten based on ideas of M.Huerzeler

See Also

ts

Other plot.special: plotBinaryTree(), plotCirc(), plotLift(), plotMiss(), plotPolar(), plotPropCI(), plotTernary(), plotTreemap(), plotWeb()

Examples

```
plotTimeSeries(AirPassengers)
```

---

plotTreemap *Treemap Plot*

Description

Draws a treemap in which the area of each rectangle is proportional to the corresponding value in x. Optionally, rectangles can be grouped into higher-level regions.

Usage

```
plotTreemap(
  x,
  groups = NULL,
  area = NULL,
  labels = NULL,
  groupArea = NULL,
  groupLabels = NULL,
  main = NULL,
  ...
)
```

Arguments

x	numeric vector of positive values determining the rectangle sizes.
groups	optional grouping variable. Values sharing the same group are placed within a common enclosing region.
area	controls the appearance of individual rectangles. • NULL or TRUE: use defaults. • FALSE or NA: suppress rectangle fill. • Atomic vector: interpreted as col. • List: graphical parameters such as col, border, and lwd.
labels	controls the labels of individual rectangles. • NULL or TRUE: use default labels (names(x)). • FALSE or NA: suppress labels. • Character vector: interpreted as label text. • List: label properties such as text, col, and cex.
groupArea	controls the appearance of enclosing group regions. Uses the same conventions as area.
groupLabels	controls the labels of enclosing group regions. Uses the same conventions as labels. By default, group names are used when more than one group is present.
main	main title of the plot.
...	additional graphical parameters passed to .applyParFromDots().

Details

The appearance of individual rectangles and groups is controlled through the area, labels, groupArea, and groupLabels arguments. These accept logical values, vectors, or lists.

Individual rectangles are sized according to the values in x. When groups is supplied, a treemap is first constructed for the groups, and each group's area is then subdivided among its members.

The arguments area, labels, groupArea, and groupLabels provide a flexible interface for controlling the appearance of the plot while keeping the main function signature compact.

Value

Invisibly returns a list containing the coordinates of group centres and the centres of their child rectangles.

See Also

Other plot.special: plotBinaryTree(), plotCirc(), plotLift(), plotMiss(), plotPolar(), plotPropCI(), plotTernary(), plotTimeSeries(), plotWeb()

Examples

```
x <- c(A = 6, B = 5, C = 4, D = 3, E = 2, F = 1)

plotTreemap(x)

plotTreemap(
  x,
  labels = list(col = "white")
)

grp <- c("G1", "G1", "G1", "G2", "G2", "G2")

plotTreemap(
  x,
  groups = grp,
  groupLabels = TRUE
)

plotTreemap(
  x,
  groups = grp,
  area = terrain.colors(length(x)),
  labels = FALSE,
  groupArea = list(border = "black", lwd = 2)
)
```

---

plotViolin *Violin Plot*

Description

Draws violin plots for one or more groups, combining kernel density estimation with optional boxplot overlays. The function follows a boxplot-like interface and supports both default and formula methods.

Usage

```
plotViolin(x, ...)

## Default S3 method:
plotViolin(
  x,
  ...,
  main = NULL,
  xlab = "",
  ylab = "",
  xlim = NULL,
```

```
  ylim = NULL,
  horizontal = FALSE,
  at = NULL,
  names = NULL,
  add = FALSE,
  bw = "nrd0",
  trim = TRUE,
  col = "grey80",
  border = "black",
  lwd = 1,
  box = TRUE,
  grid = NA,
  quantiles = NULL
)

## S3 method for class 'formula'
plotViolin(
  formula,
  data = NULL,
  subset,
  na.action = na.omit,
  ...,
  main = NULL,
  xlab = "",
  ylab = "",
  xlim = NULL,
  ylim = NULL,
  horizontal = FALSE,
  at = NULL,
  names = NULL,
  add = FALSE,
  bw = "nrd0",
  trim = TRUE,
  col = "grey80",
  border = "black",
  lwd = 1,
  box = TRUE,
  grid = NA,
  quantiles = NULL
)
```

Arguments

x	numeric vector, list of numeric vectors, or first group.
...	additional data vectors (unnamed) or graphical parameters passed to par().
main, xlab, ylab	plot labels.
xlim, ylim	axis limits.
horizontal	logical; if TRUE, draws horizontal violins.
at	numeric positions of the groups.
names	optional group labels.
add	logical; if TRUE, adds to an existing plot.

bw	bandwidth specification passed to density().
trim	logical. If TRUE (default), the kernel density estimate of each group is restricted to the observed data range (from = min(x), to = max(x)), so the violin never extends beyond the actual data — matching the default behavior of ggplot2::geom_violin(). If FALSE, density() is called with its own defaults, which extend the tails up to cut * bw beyond range(x) and may produce violins that reach into implausible values (e.g. scores above 100 or below 0).
col	fill color(s) of the violins.
border	border color(s) of the violins.
lwd	line width for violin borders.
box	logical or list controlling the boxplot overlay (see Details).
grid	logical, NA, or list controlling background grid.
quantiles	optional numeric vector of probabilities for drawing quantile lines inside each violin.
formula	A formula of the form y ~ group.
data	optional data frame.
subset	optional subset expression.
na.action	function to handle missing values.

Details

The violin shape is constructed from a kernel density estimate of each group, scaled to a fixed maximum width. Optionally, boxplots and quantile lines can be added.

Graphical elements such as the boxplot overlay and grid are controlled via a flexible interface using TRUE, FALSE, NA, or list(...) and are evaluated using bedrock::callIf().

Value

Invisibly returns NULL.

Data Handling

The function accepts:

- a numeric vector
- multiple vectors via ...
- a list of numeric vectors

Groups are handled similarly to boxplot().

See Also

boxplot, density

Other plot.univariate: plotArea(), plotBar(), plotBox(), plotCatDist(), plotDens(), plotDensBox(), plotDot(), plotECDF(), plotFdist(), plotLines(), plotQQ()

Examples

```
set.seed(1)
x <- rnorm(100)
y <- rnorm(100, 1)

plotViolin(x, y)

# horizontal violins
plotViolin(x, y, horizontal = TRUE)

# with quantiles
plotViolin(x, y, quantiles = c(0.25, 0.5, 0.75))

# untrimmed: tails extend beyond the observed data range
plotViolin(x, y, trim = FALSE)

# custom styling
plotViolin(x, y,
  col = c("lightblue", "salmon"),
  box = list(col = "white"),
  grid = TRUE
)

# formula interface
df <- data.frame(
  value = rnorm(200),
  group = rep(letters[1:4], each = 50)
)

plotViolin(value ~ group, data = df)
```

`plotWeb` *Plot a Web of Connected Points*

Description

Displays a symmetric matrix (typically a correlation matrix) as a network diagram: variables are placed equidistantly around a circle, and connecting lines between each pair are drawn with width proportional to the absolute value of the matrix entry and color indicating its sign.

Usage

```
plotWeb(
  m,
  main = NULL,
  dist = 0.5,
  col = hcol("red", "blue"),
  lty = NULL,
  lwd = NULL,
  labels = TRUE,
  points = list(pch = 21, cex = 2, col = "black", bg = "darkgrey"),
  legend = TRUE,
```

```
  stamp = .useTheme,
  ...
)
```

Arguments

m	a symmetric numeric matrix (e.g. a correlation matrix).
main	main title of the plot. NULL (default) produces no title.
dist	distance of node labels from the outer circle. Default 0.5.
col	two colors for the connecting lines: the first is used for negative values, the second for positive values. .useTheme (default) resolves to getTheme()$twin - consistent with the sign-based coloring in plotCor.
lty	line type for the connecting lines. NULL (default) inherits from par("lty").
lwd	line widths for the connecting lines. NULL (default) scales widths linearly between 0.5 and 10 in proportion to the absolute matrix values.
labels	controls node labels around the circle. TRUE (default) draws labels using colnames(m) with default styling. FALSE/NA/NULL suppresses labels. A named list overrides individual settings: labels character vector of label texts; defaults to colnames(m) las orientation: 1 horizontal (default), 2 radial, 3 vertical adj label adjustment (0/0.5/1); NULL (default) chooses automatically based on position around the circle cex character expansion; default 1.0
points	controls the node point symbols. The default (list(pch=21, cex=2, col="black", bg="darkgrey")) draws prominent filled circles, larger than the generic data-point default, since nodes represent variables rather than observations. FALSE/NA/NULL suppresses the points. A named list overrides individual elements (pch, cex, col, bg).
legend	controls the legend. TRUE (default) draws a legend showing the line widths and colors for the minimum/maximum positive and negative values. FALSE/NA suppresses it. A named list overrides arguments forwarded to legend.
stamp	controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/ NULL, a string, or a named list for stamp().
...	further graphical parameters passed to par and to the internal canvas() call.

Details

The function uses the lower triangular matrix of m; when overriding lwd or col manually, values must be supplied in the same order as m[lower.tri(m)].

Value

Invisibly returns a list of x/y coordinates of the node points, useful for adding further annotations to the plot.

See Also

plotCor, theme

Other plot.special: plotBinaryTree(), plotCirc(), plotLift(), plotMiss(), plotPolar(), plotPropCI(), plotTernary(), plotTimeSeries(), plotTreemap()

Examples

```
m <- cor(swiss)
plotWeb(m, main = "Swiss correlation")

# custom labels (abbreviations)
plotWeb(m, labels = list(labels = abbreviate(colnames(m), 4), cex = 0.9))

# show only significant correlations
p <- outer(
  (vars <- colnames(mtcars)), vars,
  Vectorize(function(v1, v2)
    cor.test(mtcars[[v1]], mtcars[[v2]])$p.value)
)
dimnames(p) <- list(vars, vars)
m2 <- cor(mtcars)
m2[p > 0.05] <- NA
plotWeb(m2, labels = list(las = 2))
```

---

plotXY *Scatterplot with Optional Smooth Lines*

Description

Draws a scatterplot of two numeric variables with optional linear and locally weighted regression lines, and an optional legend.

Usage

```
plotXY(x, ...)

## Default S3 method:
plotXY(
  x,
  y,
  main = NULL,
  xlab = "",
  ylab = "",
  xlim = NULL,
  ylim = NULL,
  col = .useTheme,
  bg = .useTheme,
  pch = .useTheme,
  cex = .useTheme,
  grid = .useTheme,
  box = .useTheme,
  lm = TRUE,
  loess = TRUE,
  legend = TRUE,
  stamp = .useTheme,
  ...
)
```

```
## S3 method for class 'formula'
plotXY(
  formula,
  data,
  subset,
  na.action = na.omit,
  main = NULL,
  xlab = "",
  ylab = "",
  xlim = NULL,
  ylim = NULL,
  col = .useTheme,
  bg = .useTheme,
  pch = .useTheme,
  cex = .useTheme,
  grid = .useTheme,
  lm = TRUE,
  loess = TRUE,
  legend = TRUE,
  box = .useTheme,
  stamp = .useTheme,
  ...
)
```

Arguments

x	numeric vector of x-values, or a formula of the form y ~ x.
...	further graphical parameters passed to par() via the internal framework.
y	numeric vector of y-values (ignored if a formula is used).
main	main title of the plot. NULL (default) derives a title from the input - deparse(y) ~ deparse(x) for the default method, or the formula's data.name for the formula method. "", NA, or FALSE suppress the title entirely (and compact the top margin accordingly); any other string is used as given (resolved internally via .resolveTitle()).
xlab	label for the x-axis.
ylab	label for the y-axis.
xlim	numeric vector of length 2; x-axis limits. If NULL (default), the range of x is used.
ylim	numeric vector of length 2; y-axis limits. If NULL (default), the range of y is used.
col	color of the points. .useTheme (default) resolves to getTheme()$points$col.
bg	background (fill) color of the points. .useTheme (default) resolves to getTheme()$points$bg.
pch	plotting character. .useTheme (default) resolves to getTheme()$points$pch.
cex	character expansion factor for points. .useTheme (default) resolves to getTheme()$points$cex.
grid	controls drawing of the background grid. Can be: • .useTheme (default): follow the active theme (getTheme()$grid) • TRUE: draw grid with theme settings

- FALSE, NULL, or NA: suppress grid
- a named list: arguments passed to grid, overriding the theme defaults for this call only

box — controls drawing of the plot box. Can be:

- .useTheme (default): follow the active theme (getTheme()$box)
- TRUE: draw box with theme settings
- FALSE, NULL, or NA: suppress box
- a named list: arguments passed to box, overriding the theme defaults for this call only

lm — controls drawing of the linear regression line. Can be:

- TRUE: draw with default settings
- FALSE, NULL, or NA: suppress
- a named list: arguments passed to lines, e.g. list(col = "blue", lwd = 2)

loess — controls drawing of the locally weighted regression line. Can be:

- TRUE: draw with default settings
- FALSE, NULL, or NA: suppress
- a named list: arguments passed to lines, e.g. list(col = "red", lty = "dashed")

legend — controls drawing of the legend. Can be:

- TRUE: draw with default settings (position "topright")
- FALSE, NULL, or NA: suppress
- a named list: arguments passed to legend, e.g. list(x = "bottomleft")

The legend is only drawn when at least one of lm or loess is active. lm/loess line colors are taken from the active theme's twin colors (getTheme()$twin).

stamp — controls the corner stamp. .useTheme (default) resolves to getTheme()$stamp. TRUE/FALSE/ NULL, a string, or a named list for stamp().

formula — a formula of the form y ~ x.

data — an optional data frame containing variables in the formula.

subset — optional expression indicating which observations to use.

na.action — a function specifying how missing values are handled. Defaults to na.omit.

Details

Optional plot components (grid, box, lm, loess, legend) follow callIf semantics:

- TRUE: draw with defaults
- FALSE, NULL, or NA: suppress component
- named list: customize component arguments

col, bg, pch, cex, grid, and box default to .useTheme, deferring to the package's active theme (see theme) rather than a hardcoded value. This means setTheme(list(points = list(col = "black"))) changes the point color for every call to plotXY() (and any other function using the same theme section) that doesn't override col explicitly.

Value

Invisibly returns NULL.

See Also

plot, lm, loess, callIf

Other plot.bivariate: plotAssoc(), plotBag(), plotCor(), plotDens2D(), plotHeatmap(), plotHexbin(), plotMosaic()

Examples

```
## Not run:
plotXY(temperature ~ delivery_min, bedrock::Pizza,
       main = "Temperature vs. Delivery Time")

# Suppress loess, customize lm line
plotXY(temperature ~ delivery_min, bedrock::Pizza,
       lm    = list(col = "darkred", lwd = 2),
       loess = FALSE)

# No title, compact top margin
plotXY(temperature ~ delivery_min, bedrock::Pizza, main = "")

## End(Not run)
```

---

polarGrid *Draw a Polar Grid with Optional Labels*

Description

Adds a polar coordinate grid (circles and radial lines) to an existing plot. Optionally includes labels for radii and angles.

Usage

```
polarGrid(
  nr = NULL,
  ntheta = NULL,
  col = "lightgray",
  lty = "dotted",
  lwd = par("lwd"),
  rlabels = NULL,
  alabels = NULL,
  lblradians = FALSE,
  cex.lab = 1,
  las = 1,
  adj = NULL,
  dist = NULL
)
```

Arguments

nr	numeric or vector controlling radial grid lines: NULL Uses default "pretty" axis values.

	single numeric Number of radial grid lines. **numeric vector** Explicit radii at which to draw circles. **all** NA Suppress radial grid lines.
ntheta	numeric or vector controlling angular grid lines: NULL Uses 12 equally spaced angles. **single numeric** Number of angular divisions. **numeric vector** Explicit angles (in radians). **all** NA Suppress angular grid lines.
col	color of grid lines.
lty	line type for grid lines.
lwd	line width for grid lines.
rlabels	optional labels for radial grid lines (excluding zero). If NULL, labels are generated automatically. Use NA to suppress labels.
alabels	optional labels for angular grid lines. If NULL, labels are generated automatically (degrees or radians). Use NA to suppress labels.
lblradians	logical; if TRUE, angle labels are shown in radians, otherwise in degrees.
cex.lab	character expansion factor for labels.
las	integer controlling label orientation (as in par).
adj	numeric vector specifying text justification.
dist	numeric distance from origin for angular labels.

Details

This function is intended to be used together with polar plotting functions such as plotPolar. It assumes an existing plot with equal aspect ratio.

Radial grid lines are drawn as concentric circles, while angular grid lines are drawn as segments from the origin.

Label placement and formatting can be customized via adj, las, and dist.

Value

Invisibly returns NULL.

See Also

grid()

Other graphics.setup: canvas(), setBackCol()

Examples

```
plot(0, 0, type = "n", xlim = c(-1, 1), ylim = c(-1, 1), asp = 1)
polarGrid()

# custom grid
plot(0, 0, type = "n", xlim = c(-2, 2), ylim = c(-2, 2), asp = 1)
polarGrid(nr = 4, ntheta = 8, col = "gray")

# suppress labels
polarGrid(rlabels = NA, alabels = NA)
```

polygon *Draw Polygonal Geometries*

Description

Generic function for drawing polygon-based geometry objects. This function extends graphics::polygon() with support for geometry objects such as circle(), ellipse(), regPolygon() and ring() and further remains fully compatible with its original interface. #' For ordinary coordinate vectors the call is forwarded to polygon. Geometry objects such as circle, ellipse, regPolygon and ring are dispatched to specialised methods.

Usage

```
polygon(x, ...)

## Default S3 method:
polygon(
  x,
  y = NULL,
  density = NULL,
  angle = 45,
  border = NULL,
  col = NA,
  lty = par("lty"),
  ...,
  fillOddEven = FALSE
)

## S3 method for class 'ringGeometry'
polygon(x, rule = "evenodd", ...)

## S3 method for class 'polygonGeometry'
polygon(x, ...)
```

Arguments

x	an object to be drawn.
...	further arguments passed to the corresponding method.
y	numeric vector of y-coordinates.
density	density of shading lines.
angle	angle of shading lines in degrees.
border	border colour.
col	fill colour.
lty	line type.
fillOddEven	logical; should the odd-even rule be used for filling?
rule	character string specifying the filling rule passed to polypath. One of "evenodd" or "winding".

Value

Invisibly returns `x`.

See Also

Other geometry.structures: `arc()`, `band()`, `bezier()`, `circle()`, `ellipse()`, `regPolygon()`, `ring()`

Examples

```
canvas()

polygon(
  circle(radius = 1),
  col = "lightblue"
)

polygon(
  regPolygon(
    radius = 0.7,
    numVertices = 5
  ),
  border = "red"
)
```

`preview` *Preview an Object*

Description

Generic function for an explicit, on-demand preview of an object, as distinct from `print()`. `preview()` exists for object types where `print()` is shared with another package's S3 generic dispatch (e.g. class `"html"`, used both by **pharos** and **htmltools** for genuinely different purposes), so that registering an own `print.*` method would silently overwrite - or be overwritten by - the other package's behaviour.

Usage

```
preview(x, ...)

## Default S3 method:
preview(x, ...)
```

Arguments

`x`	object to preview.
`...`	further arguments passed to methods.

Details

The default method simply calls `print()`, so `preview()` is always safe to call even for types with no dedicated method.

See Also

as.html

`preview.html` *Print HTML markup as readable text*

Description

Renders an `"html"` object as text in the console: tags are stripped or translated (<sub>/<sup> become _/^, <b>//<i>/ become bold/italic via ANSI codes), common HTML entities (, β, ...) are decoded, and <table> blocks are rendered as aligned text tables.

Usage

```
## S3 method for class 'html'
preview(x, ...)
```

Arguments

`x`	an object of class `"html"`
`...`	further arguments, currently unused (kept for consistency with `print`)

Details

If output does not support ANSI styling, bold/italic markup is rendered as plain text (handled automatically by **cli**).

Value

`x`, invisibly

`print.Unit` *Print Object with Unit*

Description

S3 method for printing objects with a `"Unit"` class. Displays the value along with its associated unit.

Usage

```
## S3 method for class 'Unit'
print(x, ...)
```

Arguments

`x`	an object with class `"Unit"`.
`...`	additional arguments passed to `print()`.

Value

Invisibly returns `x`.

See Also

base::attr, bedrock::label

Other format: `convUnit()`, `fm()`, `fmCI()`, `style()`, `unit()`

Examples

```
x <- 10
unit(x) <- "m"
class(x) <- "Unit"
print(x)
```

`regPolygon` *Regular Polygon Geometry*

Description

Create a regular polygon.

Usage

```
regPolygon(x = 0, y = 0, radius = 1, numVertices = 6, startAngle = 0)
```

Arguments

`x, y`	centre coordinates.
`radius`	circumradius.
`numVertices`	number of vertices.
`startAngle`	staring angle in radians.

Value

An object inheriting from class `"regPolygonGeometry"`.

See Also

Other geometry.structures: `arc()`, `band()`, `bezier()`, `circle()`, `ellipse()`, `polygon()`, `ring()`

rgbToCmy *Convert RGB to CMY*

Description

Convert RGB colors to the CMY color space.

Usage

```
rgbToCmy(col, maxColorValue = 1)
```

Arguments

col	RGB matrix or hexadecimal colors.
maxColorValue	maximum channel value.

Value

Numeric CMY matrix.

See Also

color-conversion-overview

rgbToCol *Convert RGB Colors to the Nearest Named R Color*

Description

Match RGB colors to the nearest named R color.

Usage

```
rgbToCol(col, method = c("rgb", "hsv"), metric = c("euclidean", "manhattan"))
```

Arguments

col	RGB matrix or hexadecimal colors.
method	color space used for matching.
metric	distance metric.

Value

Character vector of named R colors.

See Also

color-conversion-overview

rgbToHex *Convert RGB to Hexadecimal Colors*

Description

Convert RGB values to hexadecimal color strings.

Usage

```
rgbToHex(col)
```

Arguments

col	RGB matrix.

Value

Character vector of hexadecimal colors.

See Also

color-conversion-overview

rgbToLong *Convert RGB to Long Integers*

Description

Encode RGB colors as long integers.

Usage

```
rgbToLong(col)
```

Arguments

col	RGB matrix.

Value

Integer vector.

See Also

color-conversion-overview

ring *Ring Geometry*

Description

Create one or more rings or ring segments.

Usage

```
ring(
  x = 0,
  y = 0,
  innerRadius = 0.5,
  outerRadius = 1,
  startAngle = 0,
  endAngle = 2 * pi,
  numPoints = 100
)
```

Arguments

x, y	centre coordinates.
innerRadius	radius of the inner boundary.
outerRadius	radius of the outer boundary.
startAngle, endAngle	start and end angle in radians.
numPoints	number of points used for each boundary.

Value

An object inheriting from class "ringGeometry" or a "geometryCollection".

See Also

Other geometry.structures: arc(), band(), bezier(), circle(), ellipse(), polygon(), regPolygon()

rotate *Rotate a Geometric Structure*

Description

Rotate a geometric structure by an angle theta around a centerpoint xy.

Usage

```
rotate(x, y = NULL, mx = NULL, my = NULL, theta = pi/3, asp = 1)
```

Arguments

x, y	vectors containing the coordinates of the vertices of the polygon , which has to be rotated. The coordinates can be passed in a plotting structure (a list with x and y components), a two-column matrix, ... See xy.coords.
mx, my	xy-coordinates of the center of the rotation. If left to NULL, the centroid of the structure will be used.
theta	angle of the rotation
asp	the aspect ratio for the rotation. Helpful for rotate structures along an ellipse.

Value

The function invisibly returns a list of the coordinates for the rotated shape(s).

See Also

polygon, regPolygon, ellipse, arc

Other geometry.transformation: transformXY()

Examples

```
op <- par(no.readonly = TRUE)
# let's have a triangle
canvas(main="Rotation")
x <- regPolygon(numVertices=3)

# and rotate
sapply( (0:3) * pi/6, function(theta) {
  xy <- rotate( x=x, theta=theta )
  polygon(xy, col=addOpacity("blue", 0.2))
} )

abline(v=0, h=0)

par(op)
```

---

setBackCol *Background of a Plot*

Description

Paints the background of the plot, using either the figure region, the plot region or both. It can sometimes be cumbersome to elaborate the coordinates and base R does not provide a simple function for that.

Usage

```
setBackCol(col = "grey", region = c("plot", "figure"), border = NA)
```

Arguments

col	the color of the background, if two colors are provided, the first is used for the plot region and the second for the figure region.
region	either "plot" or "figure"
border	color for rectangle border(s). Default is NA for no borders.

See Also

rect

Other graphics.setup: canvas(), polarGrid()

Examples

```
# use two different colors for the figure region and the plot region
plot(x = rnorm(100), col="blue", cex=1.2, pch=16,
     panel.first={setBackCol(c("red", "lightyellow"))
                  grid()})
```

---

shade *Produce a shaded Curve*

Description

Sometimes the area under a density curve has to be color shaded, for instance to illustrate a p-value or a specific region under the normal curve. This function draws a curve corresponding to a function over the interval [from, to]. It can plot also an expression in the variable xname, default x.

Usage

```
shade(expr, col = par("fg"), breaks, density = 10, n = 101, xname = "x", ...)
```

Arguments

expr	the name of a function, or a call or an expression written as a function of x which will evaluate to an object of the same length as x.
col	color to fill or shade the shape with. The default is taken from par("fg").
breaks	numeric, a vector giving the breakpoints between the distinct areas to be shaded differently. Should be finite as there are no plots with infinite limits.
density	the density of the lines as needed in polygon.
n	integer; the number of x values at which to evaluate. Default is 101.
xname	character string giving the name to be used for the x axis.
...	the dots are passed on to polygon.

Details

Useful for shading the area under a curve as often needed for explaining significance tests.

Value

A list with components x and y of the points that were drawn is returned invisibly.

See Also

polygon, curve

Other plot.distribution: plotFun(), plotProbDist()

Examples

```
curve(dt(x, df=5), xlim=c(-6,6),
    main=paste("Student t-Distribution Probability Density Function, df = ", 5, ")", sep=""),
      type="n", las=1, ylab="probability", xlab="t")

shade(dt(x, df=5), breaks=c(-6, qt(0.025, df=5), qt(0.975, df=5), 6),
      col=c("deeppink4", "skyblue3"), density=c(20, 7))
```

---

splineCI *Add a Spline Smoother*

Description

Fit a smoothing spline and optionally add confidence bands.

Usage

```
splineX(x, ...)

## Default S3 method:
splineX(x, ...)

## S3 method for class 'formula'
splineX(formula, data, subset, na.action = na.omit, weights, ...)

## S3 method for class 'SplineX'
lines(
  x,
  col = pal()[1],
  lwd = 2,
  lty = "solid",
  type = "l",
  bandArgs = list(conf.level = 0.95),
  ...
)
```

Arguments

x	spline object returned by splineX().
...	further arguments passed to smooth.spline.
formula	A formula of the form lhs ~ rhs, where lhs gives the response values and rhs the corresponding groups or explanatory variables.
data	an optional matrix or data frame (or similar; see model.frame) containing the variables in the formula. By default the variables are taken from environment(formula).
subset	an optional vector specifying a subset of observations to be used in the analysis.
na.action	A function which indicates what should happen when the data contain NAs. Defaults to getOption("na.action").
weights	optional vector of weights of the same length as x.
col	line color of the smoother.
lwd	line width.
lty	line type.
type	plotting type passed to lines.
bandArgs	controls the confidence band. May be TRUE, FALSE, NULL, NA, or a named list. The confidence level is specified via conf.level. Default is list(conf.level = 0.95).

Details

Confidence bands are controlled via bandArgs. These arguments can be:

- FALSE, NULL or NA: suppress the band
- TRUE: draw the band with default settings
- a named list: customize the band appearance and confidence level

See Also

loess, scatter.smooth

Other graphics.trendlines: lines.lm(), lines.loess()

Examples

```
op <- par(no.readonly = TRUE)
par(mfrow = c(1, 2))

x <- runif(100)
y <- rnorm(100)

plot(x, y)
lines(splineX(y ~ x))

plot(dist ~ speed, cars)
lines(splineX(dist ~ speed, cars))

plot(dist ~ speed, cars)
lines(
  splineX(dist ~ speed, cars),
  bandArgs = list(
```

```
    conf.level = 0.99,
    col = addOpacity("red", 0.3),
    border = "black"
  )
)

par(op)
```

---

spreadOut *Spread Out a Vector of Numbers To a Minimum Interval*

Description

Spread the numbers of a vector so that there is a minimum interval between any two numbers (in ascending or descending order). This is helpful when we want to place textboxes on a plot and ensure, that they do not mutually overlap.

Usage

```
spreadOut(x, mindist = NULL, cex = 1)
```

Arguments

x	a numeric vector which may contain NAs.
mindist	the minimum interval between any two values. If this is left to NULL (default) the function will check if a plot is open and then use 90%% of strheight().
cex	numeric character expansion factor; multiplied by par("cex") yields the final character size; the default NULL is equivalent to 1.

Details

spreadOut() starts at or near the middle of the vector and increases the intervals between the ordered values. NAs are preserved. spreadOut() first tries to spread groups of values with intervals less than mindist out neatly away from the mean of the group. If this doesn't entirely succeed, a second pass that forces values away from the middle is performed.

spreadOut() can also be used to avoid overplotting of axis tick labels where they may be close together.

Value

On success, the spread out values. If there are less than two valid values, the original vector is returned.

Note

This function is based on plotrix::spreadOut() and has been integrated here with some minor changes.

Based on code by Jim Lemon jim@bitwrit.com.au

See Also

strheight()

Other graphics.layout: abcCoords(), axTicks, axisBreak(), isValidPlotRegion(), lineToUser(), mar(), plotFacet()

Examples

```
spreadOut(c(1, 3, 3, 3, 3, 5), 0.2)
spreadOut(c(1, 2.5, 2.5, 3.5, 3.5, 5), 0.2)
spreadOut(c(5, 2.5, 2.5, NA, 3.5, 1, 3.5, NA), 0.2)

# this will almost always invoke the brute force second pass
spreadOut(rnorm(10), 0.5)
```

---

stamp *Date/Time/Directory Stamp the Current Plot*

Description

Stamp the current plot in the extreme lower right corner. A free text or expression can be defined as text to the stamp.

Usage

```
stamp(text = .useTheme, las = NULL, cex = 0.6, col = "grey40")
```

Arguments

text	character string, expression, or toggle controlling the stamp text. .useTheme (default) or TRUE resolve to getTheme()$stamp, evaluated lazily at draw time. FALSE, NULL, or NA suppress the stamp. Any other string or unevaluated expression() is used as given.
las	orientation; see par. NULL (default) inherits the current par("las"). las = 3 places the stamp vertically along the right edge instead of horizontally along the bottom.
cex, col	size and color of the stamp text.

Details

The text can be freely defined as option. If user and date should be included by default, the following option using an expression will help:

```
setDescToolsXOption(stamp=expression(gettextf('
Sys.getenv('USERNAME'), Today() )))
```

For R results may not be satisfactory if par(mfrow=) is in effect.

See Also

text

Other graphics.annotation: barText(), boxedText(), colLegend(), errBars(), textLegend(), titleRect()

Examples

```
plot(1:20)
stamp()
```

---

strAbbr *Abbreviate Strings Uniquely*

Description

Creates abbreviations of character strings such that they remain distinguishable within the input vector.

Usage

```
strAbbr(x, minchar = 1, method = c("left", "fix"))
```

Arguments

x	a character vector
minchar	integer; minimum number of characters to retain
method	character string specifying the abbreviation strategy: • "left": abbreviate each string individually to the shortest unique prefix • "fix": use a common prefix length for all strings such that they are distinguishable

Details

The function ensures that abbreviations are unique (within the given vector) while respecting the minimum length minchar.

For method = "left", each string is shortened individually to the shortest prefix that distinguishes it from all others.

For method = "fix", a single prefix length is chosen such that all strings are distinguishable.

Unicode-aware substring operations are performed using the **stringi** package.

Value

A character vector of abbreviated strings.

See Also

stri_sub, stri_length

string-overview for an overview of all string utilities in pharos.

Examples

```
x <- c("apple", "apricot", "banana")

# individual minimal abbreviations
strAbbr(x, method = "left")

# fixed length abbreviations
strAbbr(x, method = "fix")
```

---

strAlign *Align Strings*

Description

Aligns a character vector to the left, right, center, or with respect to the first occurrence of a specified separator.

Usage

```
strAlign(x, sep = "\\r")
```

Arguments

x	a character vector
sep	a character specifying the alignment mode: • `"\l"`: left alignment • `"\r"`: right alignment (default) • `"\c"`: centered alignment • any other character: align at the first occurrence of this separator

Details

Alignment is achieved by padding strings with spaces so that all elements have equal width. This is mainly useful for monospaced output.

For left, right, and center alignment, strings are padded to the maximum width using spaces.

If a separator is provided, strings are aligned such that the first occurrence of the separator is vertically aligned. If a string does not contain the separator, it is treated as if the separator occurred at the end.

This function uses Unicode-aware string handling via the **stringi** package.

Value

A character vector of aligned strings.

See Also

stri_pad, stri_sub, stri_trim_right

string-overview for an overview of all string utilities in pharos.

Examples

```
x <- c("here", "there", "everywhere")

# right align (default)
strAlign(x)

# left align
strAlign(x, "\\l")

# center align
strAlign(x, "\\c")

# align at decimal separator
z <- c("6.0", "45.12", "784")
strAlign(z, ".")
```

---

strCap *Capitalize Strings*

Description

Capitalizes character strings in different ways: first letter, each word, or title case.

Usage

```
strCap(x, method = c("first", "word", "title"))
```

Arguments

x	a character vector
method	character string specifying the capitalization method:

- `"first"`: capitalize the first letter of the string
- `"word"`: capitalize the first letter of each word
- `"title"`: title case, excluding common stopwords

Details

The function uses Unicode-aware transformations from the **stringi** package.

For `method = "title"`, common stopwords (e.g., `"a"`, `"the"`, `"of"`) remain lowercase unless they appear as part of another word.

Value

A character vector with capitalized strings.

See Also

`stri_trans_totitle`, `stri_split_boundaries`

string-overview for an overview of all string utilities in pharos.

Examples

```
x <- c("hello world", "the lord of the rings")

# first letter
strCap(x, "first")

# each word
strCap(x, "word")

# title case
strCap(x, "title")
```

---

strChop *Split a String into a Number of Sections of Defined Length*

Description

Splitting a string into a number of sections of defined length is needed, when we want to split a table given as a number of lines without separator into columns. The cutting points can either be defined by the lengths of the sections or directly by position.

Usage

```
strChop(x, len = NULL, pos = NULL)
```

Arguments

x	the string to be cut in pieces
len	a vector with the lengths of the pieces
pos	a vector of cutting positions. Will be ignored when len has been defined.

Details

If length is going over the end of the string the last part will be returned, so if the rest of the string is needed, it's possible to simply enter a big number as last partlength.

len and pos can't be defined simultaneously, only alternatively.

Typical usages are

```
strChop(x, len) strChop(x, pos)
```

Value

a vector with the parts of the string.

See Also

strLeft, substr

string-overview for an overview of all string utilities in pharos.

Examples

```
x <- paste(letters, collapse="")
strChop(x=x, len = c(3,5,2))

# and with the rest integrated
strChop(x=x, len = c(3, 5, 2, nchar(x)))

# cutpoints at 5th and 10th position
strChop(x=x, pos=c(5, 10))
```

---

strCountW *Count Words in Strings*

Description

Counts the number of words in each element of a character vector.

Usage

```
strCountW(x)
```

Arguments

x	a character vector

Details

Words are detected using Unicode-aware word boundaries as implemented in **stringi**. This ensures robust handling of different languages, punctuation, and whitespace.

Value

An integer vector giving the number of words in each element of x.

See Also

stri_count_words

string-overview for an overview of all string utilities in pharos.

Examples

```
strCountW("This is a sentence.")

strCountW(c("One word", "Two words here", NA))
```

`strDist` *Compute Distances Between Strings*

Description

`strDist` computes distances between strings following to Levenshtein or Hamming method.

Usage

```
strDist(
  x,
  y,
  method = "levenshtein",
  mismatch = 1,
  gap = 1,
  ignoreCase = FALSE
)
```

Arguments

`x`	character vector, first string
`y`	character vector, second string
`method`	character, name of the distance method. This must be `"levenshtein"`, `"normlevenshtein"` or `"hamming"`. Default is `"levenshtein"`, the classical Levenshtein distance.
`mismatch`	numeric, distance value for a mismatch between symbols
`gap`	numeric, distance value for inserting a gap
`ignoreCase`	if `FALSE` (default), the distance measure will be case sensitive and if `TRUE`, case is ignored

Details

The function computes the Hamming and the Levenshtein (edit) distance of two given strings (sequences). The Hamming distance between two vectors is the number of mismatches between corresponding entries.

In case of the Hamming distance the two strings must have the same length.

In case of the Levenshtein (edit) distance a scoring and a trace-back matrix are computed and are saved as attributes `"ScoringMatrix"` and `"TraceBackMatrix"`. The numbers in the trace-back matrix reflect insertion of a gap in string `y` (1), match/mismatch (2), and insertion of a gap in string `x` (3).

The edit distance is useful, but normalizing the distance to fall within the interval `[0,1]` is preferred because it is somewhat difficult to judge whether an LD of for example 4 suggests a high or low degree of similarity. The method `"normlevenshtein"` for normalizing the LD is sensitive to this scenario. In this implementation, the Levenshtein distance is transformed to fall in this interval as follows:

$$lnd = 1 - \frac{ld}{max(length(x), length(y))}$$

where `ld` is the edit distance and `max(length(x), length(y))` denotes that we divide by the length of the larger of the two character strings. This normalization, referred to as the Levenshtein normalized distance (lnd), yields a statistic where 1 indicates perfect agreement between the two strings,

and a 0 denotes imperfect agreement. The closer a value is to 1, the more certain we can be that the character strings are the same; the closer to 0, the less certain.

Value

`strDist` returns an object of class `"dist"`; cf. `dist`.

Note

For distances between strings and for string alignments see also Bioconductor package **Biostrings**

Based on code by Matthias Kohl, adapted to conform to package standards.

References

R. Merkl and S. Waack (2009) *Bioinformatik Interaktiv*. Wiley.

Harold C. Doran (2010) *MiscPsycho. An R Package for Miscellaneous Psychometric Analyses*

See Also

`adist`, `dist`

string-overview for an overview of all string utilities in pharos.

Examples

```
x <- "GACGGATTATG"
y <- "GATCGGAATAG"
## Levenshtein distance
d <- strDist(x, y)
d
attr(d, "ScoringMatrix")
attr(d, "TraceBackMatrix")

## Hamming distance
strDist(x, y, method="hamming")
```

---

strExtract *Extract First Match from Strings*

Description

Extracts the first substring matching a regular expression from each element of a character vector.

Usage

```
strExtract(x, pattern, ...)
```

Arguments

`x`	a character vector
`pattern`	a character string containing a regular expression to match
`...`	additional arguments passed to `stri_extract_first_regex`

Details

This function is a thin wrapper around stri_extract_first_regex providing a simplified interface for extracting the first match of a pattern.

Value

A character vector containing the first match for each element of x. If no match is found, NA is returned.

See Also

stri_extract_first_regex, strExtractBetween

string-overview for an overview of all string utilities in pharos.

Examples

```
x <- c("abc123", "no digits", "456xyz")

# extract first number
strExtract(x, "\\d+")

# extract words
strExtract("hello world", "\\w+")
```

---

strExtractBetween *Extract Substrings Between Patterns*

Description

Extracts substrings that occur between two delimiters.

Usage

```
strExtractBetween(x, left, right, greedy = FALSE)
```

Arguments

x	a character vector
left	a character string or regular expression marking the left boundary
right	a character string or regular expression marking the right boundary
greedy	logical; if TRUE, the match is greedy (longest match). If FALSE (default), the match is non-greedy (shortest match).

Details

The function uses regular expressions to extract the first substring between left and right. Internally, it constructs a pattern of the form:

- greedy: left (.*) right
- non-greedy: left (.*?) right

Extraction is performed using stri_match_first_regex, which returns the first captured group.

Value

A character vector containing the extracted substrings. If no match is found, NA is returned for that element.

See Also

stri_match_first_regex, strExtract

string-overview for an overview of all string utilities in pharos.

Examples

```
x <- c("a[123]b", "x[abc]y", "no match")

# non-greedy (default)
strExtractBetween(x, "\\[", "\\]")

# greedy
strExtractBetween("a[1]b[2]c", "\\[", "\\]", greedy = TRUE)
```

---

string-overview *String Functions in pharos*

Description

pharos provides a family of functions for manipulating and inspecting character strings, built on top of **stringi** for Unicode-aware behavior. They fall into two groups:

Manipulation – transform, extract, or reshape string content:

Function	Purpose
strAbbr()	Abbreviate strings uniquely
strAlign()	Align strings
strCap()	Capitalize strings
strChop()	Split a string into a number of sections of defined length
strExtract()	Extract first match from strings
strExtractBetween()	Extract substrings between patterns
strLeft() / strRight()	Return the left or the right part of a string
strPad()	Pad a string with justification
strRev()	Reverse strings
strSpell()	Spell strings using phonetic alphabets
strSplit()	Split strings
strTrim()	Remove leading/trailing whitespace from a string
strTrunc()	Truncate strings and add ellipses if a string is truncated
strVal()	Extract numeric values from strings

Information – inspect properties of strings without changing them:

Function	Purpose
strCountW()	Count words in strings

strDist()	Compute distances between strings
strIsNumeric()	Check if character strings represent numeric values
strLen()	String length
strPos()	Find position of first occurrence of a string

See Also

base::substr()

---

strIsNumeric *Check if Character Strings Represent Numeric Values*

Description

Determines whether elements of a character vector represent valid numeric values. Unlike as.numeric(), this function performs a regex-based validation and does not produce warnings.

Usage

```
strIsNumeric(x, scientific = FALSE)
```

Arguments

x	a character vector to be tested
scientific	logical; if TRUE, scientific notation (e.g., "1e3") is allowed. Defaults to FALSE.

Details

The function uses regular expressions via **stringi** to validate numeric formats. Valid formats include optional leading sign (+ or -), optional decimal point, and digits. If scientific = TRUE, exponential notation using e or E is also supported.

Note that special values such as "Inf", "-Inf", and "NaN" are not considered numeric by this function.

Value

A logical vector of the same length as x, indicating whether each element represents a numeric value.

See Also

as.numeric

string-overview for an overview of all string utilities in pharos.

Examples

```
x <- c("123", "-3.141", "1e3", "foo", "Inf")

strIsNumeric(x)
strIsNumeric(x, scientific = TRUE)
```

strLeftRight *Returns the Left Or the Right Part Of a String*

Description

Returns the left part or the right part of a string. The number of characters are defined by the argument n. If n is negative, this number of characters will be cut off from the other side.

Usage

```
strLeft(x, n)

strRight(x, n)
```

Arguments

x	a vector of strings
n	a positive or a negative integer, the number of characters to cut. If n is negative, this number of characters will be cut off from the right with strLeft and from the left with strRight. n will be recycled.

Details

The functions strLeft and strRight are simple wrappers to substr.

Value

the left (right) n characters of x

See Also

string-overview for an overview of all string utilities in pharos.

Examples

```
strLeft("Hello world!", n=5)
strLeft("Hello world!", n=-5)

strRight("Hello world!", n=6)
strRight("Hello world!", n=-6)

strLeft(c("Lorem", "ipsum", "dolor","sit","amet"), n=2)

strRight(c("Lorem", "ipsum", "dolor","sit","amet"), n=c(2,3))
```

strLen *String length*

Description

Intuitive alias for nchar.

Usage

```
strLen(x, ...)
```

Arguments

x	a character vector
...	further arguments passed to nchar

See Also

nchar

string-overview for an overview of all string utilities in pharos.

strPad *Pad a String With Justification*

Description

strPad will fill a string x with defined characters to fit a given length.

Usage

```
strPad(x, width = NULL, pad = " ", adj = "left")
```

Arguments

x	a vector of strings to be padded
width	resulting width of padded string. If x is a vector and width is left to NULL, it will be set to the length of the largest string in x.
pad	string to pad with. Will be repeated as often as necessary. Default is " ".
adj	adjustment of the old string, one of "left", "right", "center". If set to "left" the old string will be adjusted on the left and the new characters will be filled in on the right side.

Details

If a string x has more characters than width, it will be chopped on the length of width.

Value

the string

See Also

strAlign, strTrunc

string-overview for an overview of all string utilities in pharos.

Examples

```
strPad("My string", 25, "XoX", "center")
 # [1] "XoXXoXXoMy stringXXoXXoXX"
```

---

strPos *Find Position of First Occurrence Of a String*

Description

Returns the numeric position of the first occurrence of a substring within a string. If the search string is not found, the result will be NA.

Usage

```
strPos(x, pattern, pos = 1)
```

Arguments

x	a character vector in which to search for the pattern, or an object which can be coerced by as.character to a character vector
pattern	character string (search string) containing the pattern to be matched in the given character vector. This can be a character string or a regular expression.
pos	integer, defining the start position for the search within x. The result will then be relative to the begin of the truncated string. Will be recycled.

Details

This is just a wrapper for the function regexpr.

Value

a vector of the first position of pattern in x

See Also

strChop, regexpr

string-overview for an overview of all string utilities in pharos.

Examples

```
strPos(x=rownames(mtcars), pattern="y")

# first t, starting with position 4
strPos(x=rownames(mtcars), pattern="t", pos=4)
```

strRev *Reverse Strings*

Description

Reverses each element of a character vector.

Usage

```
strRev(x)
```

Arguments

x	a character vector

Details

This function uses stri_reverse, which correctly handles Unicode characters and multi-byte encodings.

Value

A character vector with each string reversed.

See Also

stri_reverse

string-overview for an overview of all string utilities in pharos.

Examples

```
strRev("abc")

strRev(c("hello", "world"))

# Unicode-safe
strRev("äöü")
```

strSpell *Spell Strings Using Phonetic Alphabets*

Description

Converts characters in a string into their corresponding phonetic representations using either the NATO phonetic alphabet or Morse code.

Usage

```
strSpell(x, upr = "CAP", type = c("NATO", "Morse"))
```

Arguments

x	a character vector (typically of length 1)
upr	character string used as a prefix for uppercase letters. Default is "CAP". If NA, no prefix is added.
type	character string specifying the encoding system: • "NATO": NATO phonetic alphabet (default) • "Morse": Morse code

Details

Letters (A–Z, a–z) and digits (0–9) are mapped to their corresponding phonetic representations. Other characters are returned unchanged.

For type = "NATO", uppercase letters can optionally be prefixed (e.g., "CAP Alfa") to distinguish them from lowercase letters.

The function uses Unicode-aware character splitting via stri_split_boundaries.

Value

A character vector with each character of x replaced by its phonetic representation.

See Also

strTrim, stri_split_boundaries

string-overview for an overview of all string utilities in pharos.

Examples

```
# NATO spelling
strSpell("ABC")

# with digits
strSpell("A1B2")

# Morse code
strSpell("SOS", type = "Morse")

# without uppercase prefix
strSpell("ABC", upr = NA)
```

---

strSplit *Split Strings*

Description

Splits character vectors into substrings. This is a convenience wrapper around stri_split_fixed and stri_split_regex with simplified defaults.

Usage

```
strSplit(x, split = "", fixed = FALSE)
```

Arguments

x	a character vector to be split
split	a character string specifying the delimiter (if fixed = TRUE) or a regular expression (if fixed = FALSE)
fixed	logical; if TRUE, split is treated as a fixed string. Otherwise, it is interpreted as a regular expression.

Details

If the input x has length 1, the result is returned as a character vector instead of a list for convenience.

This function provides a simplified interface to string splitting using the **stringi** package. It avoids some of the complexity of strsplit while providing consistent and Unicode-aware behavior.

Value

A list of character vectors. If x has length 1, a character vector is returned.

See Also

stri_split_fixed, stri_split_regex, strsplit

string-overview for an overview of all string utilities in pharos.

Examples

```
strSplit("a,b,c", ",", fixed = TRUE)

strSplit(c("a b", "c d"), " ")

# regex splitting
strSplit("a1b2c3", "\\d")
```

---

strTrim *Remove Leading/Trailing Whitespace From A String*

Description

The function removes whitespace characters as spaces, tabs and newlines from the beginning and end of the supplied string. Whitespace characters occurring in the middle of the string are retained. Trimming with method "left" deletes only leading whitespaces, "right" only trailing. Designed for users who were socialized by SQL.

Usage

```
strTrim(x, pattern = " \t\n", method = "both")
```

Arguments

x	the string to be trimmed
pattern	the pattern of the whitespaces to be deleted, defaults to space, tab and newline: " \t\n"
method	one out of "both" (default), "left", "right". Determines on which side the string should be trimmed.

Details

The functions are defined depending on method as
both: gsub(pattern=gettextf("^[%s]+|[%s]+$", pattern, pattern), replacement="",x=x)
left: gsub(pattern=gettextf("^[%s]+",pattern),replacement="", x=x)
right: gsub(pattern=gettextf("[%s]+$",pattern), replacement="", x=x)

Value

the string x without whitespaces

See Also

trimws

string-overview for an overview of all string utilities in pharos.

Examples

```
strTrim("  Hello world! ")

strTrim("  Hello world! ", method="left")
strTrim("  Hello world! ", method="right")

# user defined pattern
strTrim(" ..Hello ... world! ", pattern=" \\.")
```

---

strTrunc *Truncate Strings and Add Ellipses If a String is Truncated.*

Description

Truncates one or more strings to a specified length, adding an ellipsis (...) to those strings that have been truncated. The truncation can also be performed using word boundaries. Use strAlign() to justify the strings if needed.

Usage

```
strTrunc(x, maxlen = 20, ellipsis = "...", wbound = FALSE)
```

Arguments

x	a vector of strings
maxlen	the maximum length of the returned strings (NOT counting the appended ellipsis). maxlen is recycled.
ellipsis	the string to be appended, if the string is longer than the given maximal length. The default is "...".
wbound	logical. Determines if the maximal length should be reduced to the next smaller word boundary and so words are not chopped. Default is FALSE.

Value

The string(s) passed as 'x' now with a maximum length of 'maxlen' + 3 (for the ellipsis).

See Also

strAlign, strPad

string-overview for an overview of all string utilities in pharos.

Examples

```
x <- c("this is short", "and this is a longer text",
       "whereas this is a much longer story, which could not be told shorter")

# simple truncation on 10 characters
strTrunc(x, maxlen=10)

# NAs remain NA
strTrunc(c(x, NA_character_), maxlen=15, wbound=TRUE)

# using word boundaries
for(i in 0:20)
  print(strTrunc(x, maxlen=i, wbound=TRUE))

# compare
for(i in 0:20)
  print(strTrunc(x, maxlen=i, wbound=FALSE))
```

---

strVal *Extract Numeric Values from Strings*

Description

Extracts numeric values from character strings using regular expressions. The function supports integers, decimal numbers, and scientific notation.

Usage

```
strVal(
  x,
  collapse = FALSE,
  output = c("character", "numeric"),
  dec = getOption("OutDec")
)
```

Arguments

x	a character vector
collapse	logical; if TRUE, all extracted numbers per element of x are concatenated into a single string. Otherwise, a list of character vectors is returned.
output	character; controls the type of the returned values. "character" (default) returns strings; "numeric" coerces extracted values to numeric.
dec	character; decimal separator used in the input. Defaults to getOption("OutDec").

Details

The function detects numeric values including optional signs, decimal parts, and scientific notation (e.g., "1.23e-4").

Whitespace between sign and number (e.g., "- 2.5") is tolerated and removed before returning results.

If output = "numeric" and the decimal separator dec differs from the current system setting (getOption("OutDec")), values are converted accordingly before coercion.

Value

If collapse = FALSE, a list of character or numeric vectors containing the extracted values for each element of x. If collapse = TRUE, a character or numeric vector with concatenated values per element.

See Also

as.numeric, stri_extract_all_regex

string-overview for an overview of all string utilities in pharos.

Examples

```
x <- c("value = 12.5", "x = -3.2e2 and 7", "no numbers here")

# extract as character
strVal(x)

# extract as numeric
strVal(x, output = "numeric")

# concatenate values
strVal(x, collapse = TRUE)

# different decimal separator
strVal("value = 3,14", dec = ",", output = "numeric")
```

style *Format Styles*

Description

Interface for format templates, defined as a list consisting of any accepted format features in `fm()`. This enables to define templates globally and easily change or modify them later.

Usage

```
styles()

style(
  x,
  digits = NULL,
  leadDigits = NULL,
  sci = NULL,
  bigMark = NULL,
  decMark = NULL,
  naForm = NULL,
  zeroForm = NULL,
  fmt = NULL,
  pThreshold = NULL,
  width = NULL,
  align = NULL,
  lang = NULL,
  label = NULL,
  ...
)

## S3 method for class 'Style'
print(x, ...)
```

Arguments

`x`	an object of class `Style` or a the name of a style, defined either in the global enviroment or in the options.
`digits`	integer, the desired (fixed) number of digits after the decimal point. Unlike `formatC` you will always get this number of digits even if the last digit is 0. Negative numbers of digits round to a power of ten (`digits=-2` would round to the nearest hundred).
`leadDigits`	number of leading zeros. `leadDigits=3` would make sure that at least 3 digits on the left side will be printed, say `3.4` will be printed as `003.4`. Setting `leadDigits` to `0` will yield results like `.452` for `0.452`. The default `NULL` will leave the numbers as they are (meaning at least one 0 digit).
`sci`	integer. The power of 10 to be set when deciding to print numeric values in exponential notation. Fixed notation will be preferred unless the number is larger than 10^scipen. If just one value is set it will be used for the left border 10^(-scipen) as well as for the right one (10^scipen). A negative and a positive value can also be set independently. Default is `getOption("scipen")`, whereas `scipen=0` is overridden.

bigMark	character; if not empty used as mark between every 3 decimals before the decimal point. Default is "" (none).
decMark	character, specifying the decimal mark to be used. If not provided, the default set as decMark option is used.
naForm	character, string specifying how NAs should be specially formatted. If set to NULL (default) no special action will be taken.
zeroForm	character, string specifying how zeros should be specially formatted. Useful for pretty printing 'sparse' objects. If set to NULL (default) no special action will be taken.
fmt	either a format string, allowing to flexibly define special formats or an object of class style, consisting of a list of fdm arguments. See Details.
pThreshold	a numerical tolerance used mainly for formatting p values, those less than pThreshold are formatted as "\code{< [pThreshold]}" (where '[pThreshold]' stands for format(pThreshold, digits)). Default is 0.001.
width	integer, the defined fixed width of the strings.
align	the character on whose position the strings will be aligned. Left alignment can be requested by setting sep = "\l", right alignment by "\r" and center alignment by "\c". Mind the backslashes, as if they are omitted, strings would be aligned to the **character** l, r or c respectively. The default is NULL which would just leave the strings as they are. This argument is send directly to the function strAlign() as argument sep.
lang	optional value setting the language for the months and daynames. Can be either "local" for current locale or "en" for english. If left to NULL, the DescToolsOption "lang" will be searched for and if not found "local" will be taken as default.
label	a description for the style
...	further arguments to be passed to or from methods.

Details

style() can either create new styles or edit existing ones. style() can be used to create new styles. It takes any of the arguments from fm() and combines them to an object of class "Style", which then can be handed over to fm() as argument fmt.
Following will define a new format template named "num.sty". Passed to fm() this will result in a number displayed with 2 fixed digits and a comma as big mark:

```
num.sty <- style(digits=2, bigMark=",")
fm(12222.89345, fmt=num.sty) = 12,222.89
```

This is the same result as if the arguments would have been supplied directly, but helps to avoid boilerplate code:
fm(12222.89345,digits=2, bigMark=",").

To edit a style we can provide style() with its name and overwrite, resp. add new format options. style("num.sty", digits=1, sci=10) will use the current version of the numeric format and change the digits to 1 and the threshold to switch to scientifc presentation to numbers > 1e10 and < 1e-10.

styles() returns all found style definitions in the global environment or in the options.

The styles can be stored as options for convenience. To store a new format we use the default options() approach: options(num.sty = style(digits=1, bigMark=" ")) Defined styles in the options can be passed on to fm() simply by their name.

Many report functions (e.g. `DescToolsX::tOne()`) in **DescToolsX** use three default formats for counts (named `"abs.sty"`), numeric values (`"num.sty"`) and percentages (`"per.sty"`).

Value

`style()` returns an object of class `Style`
`styles()` returns a list of styles

See Also

theme

Other format: `convUnit()`, `fm()`, `fmCI()`, `print.Unit()`, `unit()`

Examples

```
# use style() to get and define new formats stored as option
num.sty <- style(digits=2, bigMark=" ")
abs.sty <- style(digits=0, bigMark=" ")
dat.sty <- style(fmt="MM, dd yyyy")

num.sty                             # displays the details of the style
# editing styles
style("abs.sty")                    # looks for format "abs.sty"
# style("nexist")                     # return for nonexisting style
style("abs.sty", bigMark="")       # get Style("abs") and overwrite bigMark
style("abs.sty", naForm="-")       # get Style("abs") and add user defined naForm

styles()                            # all defined formats
styles()[c("num.sty", "abs.sty")]   # numeric and integer styles

# define totally new format and store as option
options(nob.sty=style(digits=5, naForm="nodat"))

# using styles
fm(314.1563, fmt=abs.sty)
fm(314.1563, fmt=num.sty)

fm(Sys.Date(), fmt=dat.sty)
```

---

textLegend *Direct Labels in the Right Margin*

Description

Draw a legend in the right margin consisting of short line segments and text labels, placed next to the (typically last) values of the plotted series. This labels lines directly instead of using a boxed `legend()`, as commonly preferred for time series and profile plots.

Usage

```
textLegend(
  y,
  labels = names(y),
  line = c(1, 1),
  width = 1,
  col = par("fg"),
  lty = par("lty"),
  lwd = par("lwd"),
  cex = par("cex"),
  main = NULL,
  mindist = NULL
)
```

Arguments

y	numeric vector with the vertical anchor positions in user coordinates, typically the last observed value of each series, in series (column) order. Sorting and collision handling are done internally.
labels	character vector of labels, parallel to y. Defaults to names(y), or the series index if y is unnamed.
line	numeric vector of length 1 or 2 (recycled), in margin lines of side 4. The first element is the offset of the segments from the plot region, the second the gap between segments and labels.
width	length of the line segments in margin lines. Set to NA to suppress the segments; the labels are then placed directly at line[1].
col, lty, lwd	color, line type and width of the segments, parallel to y (in series order, recycled). Ignored if width is NA.
cex	character expansion for the labels. Also enters the default vertical spacing, so larger text automatically gets wider spacing.
main	optional title for the legend, drawn at the top of the plot region. Default is NULL (none).
mindist	minimal vertical distance between labels in user coordinates, passed to spreadOut(). Default is 1.2 * strheight("M") * cex.

Details

The function owns the legend geometry: the anchor positions y are sorted internally so the legend follows the vertical order of the lines, the graphical parameters are recycled accordingly, and vertical label collisions are resolved via spreadOut(). Callers therefore simply pass the values in series order, parallel to col, lty and lwd.

Drawing uses mtext() and segments() in the device margin; xpd is set to TRUE and restored on exit. Horizontal positions are given in margin lines of side 4 and converted to user coordinates via lineToUser().

Make sure the right margin is wide enough for segments and labels, e.g. via the mar argument of the calling plot function.

Value

The (sorted and spread) vertical label positions, invisibly. Useful for adding further annotation next to the labels.

See Also

plotLines, mtext, segments, legend

Other graphics.annotation: barText(), boxedText(), colLegend(), errBars(), stamp(), titleRect()

Examples

```
m <- EuStockMarkets[seq(1, 1860, 10), ]
op <- par(mar = c(5, 4, 4, 8))
matplot(m, type = "l", lty = 1, lwd = 2, col = 1:4, las = 1,
        main = "EU Stock Markets")
textLegend(y = m[nrow(m), ], labels = colnames(m),
           col = 1:4, lwd = 2, main = "Index")
par(op)
```

---

theme *pharos's Graphics and Formatting Theme*

Description

getTheme(), setTheme(), and resetTheme() are the user-facing entry points to pharos's theme system: a single, named list of graphical and formatting defaults, consulted by essentially every plotting function in the package (and by fm() for numeric/percentage/p-value formatting) whenever the corresponding function argument is left at its default value.

Usage

```
getTheme()

setTheme(theme)

resetTheme()
```

Arguments

theme	either a named list of theme components to merge into the active theme (only the given top-level elements are replaced; e.g. setTheme(list(palette = "Set2")) changes only the palette, leaving grid, box, twin, etc. untouched), or a single string naming a preset registered in the (currently empty) preset registry.

Value

getTheme() and resetTheme() return the (new) active theme, a named list; setTheme() returns the new active theme as well, invisibly.

What the theme is for

Most graphical parameters in pharos's plotting functions (col, grid, box, pch, ...) default to a sentinel value, .useTheme, rather than to a hardcoded color or number. At call time, that sentinel is resolved against getTheme() - so changing the active theme changes the look of *every* plot produced afterwards that didn't explicitly override that argument, without touching a single plotting function's code.

This gives three independent ways to control a plot's appearance, in order of precedence (highest first):

1. **Explicit argument**, e.g. `plotXY(x, y, col = "red")` - always wins, for that one call only.
2. **Function-specific default** - some functions deliberately hardcode a value that differs from the generic theme baseline instead of using `.useTheme` (e.g. `plotBar()`'s `box` defaults to `FALSE` rather than the theme's box setting, and `plotDot()`'s grid line color/style stays its own orange/grey dashed look regardless of the active theme). This is a conscious per-function design choice, not a bug - see the individual function's documentation for which arguments opt out of theme-following this way.
3. **Active theme** (`getTheme()`) - the package-wide fallback used whenever neither of the above applies.

Structure of the theme

The theme is a named list with the following top-level components. Nested components (`grid`, `box`, `points`, `bar`, `sty`) are themselves named lists; `setTheme()` replaces them wholesale (it does not merge one level deeper), so to change a single nested value, supply the complete sub-list, e.g. `setTheme(list(grid = list(col = "red", lwd = 1, lty = "dotted")))`.

`par` `col.axis="grey40"`, `las=1`, `cex=1.1`. Global `par()` pass applied by every `.applyParFromDots()` call (axis label color, axis label orientation, global text scaling).

`grid` `col="grey80"`, `lwd=1`, `lty="dotted"`, plus `group.*` variants. Background grid lines, via `.drawGrid()`. The `group.*` entries style a subordinate/secondary grid (e.g. group separators) where a function draws one.

`box` `col="grey50"`, `lwd=1`, `lty="solid"`. The frame drawn around a plot region, via `.drawBox()`.

`points` `pch=21`, `col="grey50"`, `bg=addOpacity("grey")`, `cex=1.1`. Default point styling for scatterplot-like functions (e.g. `plotXY()`, `plotDot()`).

`twin` `pal("helsana")[c(6, 1)]`. A fixed pair of colors for contexts that inherently need exactly two contrasting colors (e.g. a fit line and a smoother in `plotXY()`, the two poles of a diverging color ramp in `plotCor()`, a single accent color via `twin[1]` in `lines.loess`/`plotQQ()`'s confidence band). Never used as a substitute for `palette` when more than two colors are needed.

`palette` `"helsana"`. Name of the qualitative (categorical) palette used whenever more than two unordered colors are needed (e.g. `plotMosaic()`'s fill colors), resolved via `pal()`. Deliberately not used for sequential or diverging numeric scales – see the next item.

(none – by design) Sequential/diverging numeric color scales (e.g. `plotDens2D()`'s density heatmap, `plotHeatmap()`'s cell shading) are deliberately *not* theme-driven; they use a hardcoded, purpose-built palette via `pal()` instead (e.g. `pal("red-black")`, `pal("Blues")`). Neither `palette` (categorical) nor `twin` (a fixed pair) is the right semantic fit for an ordered, continuous scale – see `pal` for the registry of named continuous palettes.

`bar` `col="grey80"`, `border=NA`. Default bar fill/border in `plotBar()`.

`sty` `abs="abs.sty"`, `perc="per.sty"`, `num="num.sty"`, `pval="pval.sty"`. Names of `fm()` format styles (see `styles()`) used for absolute counts, percentages, plain numbers, and p-values respectively.

`stamp` `expression(...)` – unevaluated. The corner stamp text drawn by every `stamp()`/`.withGraphicsState()` call. Stored as an unevaluated `expression()` and `eval()`'d at draw time (not at theme-load or theme-set time), so it always reflects the current user and date rather than freezing whatever they were when the theme was defined or last changed. Default: `"<username> / <YYYY-MM-DD>"`.

The .useTheme sentinel

Internally, a function argument that should follow the active theme (rather than some fixed value) defaults to a dedicated sentinel object, .useTheme, not to TRUE, FALSE, NA, or NULL. Those four are frequently legitimate, explicit values in their own right (e.g. grid = NULL commonly means "suppress the grid" for a given function) - using any of them to also mean "no value was given, defer to the theme" would make that case ambiguous. A dedicated sentinel avoids the ambiguity entirely and keeps the resolution logic a single, explicit equality check (identical(x, .useTheme)) rather than an implicit, error-prone guess based on data type.

Two small internal helpers resolve it:

- .useThemeValue(value, ...) - for a simple value taken from a nested theme key, e.g. col <- .useThemeValue(col, "points", "col").
- .resolveToggle(spec, themeValue) - for an on/off-style argument (such as grid/box) that may also be a TRUE/FALSE/NA/NULL/list spec in its own right, used internally by .drawGrid()/.drawBox().

Some theme values require more than a simple key lookup to resolve (e.g. building a color ramp from twin, or constructing a list of point parameters from points); those are resolved with a small inline identical(x, .useTheme) check directly in the consuming function rather than forcing them through one of the two generic helpers above. See e.g. plotCor()'s col argument or plotDot()'s pch argument for worked examples.

How plotting functions consume the theme

.applyParFromDots() Applies getTheme()$par as the lowest-precedence par() pass, before the function's own defaults and the user's ....

.drawGrid(grid, defaults = list()) / .drawBox(box, defaults = list()) Generic dispatchers wrapping graphics::grid()/ graphics::box(). Resolve the .useTheme sentinel, merge the theme's style values with any function-specific defaults (e.g. plotBar() suppressing the axis-parallel grid direction via nx/ny), and dispatch via callIf. **Not** used by every function that draws a grid or frame: a few (plotCor(), plotHeatmap(), plotDot()) have grid/box geometry tied to exact data coordinates (e.g. half-integer cell boundaries) that doesn't match graphics::grid()'s axis-tick-based geometry: these resolve theme values directly via getTheme()$grid/ getTheme()$box but draw with their own clipped rect()/abline() calls instead.

.withGraphicsState(expr, stamp = .useTheme, ...) Wraps a plotting function's body, restores par() afterwards, and - after expr has run successfully - calls stamp() with the (possibly theme-resolved) corner stamp text/arguments.

Presets

setTheme() also accepts a single preset name (a string) instead of a list, looked up in an internal preset registry (.themePresets). No presets are currently registered; the registry exists so named, complete theme variants (e.g. a monochrome or high-contrast theme) can be added later without changing the setTheme() interface.

See Also

pal for the color palette registry, fm for the formatting styles referenced by sty, stamp for the corner stamp mechanism.

Examples

```
# inspect the active theme
getTheme()

# change only the qualitative palette for the rest of the session
setTheme(list(palette = "Set2"))

# change the accent color pair used for e.g. lm/loess fit lines
setTheme(list(twin = c("firebrick", "navy")))

# turn grid lines off package-wide (any .useTheme-driven grid argument
# everywhere now resolves to "off", unless a function overrides it)
setTheme(list(grid = FALSE))

# back to package defaults
resetTheme()
```

`titleRect` *Plot Boxed Annotation*

Description

The function can be used to add a title to a plot surrounded by a rectangular box. This is useful for plotting several plots in narrow distances.

Usage

```
titleRect(
  label,
  bg = "grey",
  border = 1,
  col = "black",
  xjust = 0.5,
  line = 2,
  ...
)
```

Arguments

`label`	the main title
`bg`	the background color of the box.
`border`	the border color of the box
`col`	the font color of the title
`xjust`	the x-justification of the text. This can be `c(0, 0.5,1)` for left, middle- and right alignement.
`line`	on which MARgin line, starting at 0 counting outwards
`...`	the dots are passed to the `text` function, which can be used to change font and similar arguments.

Value

nothing is returned

See Also

title

Other graphics.annotation: barText(), boxedText(), colLegend(), errBars(), stamp(), textLegend()

Examples

```
plot(pressure)
titleRect("pressure")
```

---

toHtmlTable *Render a matrix as an HTML table*

Description

Converts a matrix (or vector) to a <table> HTML fragment, with optional row/column headers, caption, per-column alignment and widths. The result has class c("html", "character") (see as.html) and prints as a formatted text table via preview.html.

Usage

```
toHtmlTable(
  m,
  sepCol = FALSE,
  caption = "",
  bodyAlign = "center",
  valign = "top",
  width = NULL,
  cellpadding = 3,
  border = 0,
  tableWidth = NA,
  captionAlign = "center",
  frame = TRUE,
  rowNames = TRUE,
  colNames = TRUE
)
```

Arguments

m	a matrix or vector
sepCol	logical; if TRUE, insert a narrow empty separator column between each pair of columns
caption	table caption text
bodyAlign	horizontal alignment of body cells ("left", "center", "right"), recycled to the number of columns

valign	vertical alignment of body cells (HTML valign attribute: "top", "middle", "bottom"), recycled to the number of columns
width	column width(s) (HTML width attribute), recycled to the number of columns including an optional rowname column; use NA for columns without an explicit width
cellpadding	HTML cellpadding attribute
border	HTML border attribute
tableWidth	overall table width (HTML width attribute on <table>), or NA for none
captionAlign	horizontal alignment of the header row cells
frame	logical; if TRUE, draw outer frame and group rules (frame="hsides" rules="groups")
rowNames	logical; render the row names as a leading header column. Ignored when m has none
colNames	logical; render the column names as a header row. Ignored when m has none

Value

an object of class c("html", "character")

See Also

bedrock::appendEnum

Other html: as.fileLink(), as.html(), as.img(), embedFile(), escapeHtml(), htmlNotation, htmlSubscript

---

transformXY *Apply Geometric Transformations to Coordinates*

Description

Applies scaling, rotation, and translation to a set of 2D coordinates. The transformations are applied in the following order:

1. Scaling
2. Rotation (see rotate)
3. Translation

Usage

```
transformXY(
  x,
  y = NULL,
  translate = c(0, 0),
  scale = c(1, 1),
  theta = 0,
  asp = 1
)
```

Arguments

x	numeric vector of x coordinates, or an object coercible by xy.coords.
y	numeric vector of y coordinates. Ignored if x already contains both coordinates.
translate	numeric vector of length 1 or 2 specifying translation in x and y direction. Recycled if necessary. Default is c(0, 0).
scale	numeric vector of length 1 or 2 specifying scaling factors for x and y. Recycled if necessary. Default is c(1, 1).
theta	rotation angle in radians. Default is 0.
asp	aspect ratio adjustment passed to rotate. Default is 1.

Details

This function is a convenience wrapper combining basic affine transformations. Internally, it uses rotate for rotation.

Value

A list with components x and y, as returned by xy.coords.

See Also

Other geometry.transformation: rotate()

Examples

```
x <- c(0, 1, 1, 0)
y <- c(0, 0, 1, 1)

# scale, rotate and translate a square
transformXY(x, y,
            scale = 2,
            theta = pi / 4,
            translate = c(1, 1))

# matrix input
m <- cbind(x, y)
transformXY(m, scale = 0.5, theta = pi/6)
```

---

unit *Get or Set Unit Attribute*

Description

Retrieves or assigns a unit attribute to an R object.

Usage

```
unit(x)

unit(x) <- value
```

Arguments

x	an R object.
value	A single character string specifying the unit, or NULL to remove the unit attribute.

Details

The unit is stored as a simple character string in the "unit" attribute. This provides a lightweight mechanism for attaching unit metadata to objects, without enforcing any automatic conversion or validation.

The setter does not validate whether the unit is physically meaningful. Validation and conversion should be handled externally (e.g., via a unit conversion engine such as ConvUnit7).

Assigning NULL removes the unit attribute.

Value

- unit(x) returns the unit as a character string or NULL.
- unit(x) <- value returns x with updated unit attribute.

See Also

Other format: convUnit(), fm(), fmCI(), print.Unit(), style()

Examples

```
x <- 10
unit(x) <- "m"
unit(x)

y <- 5
unit(y) <- "kg"

# remove unit
unit(y) <- NULL
```

Index